Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное
образовательное учреждение
высшего образования
Тульский государственный университет

Кафедра
Информационная безопасность

МЕТОДИЧЕСКИЕ УКАЗАНАИЯ К ЛАБОРАТОРНЫМ РАБОТАМ

по дисциплине

Базы данных и их безопасность

Разработчики:
к.т.н., доц. Баранов А.Н., к.т.н., доц. Баранова Е.М.

Тула, 2026

Лабораторная работа №1
Установка среды программирования, синтаксис и простейшие программы на языке Python

1.1. Цель лабораторной работы

Изучение синтаксиса и написание простейших программ на языке Python

1.2 Теоретические положения

Установка среды программирования

Можно сказать, что язык программирования Python практически ничем не ограничен, поэтому может быть использован для создания крупнейших проектов, в основу которых положены принципы обработки информации, хранящейся в файлах, созданных, например, в электронных таблицах или системах управления базами данных (СУБД).

Сегодня Python используется, например, разработчиками Google и Yandex. Основные характеристики языка Python – это простота и универсальность.

Среда программирования на языке Python называется IDLE (Integrated Development and Learning Environment). IDLE представляет собой интегрированную среду разработки и обучения на языке Python (программу для написания проектов на языке Python), созданную с помощью библиотеки Tkinter. Программа является бесплатной и ее можно скачать по ссылке: https://python.org/downloads/windows/.

Для установки программы под операционную систему Windows необходимо выбрать Latest Python 3 Release – Python 3.4.1 (версия может быть и другая). Затем необходимо выбрать download page. Откроется список файлов, которые можно загрузить.

Если операционная система Windows 32-х битная, то потребуется файл Windows x86 MSI installer. Если операционная система Windows 64-х битная, то потребуется файл Windows x86-64 MSI installer. После этого открывается загрузившийся файл. Файл подписан Python Software Foundation. Далее следует нажать на кнопку Выполнить и следовать инструкции по установке. Компоненты программы можно оставить те, которые предложены по умолчанию.

Для установки программы под операционную систему Linux системы (Ubuntu, Linux Mint и другие) необходимо проверить, установлен ли Python по умолчанию. Для этого в консоли (вызов консоли часто ctrl+alt+t) необходимо набрать команду: python3. Если Python установлен, то консоль выдаст сообщение о том, какая версия программы установлена.

Если Python не установлен, то в консоли необходимо набрать команду: sudo apt-get install python3, после чего будет установлен пакет Python 3.

Также установить пакет Python 3 можно через Mintinstaller/Synaptic/центр приложений Ubuntu.

В Python для Linux нет предустановленной среды IDLE. Для установки этой среды потребуется установить пакет Idle3 или Python3-idle.

Установка среды IDLE под Linux необязательна, так как программы можно писать в текстовом редакторе и запускать их через консоль.

Запуск среды

После запуска среды IDLE, откроется окно для написания текстов программ на языке Python (рис.1.1).

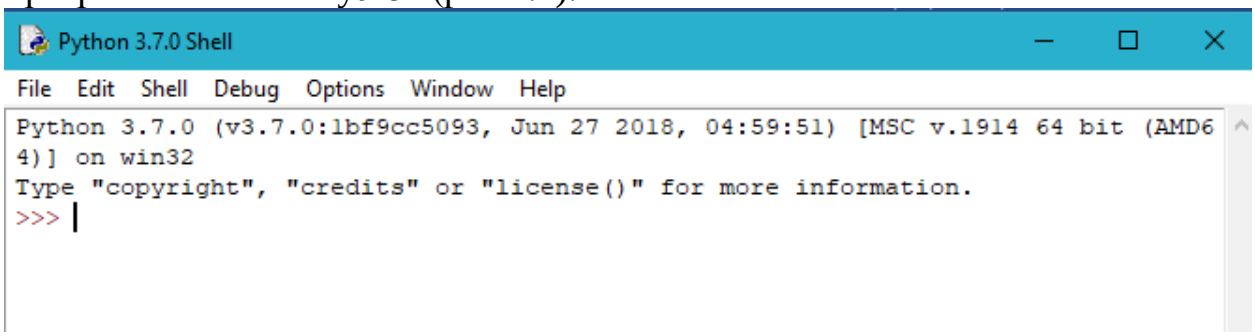


Рисунок 1.1 – Окно для написания текстов программ на языке Python (интерактивный режим)

Первый запуск среды IDLE позволяет писать программы в интерактивном режиме, без сохранения файла.

Для перехода в основной режим необходимо выбрать меню File/New File или использовать для этого сочетание клавиш Ctrl + N.

Синтаксис языка Python

При написании программ на языке Python следует пользоваться правилами:

1. Конец строки является концом инструкции (точка с запятой не требуется).

2. Вложенные инструкции объединяются в блоки по величине отступов. Отступ может быть любым, главное, чтобы в пределах одного вложенного блока отступ был одинаков. Целесообразно использовать несколько пробелов или знак табуляции.

3. Вложенные инструкции в Python записываются в соответствии с одним и тем же шаблоном, когда основная инструкция завершается двоеточием, вслед за которым располагается вложенный блок кода, обычно с отступом под строкой основной инструкции, например,

if x > y:

print(x).

В примере *if x > y:* – основная инструкция, *print(x)* – вложенный блок инструкций.

4. Можно записать несколько инструкций в одной строке, разделяя их точкой с запятой, например,

a = 1; b = 2; print(a, b),

однако этот случай подходит для небольших программ, если структура программы велика, это может привести не к читаемости кода.

5. Допустимо записывать одну инструкцию в нескольких строках. Достаточно ее заключить в пару круглых, квадратных или фигурных скобок, например,

if (a =1 and b =2 and

c =3 and d = 4):

print('d').

6. Тело составной инструкции может располагаться в той же строке, что и тело основной, если тело составной инструкции не содержит составных инструкций, например,

if x > y: print(x).

В примере if x > y – тело составной инструкции, print(x) – тело основной инструкции, : - разделение инструкций, записанных в одной строке.

Простейшие операторы и программы языка Python

- input – оператор ввода;
- print – оператор вывода;
- инструкция if-elif-else – оператор условия, причем сначала записывается часть if с условным выражением, далее могут следовать одна или более необязательных частей elif, и, наконец, необязательная часть else (часть elif может заменять конструкции switch – case of в других языках программирования).
- цикл for проходится по любому объекту (например, массиву, строке или списку), и во время каждого прохода выполняет тело цикла;
- оператор break досрочно прерывает цикл;
- оператор continue начинает следующий проход цикла, минуя оставшееся тело цикла.

Пример 1

Написать программу, которая каждую букву в заданной фразе Hello word будет сравнивать с введенной с клавиатурой буквой. При совпадении букв цикл будет прерываться, если буквы не совпадают, то будет выводиться фраза, указывающая на это.

Выполнение примера 1

На рисунке 1.2 представлен код программы к примеру 1.

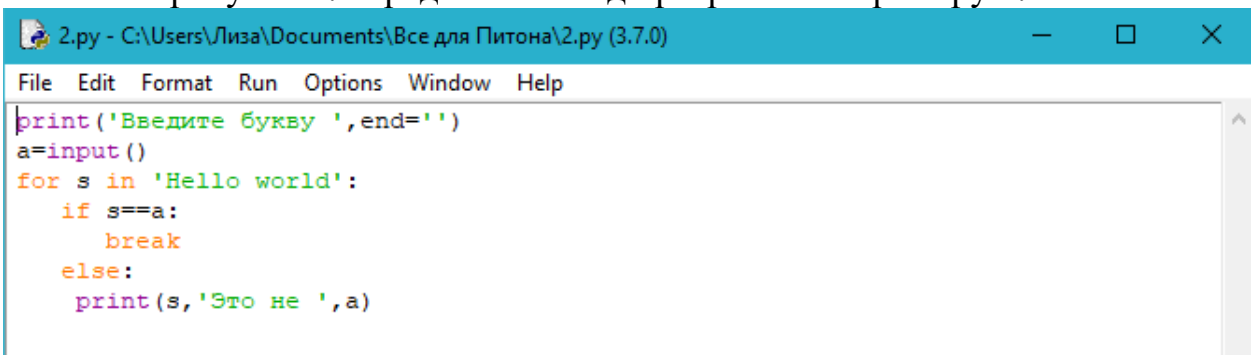


Рисунок 1.2 – Код программы к примеру 1

В представленном коде:

print (‘Введите букву’, end= ‘’) – вывод на экран фразы Введите букву, при этом end= ‘’ означает, что курсор остается в текущей строке.

a=input () – присвоение символу a введенной буквы или ввод буквы в память;

for s in ‘Hello word’: if s==a: – начало цикла проверки совпадения каждой буквы фразы Hello word с введенным ранее символом, хранящимся в переменной a;

break – прерывание цикла (если буквы совпадут);

else: print (s, ‘Это не’, a) – иначе вывести запись, что текущая буква фразы Hello word это не буква, введенная с клавиатуры.

Запуск программы в среде IDLE осуществляется клавишей F5 или командой Run/Run Modul. Перед запуском файл с программой следует сохранить, если файл не сохранялся, то среда IDLE задаст вопрос о сохранении файла.

Результаты выполнения программы в соответствии с примером 1 представлены на рисунках 1.3а и 1.3б.

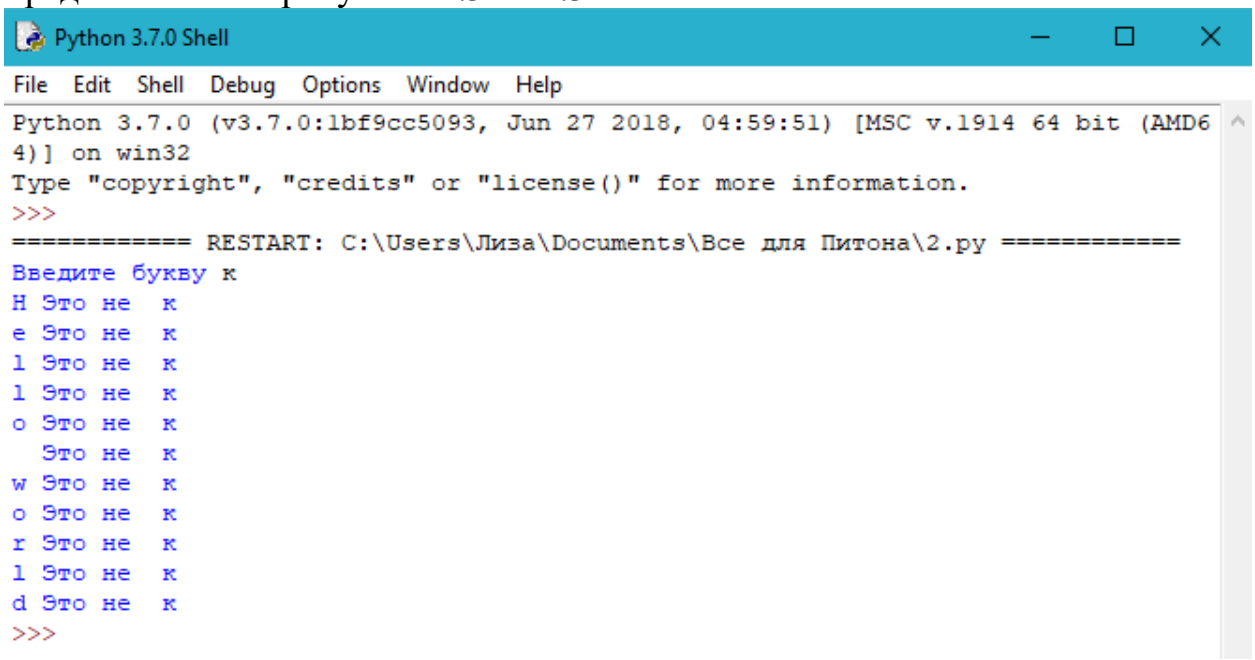


Рисунок 1.3а – Результат работы программы в соответствии с примером 1 (в заданной фразе введенная буква не обнаружена)

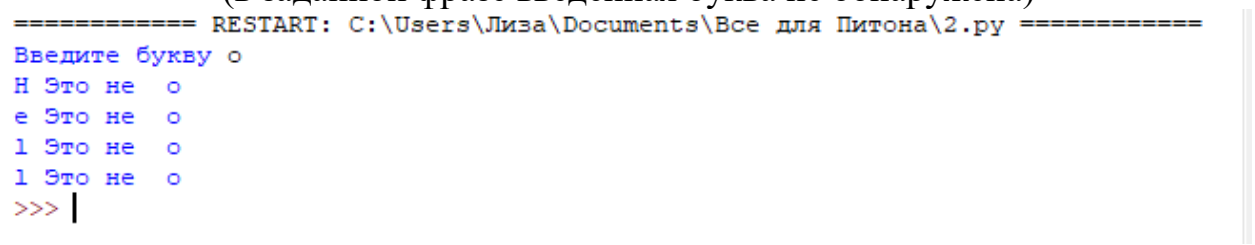


Рисунок 1.3б – Результат работы программы в соответствии с примером 1 (в заданной фразе введенная буква обнаружена и цикл прерван, так как следующая буква в фразе – это буква о)

Пример 2

Написать программу, которая из заданного слова выведет букву, соответствующую заданному индексу.

Выполнение примера 2

Следует отметить, что при переборе символов слева направо, цикл начинается с 0, при переборе символов справа налево, при работе цикла первому значению с права присваивается индекс -1.

На рисунке 1.4 представлен код программы к примеру 2.

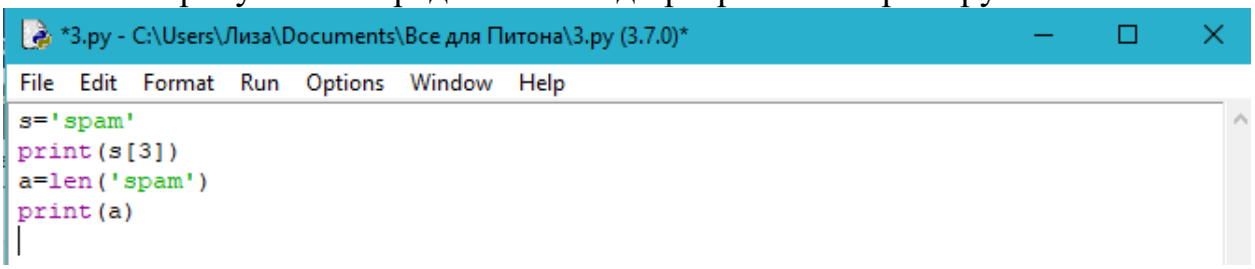


Рисунок 1.4 – Код программы к примеру 2

На рисунке 1.5 представлен результат работы программы в соответствии с примером 2.

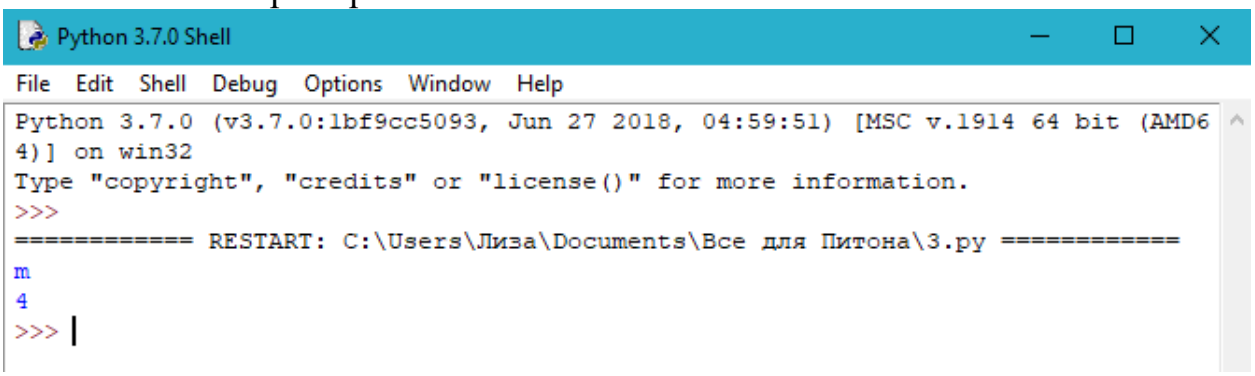


Рисунок 1.5 – Результат работы программы в соответствии с примером 2

Действительно, в коде указано print (s [3]) – что означает вывести из слова spam букву, соответствующую 3 индексу. При организации цикла слева направо буква s будет иметь индекс 0, p – индекс 1, a – индекс 2, m – индекс 3. Всего четыре индекса, поэтому программа выводит ответ:

m

4.

Пример 3

Изменить программу задания 2 так, чтобы символы проверялись справа налево.

Выполнение примера 3

На рисунке 1.6 показан код программы к примеру 3.

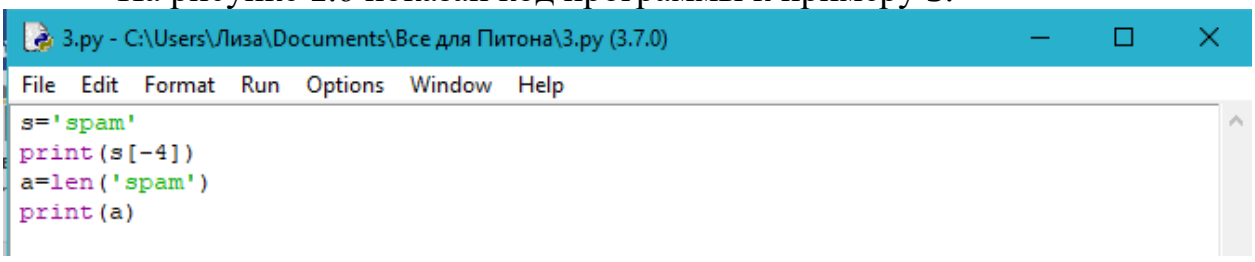


Рисунок 1.6 – Код программы к примеру 3

Требуется вывести четвертую букву с конца, поэтому код программы содержит строку: print (s [-4]).

Функция len () возвращает число элементов в указанном объекте-контейнере. Объект-контейнер указывается в круглых скобках, в данном случае это слово spam.

На рисунке 1.7 показан результат работы программы в соответствии с примером 3.

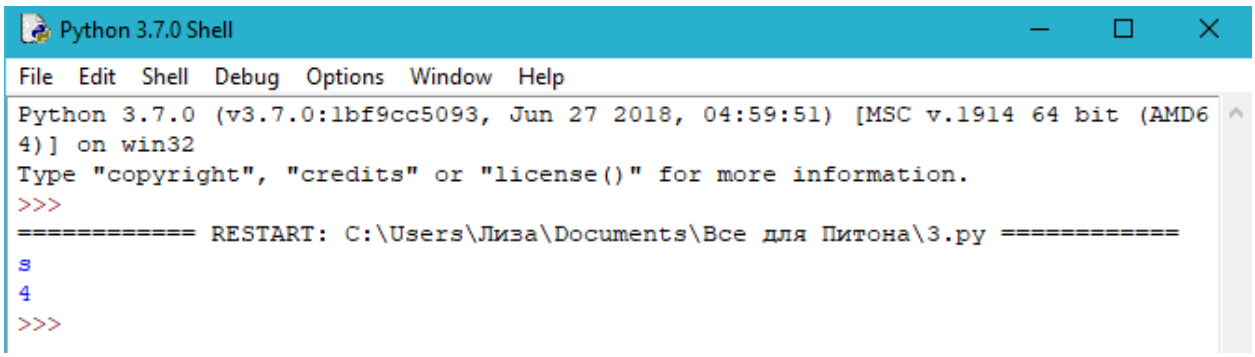


Рисунок 1.7 – Результат работы программы в соответствии с примером 3

В слове spam буква s четвертая с конца буква.

Если индекс заменить на значение большее, в сравнении с количеством букв в слове (индексов), то программа выдаст сообщение об ошибке (рисунок 1.8).

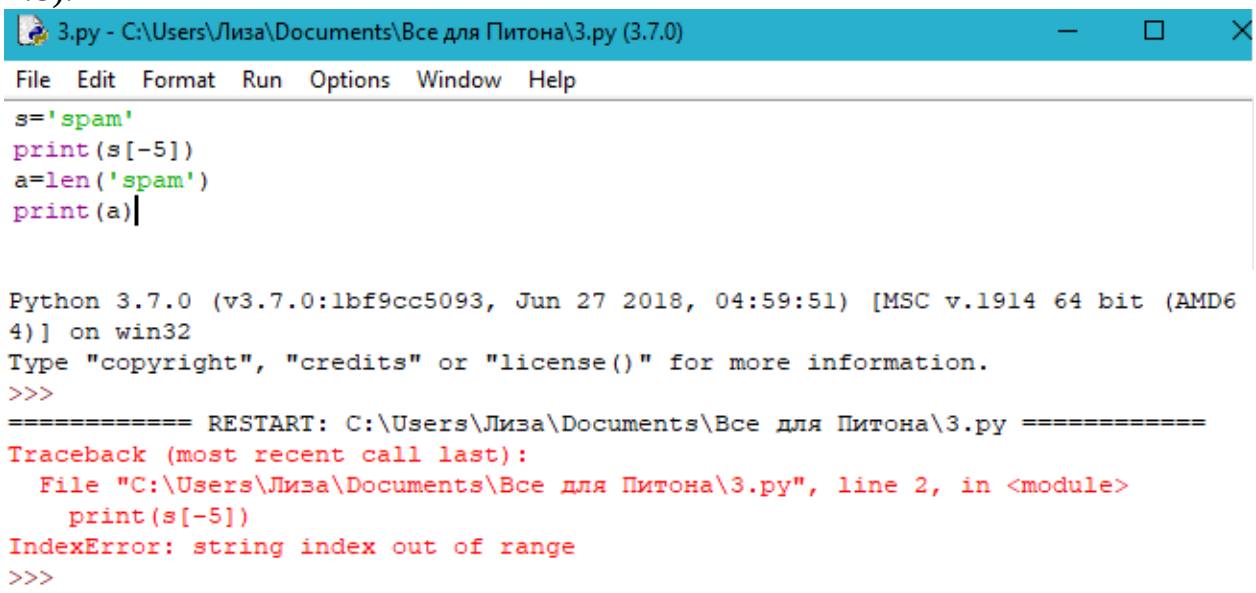


Рисунок 1.8 – Ошибка в коде программы и сообщение об ошибке

1.3. Оборудование и программное обеспечение

1. Персональная ЭВМ.
2. Операционная система Windows/Linux.
3. Среда программирования IDLE или другая среда, содержащая язык программирования Python.

1.4. Задание на работу

1. Изучить теоретические положения лабораторной работы.
2. Скачать и установить среду IDLE или другую среду, содержащую язык программирования Python.
3. Изучить команды меню главного окна программы, синтаксис и простейшие операторы языка Python.
4. Научиться сохранять набранные тексты программ в файлы.
5. Научиться загружать сохраненные файлы с текстами программ в окно редактирования среды IDLE (другой среды).
6. Получить результаты работы программ в соответствии с примерами 1-3.

7. В примерах 1-3 произвольно изменить фразу и объект-контейнер, произвольно изменить индексы для работы цикла for.

8. Показать результаты работы преподавателю.

9. Ответить на контрольные вопросы.

10. Оформить отчет.

1.5. Контрольные вопросы

1. Опишите синтаксис языка Python.
2. Опишите простейшие операторы языка Python.
3. Опишите структуру ветвления (выбора) языка Python.
4. Что позволяет осуществлять функция len в языке Python?
5. Что такое объект-контейнер?
6. Что означает запись end= ‘’ на языке Python?

1.6. Правила оформления отчета

Отчет должен содержать:

1. Номер, название и цель лабораторной работы.
2. Описание процесса выполнения примеров 1-3 (коды и результаты работы программ).
3. Описание процесса изменения примеров 1-3 в соответствии с заданием (коды и результаты работы программ).
4. Вывод по работе.

Лабораторная работа №2
Работа с циклами на языке Python
2.1. Цель лабораторной работы

Научиться работать с циклами на языке Python

2.2 Теоретические положения

Функция int () языка Python

Встроенная функция int () языка Python позволяет задать целочисленный тип данных, обрабатываемых в программе. Указать тип можно после того, как осуществлены операции ввода и вывода, например,

```
print ('Введите количество элементов ', end='')
k=input()
k=int(k)
```

Также встроенная функция int () языка Python возвращает целое число в десятичной системе счисления (класс int).

Если вызвать функцию int() без аргументов, она вернет 0. Если передать функции int() целое число, то она вернет это же целое число. Если передать функции int() вещественное число, то оно будет округлено до целого в сторону нуля (дробная часть будет отброшена), например:

```
>>> int(25)
25
>>> int(-80)
-80
>>> int(126.133)
126
>>> int(0.5)
0
>>> int(-0.9)
0.
```

Функция range () языка Python

Встроенная функция range() является универсальной функцией для создания списков (list) содержащих арифметическую прогрессию. Чаще всего она используется в циклах for. Функция range() может принимать от одного до трех агрументов, при этом аргументами должны быть целые числа (int). Функция range(старт, стоп, шаг) - так выглядит стандартный вызов функции range() в Python.

По умолчанию старт равняется нулю, шаг единице.

Функция range () возвращает список целых чисел в форме [старт, старт + шаг, старт + шаг*2...]. Если шаг положительное число, последним элементом списка будет наибольшее старт + i * шаг меньшее числа стоп.

Если шаг отрицательное число, то последний элемент будет наименьшее старт + i * шаг большее числа стоп.

Шаг не должен равняться нулю, иначе возникнет ошибка (ValueError).

Примеры использований функции range() в Python:

```
>>> range(7)
```

[0, 1, 2, 3, 4, 5, 6]
>>> range(1,8)
[1, 2, 3, 4, 5, 6, 7]
>>> range(0, 20, 5)
[0, 5, 10, 15]
>>> range(0, -7, -1)
[0,-1,-2,-3,-4,-5,-6].

Цикл for языка Python

Использование цикла for в простейших программах было описано выше.

Цикл for позволяет повторять одни и те же действия в программе до тех пор, пока не будет достигнуто предельное значение, при котором цикл перестанет работать. Цикл for по умолчанию работает с шагом 1, а следовательно, в этом случае для организации этого цикла требуется указать только начальное (при котором цикл начнет работать) и конечное (при котором цикл завершит работу) значение. Цикл for отлично подходит для перебора каждого значения и вычисления, например, суммы значений.

Цикл for на языке Python может работать, если задано начальное значение цикла и шаг.

Цикл for также называют циклом с параметром.

Схема работы цикла for представлена на рисунке 2.1.

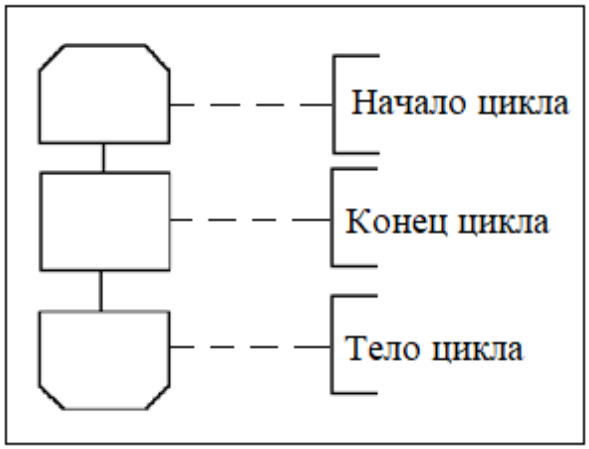


Рисунок 2.1 – Схема работы цикла *for*

Пример 4

Определить сумму чисел с использованием цикла for, если с клавиатуры вводится количество чисел, первое значение и шаг.

Выполнение примера 4

На рисунке 2.2 показан код программы к примеру 4.

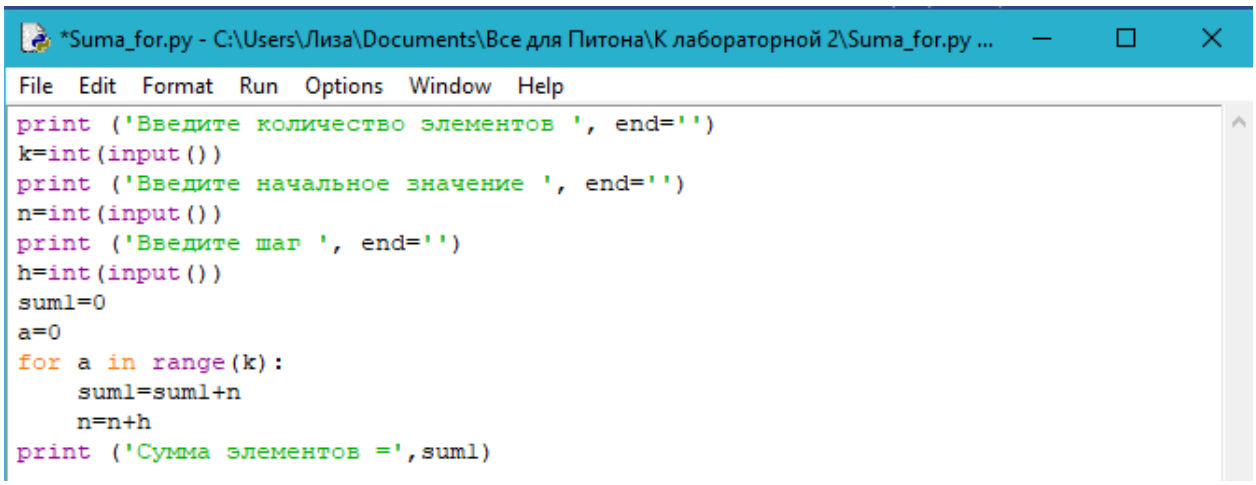


Рисунок 2.2 – Код программы к примеру 4

На рисунке 2.3 показан результат работы программы к примеру 4.

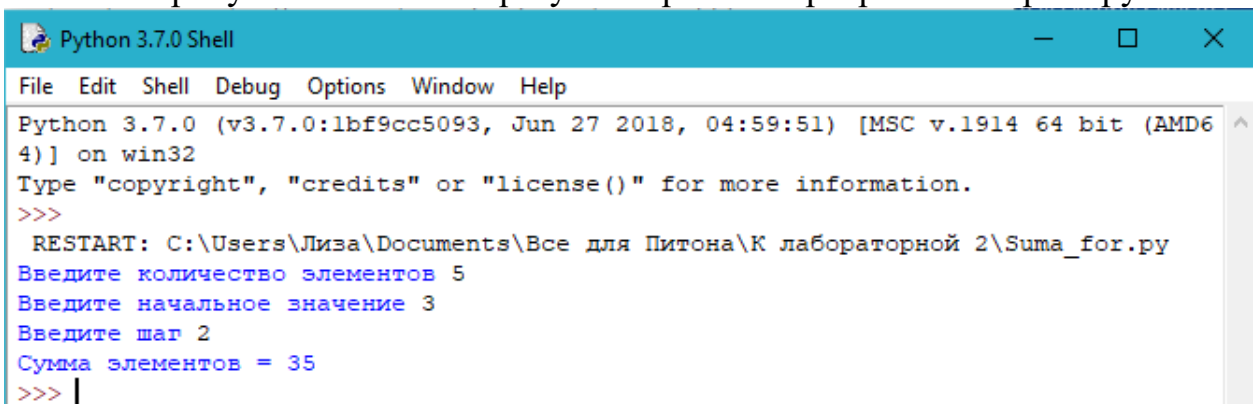


Рисунок 2.3 – Результат работы программы к примеру 4

Проверка: 3+(3+2)+(5+2)+(7+2)+(9+2)=35.

Примечание: запись k=int(input()) означает, что вводится в память машины целое число (одновременно осуществляется ввод данных и присвоение вводимым данным типа).

Если будут вводится вещественные числа, то программа выдаст сообщение об ошибке (рисунок 2.4).

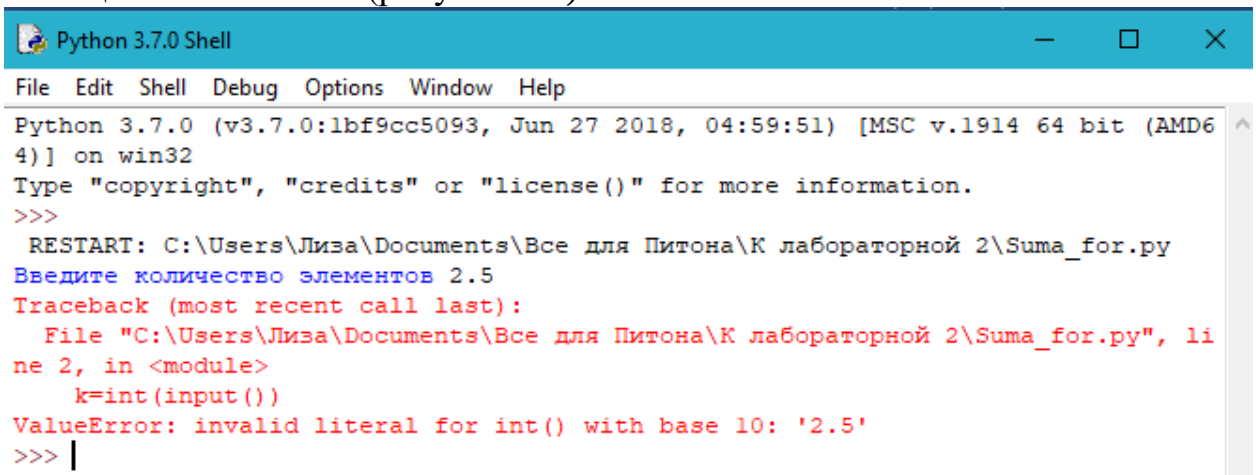


Рисунок 2.4 – Сообщение об ошибке после ввода вещественного значения (что должно соответствовать количеству суммируемых элементов)

Цикл while языка Python

Цикл while используется для повторения частей кода, он является условным оператором и выполняет работу до тех пор, пока не достигнет определенного условия, например, если требуется вывести цифры от 0 до 9 (каждая цифра будет находится в отдельной строке), то можно организовать следующий цикл:

```
i = 0
while i < 10:
   print(i)
   i = i + 1
```

Цикл while подходит и для определения суммы чисел. Если некоторой переменной, отвечающей за накопление суммируемых значений, перед работой цикла присвоить значение 0, то по мере работы тела цикла (все то, что после слова while) эта переменная сможет накапливать сумму, так как к 0 сначала прибавится первое число, затем к первому числу прибавится второе и так далее.

Цикл while можно использовать и для определения положительных, отрицательных, дробных, целых чисел в заданной последовательности, при этом переменная, отвечающая за накопление, в теле цикла будет увеличиваться на 1, если в последовательности обнаружено требуемое по характеристикам число (положительное, отрицательное, дробное, целое и прочее).

Структура цикла while описывается схемой, представленной на рисунке 2.5.

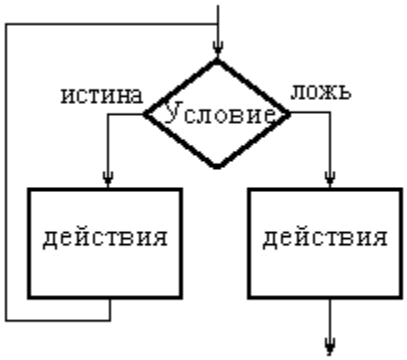


Рисунок 2.5 – Схема работы цикла while

Пример 5

Определить сумму чисел с применением цикла while, если с клавиатуры вводится количество чисел, первое значение и шаг.

Выполнение примера 5

На рисунке 2.6 показан код программы к примеру 5.

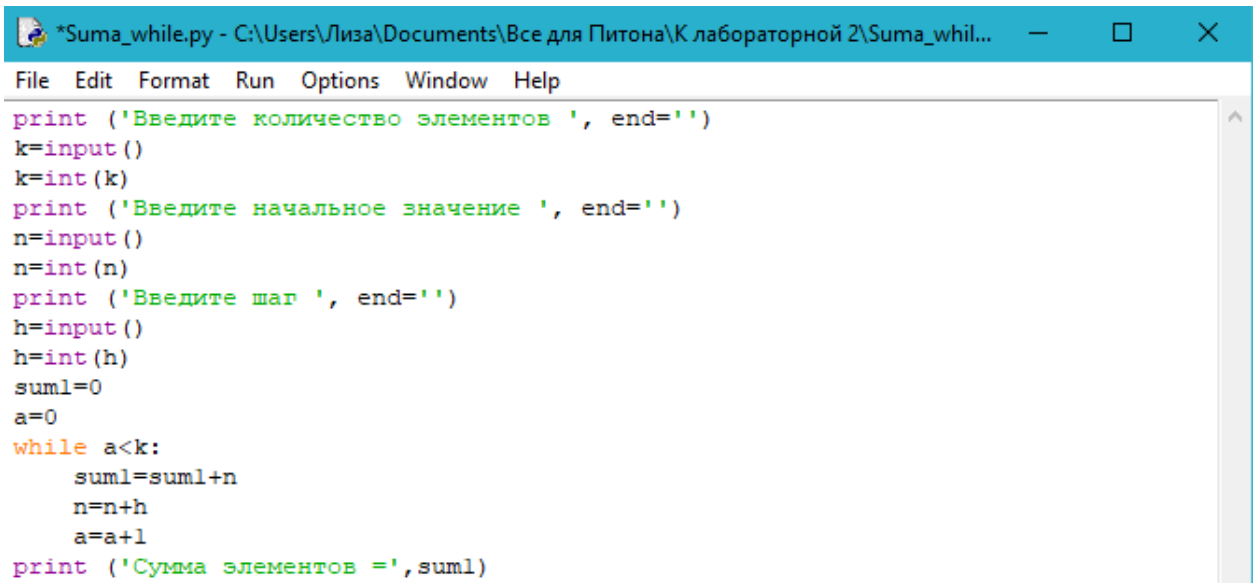


Рисунок 2.6 – Код программы к примеру 5

На рисунке 2.7 показан результат работы программы к примеру 5.

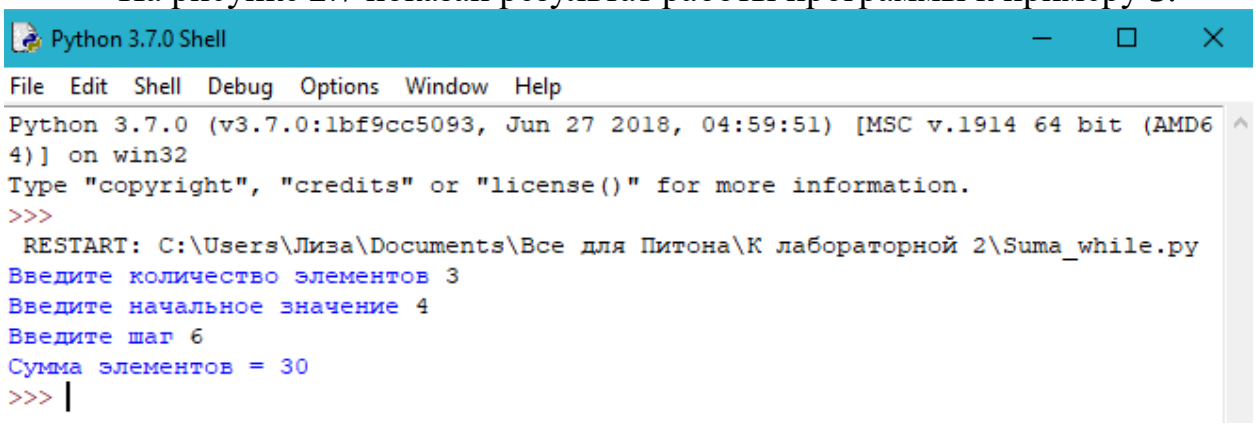


Рисунок 2.7 – Результат работы программы к примеру 5

Проверка: 4+(4+6)+(10+6)=30.

Примечание: код программы к примеру 5 можно было бы сократить путем объединения нескольких действий в одну строку (ввод данных в память машины и объявление типа данных).

На рисунке 2.8 показан сокращенный код программы к примеру 5.

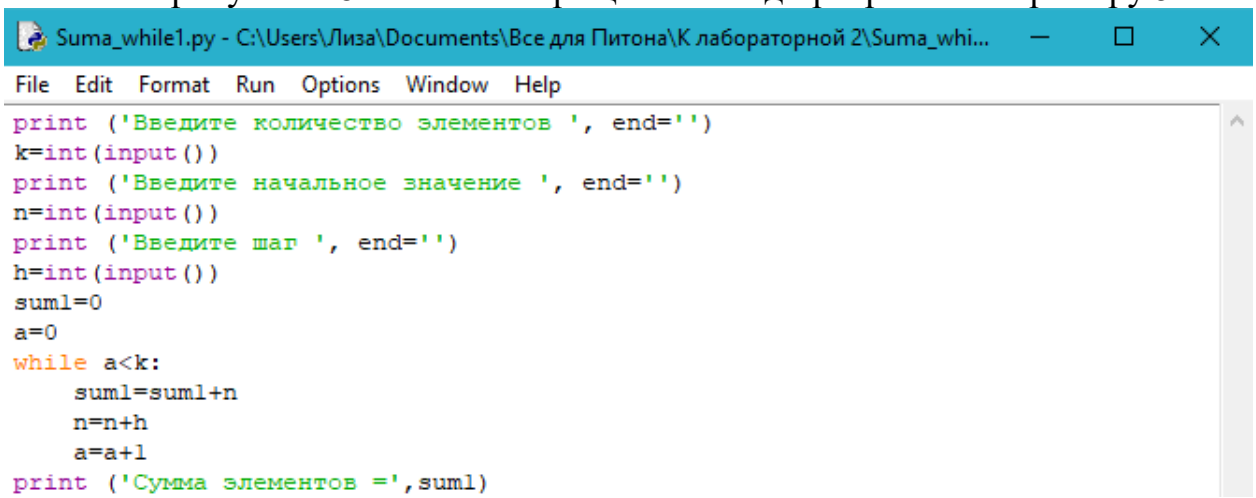


Рисунок 2.8 – Сокращенный код программы к примеру 5.

Пример 6

В заданном диапазоне чисел определить количество простых чисел, их сумму и произведение.

Примечание: простое число – это число, которое делится на само себя и на единицу (не имеет иных делителей).

Выполнение примера 6

На рисунке 2.9 показан код программы к примеру 6.

File Edit Format Run Options Window Help

```
for v in range(100):
    print ('N= ',end='')
    n=int(input())
    print ('M= ',end='')
    m=int(input())
    if (n>0)and(m>0)and(n<m):
        break
k=0
Sum=0
P=1
for i in range(n,m):
    Pr=True
    if i==4:
       Pr=False
    for j in range(2,i//2):
        if (i%j==0):
            Pr=False
            break
    if Pr==True:
        k=k+1
        Sum=Sum+i
        P=P*i
        print(i)
if k==0:
    print('Простых чисел в диапазоне нет')
else:
    print('Простых чисел в диапазоне ',k)
    print('Сумма простых чисел в диапазоне ',Sum)
    print('Произведение простых чисел в диапазоне ',P)
```

Рисунок 2.9 – Код программы к примеру 6

В указанном коде программы:

Блок

```
for v in range(100):
    print ('N= ',end='')
    n=int(input())
    print ('M= ',end='')
    m=int(input())
    if (n>0)and(m>0)and(n<m):
        break
```

отвечает за формирование последовательности натуральных чисел, при этом последовательность ограничена числами N и M, каждое из которых больше 0 и N>M. После ввода значений N и M и проверки их на указанные условия, цикл прерывается (последовательность становится известной, то есть

заданной). Запись *in range(100)* означает, что цикл может повториться 100 раз (100 раз может не выполниться условие).

Примечание: число, ограничивающее последовательность справа (число M), не входит в диапазон, в котором определяются простые числа, их сумма и произведение, то есть исключается из диапазона.

Эта часть кода могла быть представлена циклом repeat until (повторять пока или повторять до), однако такой структуры в языке Python нет. Структура цикла repeat until в языке Python заменена структурой цикла for и условным оператором if.

Схема работы цикла repeat until представлена на рисунке 2.10.

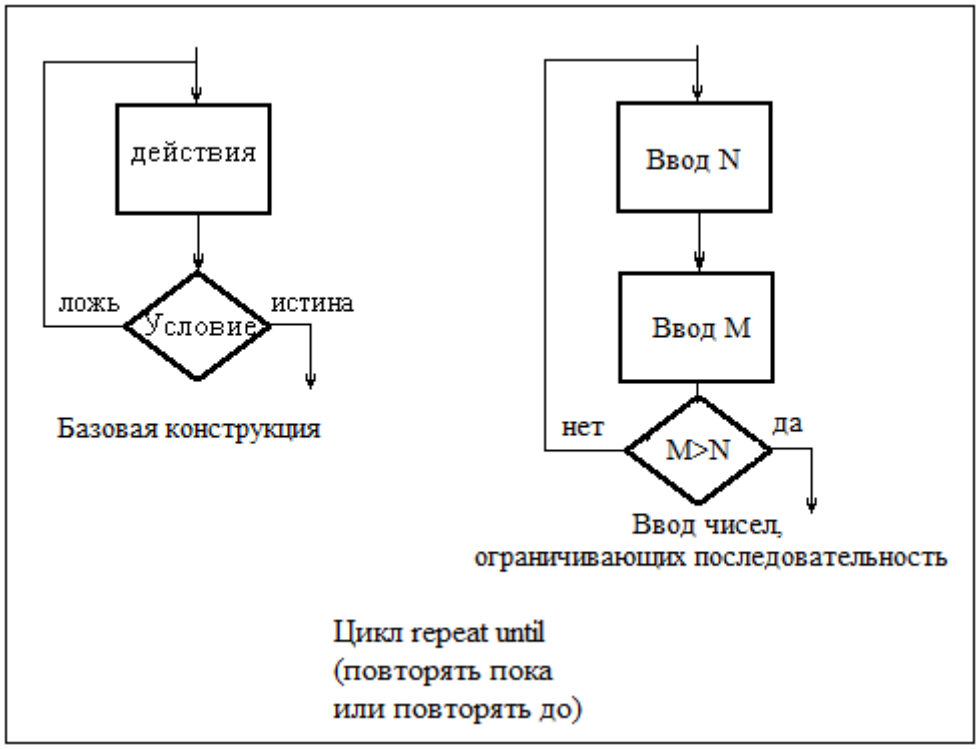


Рисунок 2.10 – Схема работы цикла repeat until

Блок

k=0

Sum=0

P=1

отвечает за присвоение накапливающим переменным (переменным-счетчикам) начальных значений.

Если переменные будут накапливать значения путем их суммирования, то начальные значения переменных равны 0 (к=0, затем к=0+1=1, затем к=1+1=2 и т.д.,, если считается количество каких-либо объектов или Sum=0, затем Sum=0+25=25, Sum=25+35=60 и т.д.,, если считается сумма каких-либо значений).

Если переменные будут накапливать значения путем их перемножения, то начальные значения переменных равны 1 (p=1, затем p=1*2=2, затем p=2*4=8 и т.д., если считается произведение каких-либо значений).

Блок

for i in range(n,m):

```
Pr=True
if i==4:
  Pr=False
```

отвечает за то, что в последовательности, ограниченной числами n,m переменная Pr принимает значение истина, в противном случае – ложь. Иными словами, в последовательности натуральных чисел 1,2,3,4 – число 4 – первое число, не являющееся простым.

Блок

```
for j in range(2,i//2):
    if (i%j==0):
      Pr=False
      break
```

отвечает за присвоение переменной Pr значения ложь в случае, если в последовательности чисел от 2 до i//2 (где i//2 – позволяет получить целую часть целочисленного деления числа, хранящегося в переменной i, на число 2) при целочисленном делении i на j (i%j – определяет целочисленный остаток от деления) обнаруживается нулевой остаток.

Пример работы текущего блока:

Пусть имеется последовательность чисел 6,7,8.

// - целая часть при делении;

% - целочисленный остаток от деления.

Тогда:

- для числа 6: в последовательности от 2, до 6//2 или от 2 до 3: если 6%2=0 (так как 6:2=3 остаток 0), 6%3=0 (так как 6:3=2 остаток 0), pr=false;

- для числа 7: в последовательности от 2, до 7//2 или от 2 до 3: если 7%2=1 (так как 7:2=3 остаток 1), 7%3=1 (так как 7:3=2 остаток 1), pr=true;

- для числа 8: в последовательности от 2, до 8//2 или от 2 до 4: если 8%2=0 (так как 8:2=4 остаток 0), 8%3=2 (так как 8:3=2 остаток 2), 8%4=0 (так как 8:4=2 остаток 0), pr= false.

Иными словами, в последовательности чисел 6,7,8 выявлено одно простое число – это число 7, так как переменная в сформированном цикле приняла значение true.

Примечание:

а) при рассмотрении числа 1: в последовательности от 2, до 1//2 или от 2 до 0: если 1%2=1 (так как 1:2=0 остаток 1), pr=true; иными словами, 1 – простое число (далее нет необходимости рассматривать цикл, так как переменная pr уже приняла значение true, тем более, что цикл от 2 до 0 требует отрицательного шага);

б) при рассмотрении числа 2: в последовательности от 2, до 2//2 или от 2 до 1: если 2%2=0 (так как 2:2=1 остаток 0), если 2%1=0 (так как 2:1=2 остаток 0), pr=false; иными словами, 2 – не является простым числом;

в) при рассмотрении числа 3: в последовательности от 2, до 3//2 или от 2 до 1: если 3%2=1 (так как 3:2=1 остаток 1), pr= true; иными словами, 3 – простое число;

г) число 4 рассмотрено в отдельном блоке программы;

д) при рассмотрении числа 5: в последовательности от 2, до 5//2 или от 2 до 2: если 5%2=1 (так как 5:2=2 остаток 1), pr= true; иными словами, 5 – простое число.

Блок

```
if Pr==True:
    k=k+1
    Sum=Sum+i
    P=P*i
    print(i)
```

отвечает за подсчет простых чисел: если переменная Pr принимает значение истина, то:

- переменная k увеличивается на единицу;
- переменная Sum возрастает на величину i, где i – значение элемента в последовательности;
- переменная P возрастает в i раз, где i – значение элемента в последовательности.

Блок

```
if k==0:
   print('Простых чисел в диапазоне нет')
else:
   print('Простых чисел в диапазоне ',k)
   print('Сумма простых чисел в диапазоне ',Sum)
   print('Произведение простых чисел в диапазоне ',P)
```

отвечает за вывод информации.

На рисунке 2.11а показан результат работы программы в соответствии с примером 6 (число M – не является простым).

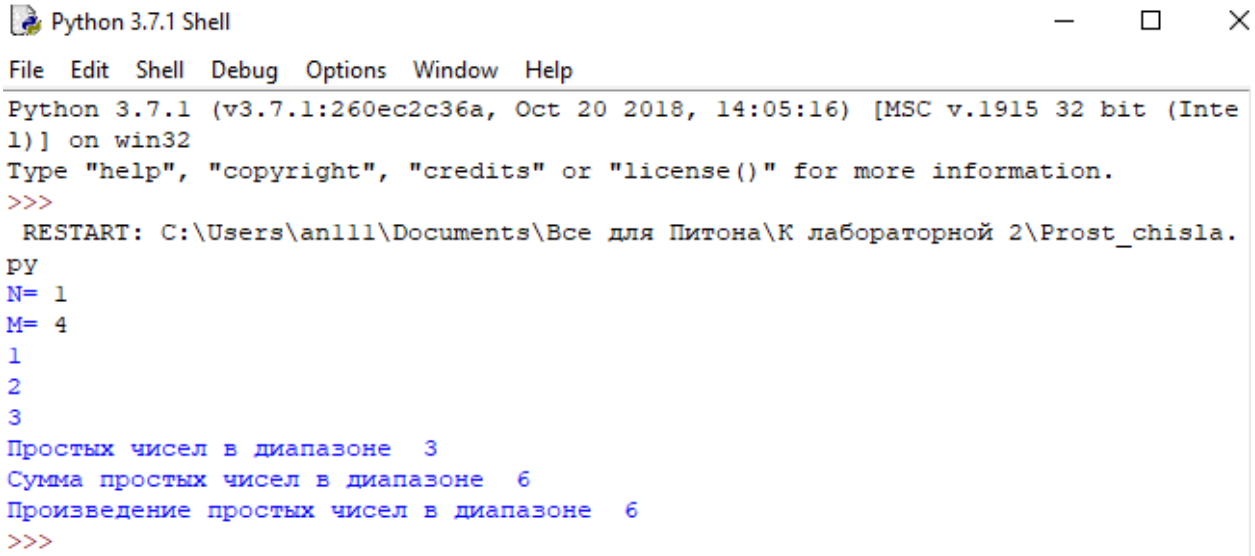


Рисунок 2.11а – Результат работы программы в соответствии с примером 6 (число M – не является простым)

Проверка: в диапазоне натуральных чисел от 1 до 4 (не включая число 4) имеется 3 простых числа – это числа 1,2,3. Их сумма – 1+2+3=6, их произведение – 1*2*3=6.

На рисунке 2.11б показан результат работы программы в соответствии с примером 6 (число M – является простым).

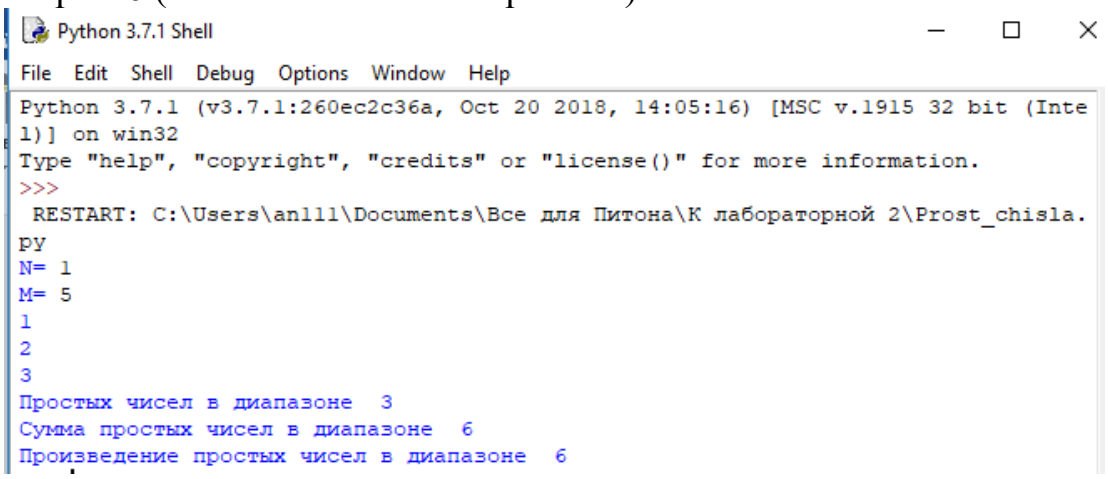


Рисунок 2.11б – Результат работы программы в соответствии с примером 6 (число M – является простым)

На рисунке 2.11в показан результат работы программы в соответствии с примером 6 (число M=7).

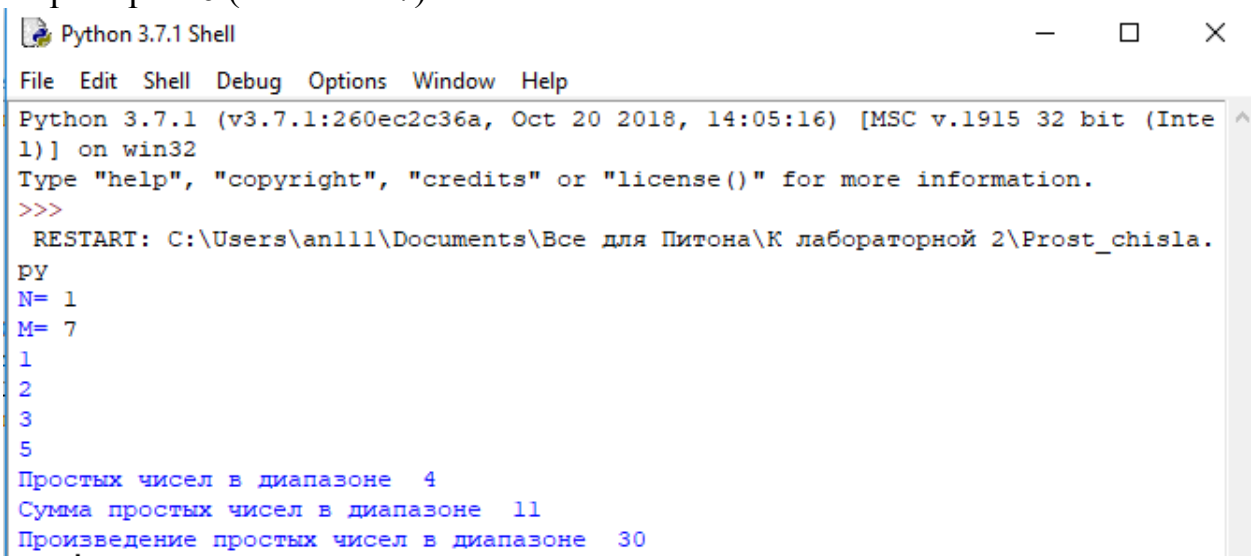


Рисунок 2.11в – Результат работы программы в соответствии с примером 6 (число M =7)

На рисунке 2.12 показан результат работы программы в соответствии с примером 7 в случае, если условие M>N не срабатывает (цикл повторяется до тех пор, пока условие M>N не будет выполнено).

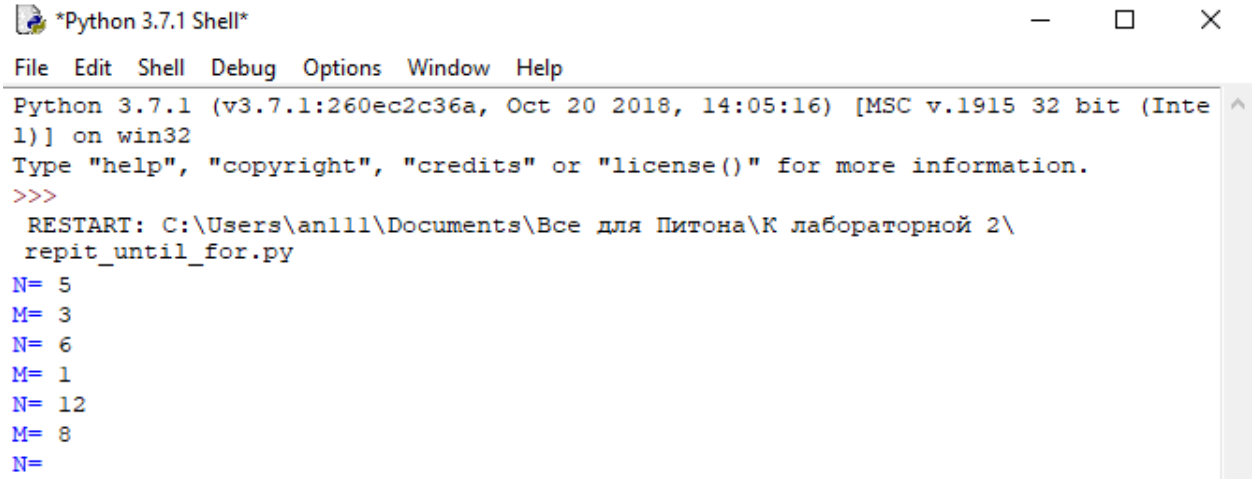


Рисунок 2.12 – Результат работы программы в случае, если условие M>N не срабатывает

2.3. Оборудование и программное обеспечение

1. Персональная ЭВМ.
2. Операционная система Windows/Linux.
3. Среда программирования IDLE или другая среда, содержащая язык программирования Python.

2.4. Задание на работу

1. Изучить теоретические положения лабораторной работы.
2. Получить результаты работы программ в соответствии с примерами 4-6.
3. Организовать вод цифр от 0 до 9 с помощью:

а) цикла for;

б) цикла while.

3. С применением цикла while определить:

а) сумму положительных элементов массива;

б) сумму отрицательных элементов массива.

4. С применением цикла for определить

а) сумму положительных элементов массива;

б) сумму отрицательных элементов массива.

5. Написать программу для подсчета в заданной последовательности натуральных чисел количества чисел, имеющих делитель равный 2.
6. Написать программу для подсчета в заданной последовательности натуральных чисел количества чисел, имеющих делитель равный 3, а также для подсчета суммы и произведения таких чисел.
7. Изобразить схемы работы программ в соответствии с ГОСТ 19.701-90 (использовать графический редактор).
8. Показать результаты работы преподавателю.
9. Ответить на контрольные вопросы.
10. Оформить отчет.

2.5. Контрольные вопросы

1. Как работает функция int () языка Python?
2. Как работает функция range () языка Python?

3. Как работает цикл for?

4. Как работает цикл while?

5. Как можно сократить число строк программы, составленной на языке Python?

6. Чему равно начальное значение переменной-счетчика для определения количества объектов определенного свойства в заданной последовательности?

7. Чему равно начальное значение переменной-счетчика для определения произведения/суммы объектов определенного свойства в заданной последовательности?

8. Для чего необходим символ // в языке Python?

9. Для чего необходим символ % в языке Python?

10. Как определить простое число при помощи операций целочисленного деления?

2.6. Правила оформления отчета

Отчет должен содержать:

1. Номер, название и цель лабораторной работы.

2. Описание процесса выполнения примеров (коды, схемы и результаты работы программ).

3. Вывод по работе.

Лабораторная работа №3
Организация массивов на языке Python
3.1. Цель лабораторной работы

Научиться организовывать массивы на языке Python

3.2 Теоретические положения

Служебное слово *row* на языке Python необходимо для осуществления манипуляций со строками матриц.

Пример 7

Сформировать массив, имеющий шесть строк и пять столбцов, причем, чтобы каждый элемент массива, записывающийся при движении по строке этого массива, был равен произведению текущего номера строки на текущий номер столбца.

Выполнение примера 7

На рисунке 3.1 показан код программы к примеру 7.

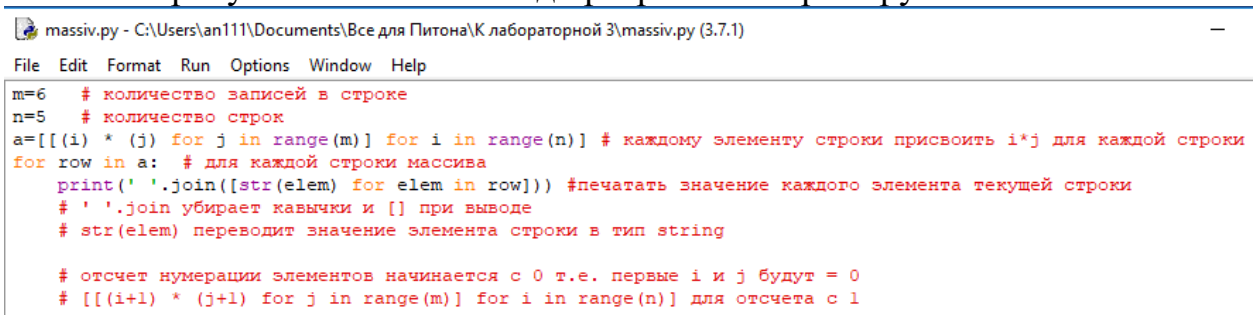


Рисунок 3.1 – Код программы к примеру 7

Примечание: на языке Python комментарии начинаются со знака #.

В представленном коде программы:

m=6

n=5

указание количества строк и столбцов массива (соответствующие комментарии к коду представлены).

Примечание: указывается количество записей в строке и в столбце соответственно.

Фрагмент кода

*a=[[(i) * (j) for j in range(m)] for i in range(n)]*

отвечает за то, что каждый элемент массива формируется путем перемножения номеров строк и столбцов массива.

Примечание: нумерация строк и столбцов массива начинается с нуля.

Фрагмент кода

for row in a:

print(' '.join([str(elem) for elem in row]))

отвечает за вывод полученных значений элементов массива.

Примечание:

Запись *' '.join* убирает кавычки и [] при выводе.

Запись *str(elem)* переводит значение элемента строки в тип string.

Для отсчета номеров строк и столбцов, начиная с 1, в коде программы необходимо предусмотреть *[[(i+1) * (j+1) for j in range(m)] for i in range(n)]*.

Соответствующие комментарии представлены в тексте кода программы.

На рисунке 3.2 представлен результат работы программы в соответствии с примером 7.

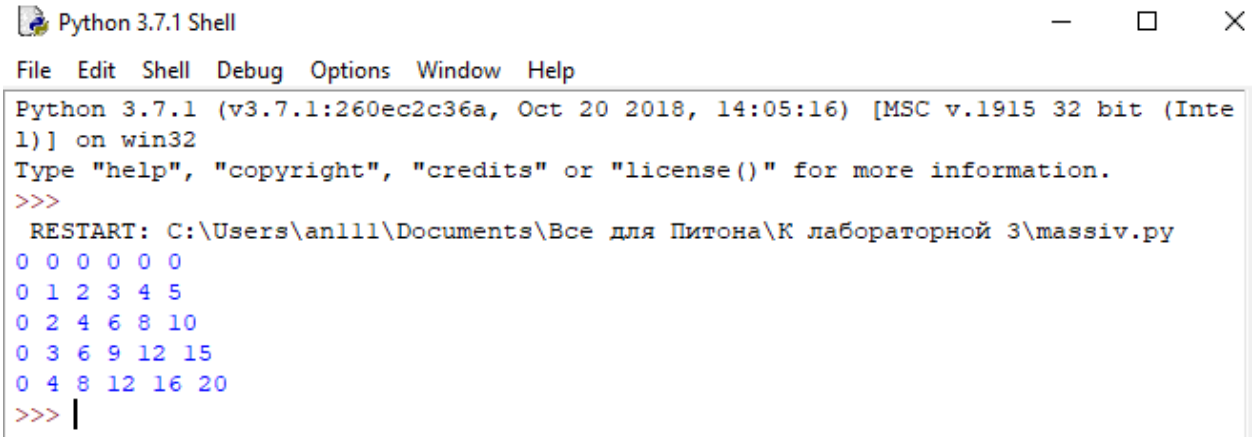


Рисунок 3.2 – Результат работы программы в соответствии с примером 7

Рассмотрим первую строку матрицы: индекс строки (i) равен 0, индекс столбца (j) изменяется от 1 до 6 (количество записей в строке 6).

Тогда:

- первый элемент равен: 0*0=0;
- второй элемент равен: 0*1=0;
- третий элемент равен: 0*2=0;
- четвертый элемент равен: 0*3=0;
- пятый элемент равен: 0*4=0;
- шестой элемент равен: 0*5=0.

Рассмотрим вторую строку матрицы: индекс строки (i) равен 1, индекс столбца (j) изменяется от 1 до 6 (количество записей в строке 6).

Тогда:

- первый элемент равен: 1*0=0;
- второй элемент равен: 1*1=1;
- третий элемент равен: 1*2=2;
- четвертый элемент равен: 1*3=3;
- пятый элемент равен: 1*4=4;
- шестой элемент равен: 1*5=5.

Рассмотрим третью строку матрицы: индекс строки (i) равен 2, индекс столбца (j) изменяется от 1 до 6 (количество записей в строке 6).

Тогда:

- первый элемент равен: 2*0=0;
- второй элемент равен: 2*1=2;
- третий элемент равен: 2*2=4;
- четвертый элемент равен: 2*3=6;
- пятый элемент равен: 2*4=8;
- шестой элемент равен: 2*5=10.

Рассмотрим четвертую строку матрицы: индекс строки (i) равен 3, индекс столбца (j) изменяется от 1 до 6 (количество записей в строке 6).

Тогда:

- первый элемент равен: 3*0=0;
- второй элемент равен: 3*1=3;
- третий элемент равен: 3*2=6;
- четвертый элемент равен: 3*3=9;
- пятый элемент равен: 3*4=12;
- шестой элемент равен: 3*5=15.

Рассмотрим пятую строку матрицы: индекс строки (i) равен 4, индекс столбца (j) изменяется от 1 до 6 (количество записей в строке 6).

Тогда:

- первый элемент равен: 4*0=0;
- второй элемент равен: 4*1=4;
- третий элемент равен: 4*2=8;
- четвертый элемент равен: 4*3=12;
- пятый элемент равен: 4*4=16;
- шестой элемент равен: 4*5=20.

Пример 8

Сформировать массив, имеющий шесть строк и пять столбцов, причем, чтобы каждый элемент массива, записывающийся при движении по строке этого массива, был равен произведению текущего номера строки на текущий номер столбца и, кроме того, если значение элемента массива больше 5, заменить этот элемент на элемент 44.

Выполнение примера 8

На рисунке 3.3 показан код программы к примеру 8.

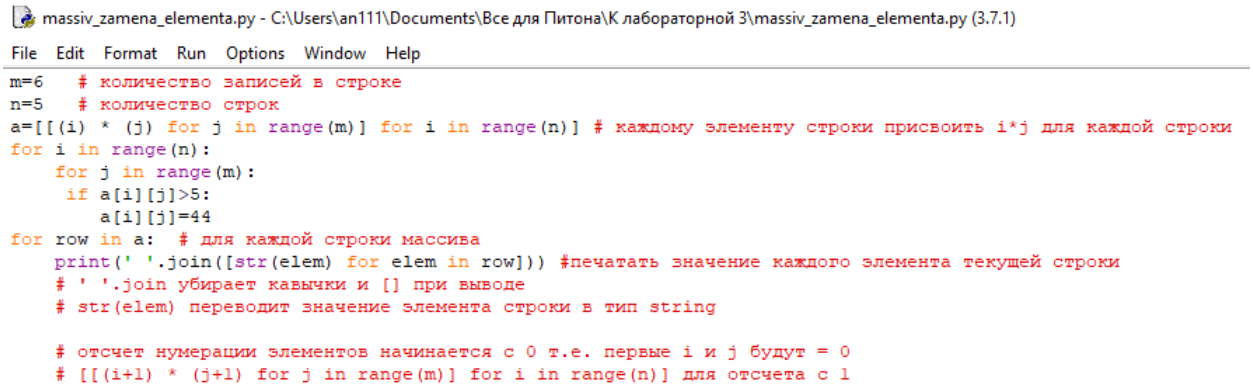


Рисунок 3.3 – Код программы к примеру 8

Программа содержит цикл, необходимый для замена переменной.

Фрагмент кода

```
for j in range(m):
    if a[i][j]>5:
      a[i][j]=44
```

отвечает за замену элемента, значение которого больше 5, на элемент 44.

На рисунке 3.4 представлен результат работы программы в соответствии с примером 8.

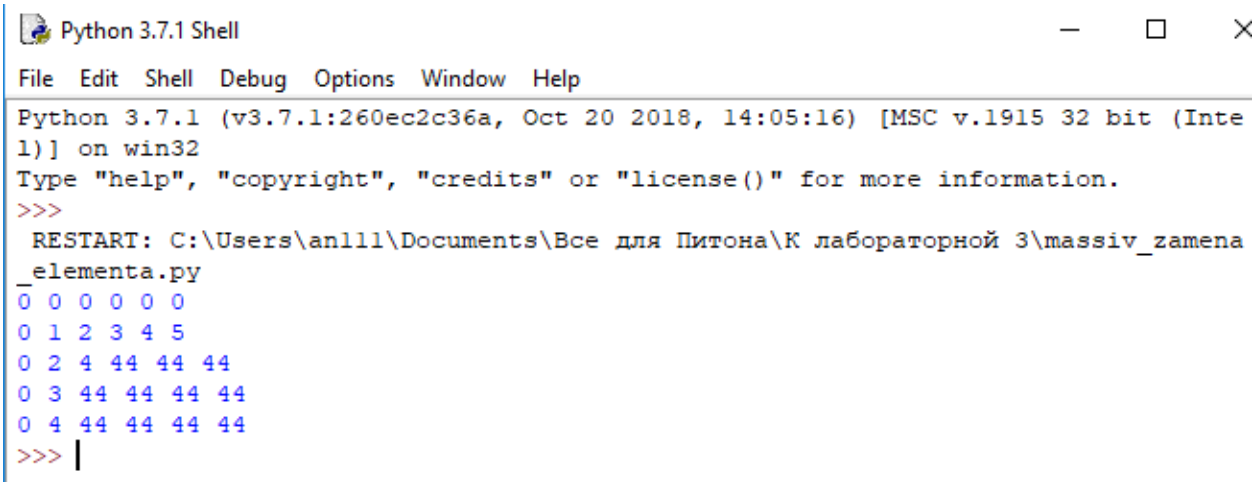


Рисунок 3.4 – Результат работы программы в соответствии с примером 8

Пример 9

Сформировать массив, имеющий шесть строк и пять столбцов, причем, чтобы каждый элемент массива, записывающийся при движении по строке этого массива, был равен произведению текущего номера строки на текущий номер столбца и, кроме того, значения элементов на главной диагонали были бы равны 1, значения элементов над главной диагональю – 2, под главной диагональю – 0.

Выполнение примера 9

На рисунке 3.5 показан код программы к примеру 9.

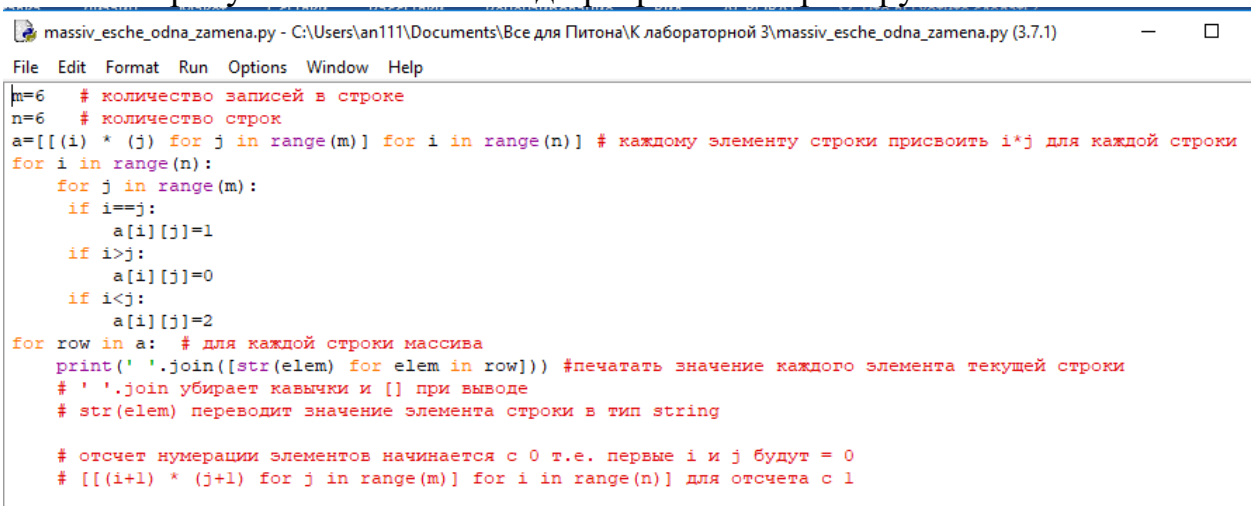


Рисунок 3.5 – Код программы к примеру 9

Программа содержит цикл, необходимый для замена переменной.

Фрагмент кода

```
for j in range(m):
    if i==j:
        a[i][j]=1
    if i>j:
        a[i][j]=0
    if i<j:
        a[i][j]=2
```

отвечает за замену элементов в соответствии с заданием.

На рисунке 3.6 представлен результат работы программы в соответствии с примером 9.

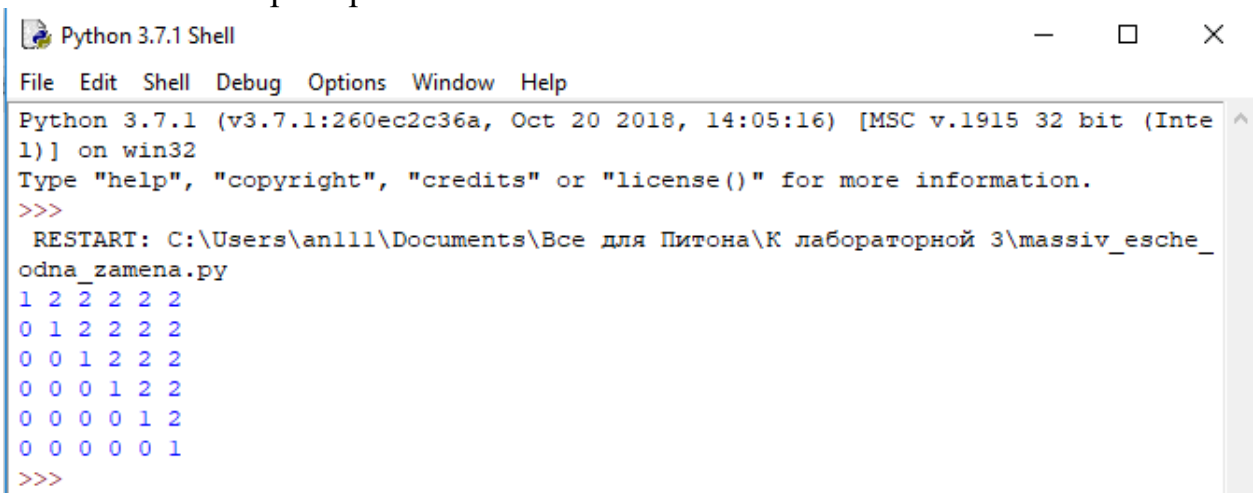


Рисунок 3.6 – Результат работы программы в соответствии с примером 9

3.3. Оборудование и программное обеспечение

1. Персональная ЭВМ.
2. Операционная система Windows/Linux.
3. Среда программирования IDLE или другая среда, содержащая язык программирования Python.

3.4. Задание на работу

1. Изучить теоретические положения лабораторной работы.
2. Получить результаты работы программ в соответствии с примерами 7-9.
3. Для примера 7:

а) произвольно изменить количество записей по строке и столбцу и проверить правильность работы программы путем расчетов;

б) организовать заполнение массива элементами при условии, что подсчет индекса строки и столбца начинается с 1 (проверить правильность работы программы путем расчетов);

в) организовать заполнение массива элементами при условии, что подсчет индекса строки и столбца начинается с 2 (проверить правильность работы программы путем расчетов).

4. Для примера 8:

а) произвольно изменить количество записей по строке и столбцу и проверить правильность работы программы путем расчетов.

б) осуществить замену элемента, значение которого равно 0, на любой произвольный элемент;

в) осуществить замену элемента, значение которого меньше b, на элемент, (i, j), если b, i, j – произвольные параметры (отсчет индекса строки и столбцов начинается с 0).

5. Для примера 9:

а) произвольно изменить количество записей по строке и столбцу и проверить правильность работы программы путем расчетов.

б) осуществить замену элементов на произвольный элемент, стоящих во второй строке матрицы (отсчет индексов элементов начинается с 0);

в) осуществить замену элементов на произвольный элемент, стоящих на второй (не главной) диагонали матрицы (отсчет индексов элементов начинается с 0);

в) осуществить замену элементов на произвольный элемент, стоящих в третьем столбце матрицы (отсчет индексов элементов начинается с 0);

г) осуществить замену элементов на произвольный элемент, в соответствии с тремя предыдущими условиями (отсчет индексов элементов начинается с 1).

6. Разработать программу для формирования элементов матрицы путем сложения индексов строк и столбцов, выполнить в программе произвольные замены элементов.

7. Изобразить схемы работы программ в соответствии с ГОСТ 19.701-90 (использовать графический редактор).

8. Показать результаты работы преподавателю.

9. Ответить на контрольные вопросы.

10. Оформить отчет.

3.5. Контрольные вопросы

1. Что означает запись row на языке Python?
2. Что означает запись *join* на языке Python?
3. Что означает запись *[str(elem) for elem in row]* на языке Python?
3. Как сформировать двумерный массив (матрицу) на языке Python?
4. Какими способами можно заполнить матрицу элементами?
5. Как реализовать замены элементов?
6. Как осуществляется подсчет индексов строк и столбцов матриц в языке Python?
7. Что необходимо сделать, чтобы подсчет индексов строк и столбцов матриц в языке Python начинался не с 0?
8. Какие операторы были использованы для решения примеров в текущей лабораторной работе?
9. Какие циклы были использованы для решения примеров в текущей лабораторной работе?
10. Как оформляется комментарий в программах, написанных на языке Python?

3.6. Правила оформления отчета

Отчет должен содержать:

1. Номер, название и цель лабораторной работы.
2. Описание процесса выполнения примеров (коды, схемы и результаты работы программ).
3. Вывод по работе.

Лабораторная работа №4
Графические возможности языка Python
4.1. Цель лабораторной работы

Научиться работать с графикой на языке Python

4.2 Теоретические положения

Для языка программирования Python графические возможности представлены специальной библиотекой – *tkinter*. Как и любой модуль, библиотеку *tkinter* в Python можно импортировать двумя способами: командами *import tkinter* или *from tkinter import*.

Удобнее пользоваться вторым способом, поскольку это позволяет не указывать каждый раз имя модуля при обращении к объектам, которые в нем содержатся. Следует обратить внимание, что в версии Python 3 имя модуля пишется со строчной буквы (*tkinter*), хотя в более ранних версиях Python использовалась прописная (*Tkinter*). Строчка кода, отвечающая за вызов библиотеки, отвечающей за графику, имеет вид: *from tkinter import **.

Символ * в Python 3 указывает на то, что могут быть использованы в программе все методы, заключенные в библиотеку.

Любое действие в программе может быть заключено в окно (так называемый, оконный интерфейс программы). Объект «окно» создается при обращении к классу Tk модуля *tkinter*. Переменную, связанную с объектом-окном, принято называть *root* (можно назвать как угодно, но так принято среди разработчиков программ на языке Python). Строчка кода, отвечающая за создание окна имеет вид: *root = Tk()*.

В Python можно указывать размер созданного окна. По умолчанию окно приложения появляется в верхнем левом углу экрана. Его размер (ширина и высота) определяется совокупностью размеров, расположенных в нем виджетов. В случае если окно пустое, то tkinter устанавливает его размер в 200 на 200 пикселей.

При использовании конструкции переменная = значение (например, *size =600*) также можно установить размер окна приложения.

В окнах можно создавать холст, то есть пространство, на котором будут воспроизводится графические изображения.

Создание холста в Python также возможно при использовании библиотеки *tkinter*. В библиотеке *tkinter* при использовании класса *Canvas* создаются объекты-холсты, на которых можно «рисовать», размещая различные фигуры и объекты. Делается это с помощью вызовов соответствующих методов.

При создании экземпляра Canvas необходимо указать его ширину и высоту. При размещении геометрических примитивов и других объектов указываются их координаты на холсте. Точкой отсчета является верхний левый угол.

Например, строка кода *canvas = Canvas(root, width=size, height=size)* необходима для создания холста с использованием значения переменной *size* (значение которой должно быть обвялено заранее).

Для того, чтобы холст вписывался в размер окна можно использовать конструкцию *canvas.pack()*,то есть указание того, что холст следует расположить внутри созданного окна.

Иногда для создания графических изображений целесообразно использовать модуль *random* предоставляет функции для генерации случайных чисел, букв, случайного выбора элементов последовательности.

Модуль *random* содержит ряд функций для работы со случайностями (вызова случайных чисел, символов, элементов и прочее).

В таблице 4.1 представлены возможности модуля *random* в Python.

Таблица 4.1 – Возможности модуля *random* в Python

Модуль random	Использование
random()	Случайное десятичное [0.0, 1.0)
randint(a, b)	Случайное целое число [a, b]
randrange(a, b[, step])	Случайное целое число [a, b)
uniform(a, b)	Случайное десятичное [a, b]
choice(seq)	Случайный элемент из seq
choices(seq, k=1)	Случайные k элементы из seq с заменой
sample(population, k)	Случайные k элементы из seq без замены
shuffle(x[, random])	Перемешивание порядка
normalvariate(mu, sigma) или gauss(mu, sigma)	Пример из нормального распределения со средним значением mu и стандартным отклонением sigma

Например, строка кода *x = randint(0, size)* позволяет формировать целое случайное число в диапазоне от 0 до значения переменной *size* (значение которой ранее должно быть объявлено).

Для создания графического объекта внутри холста потребуется кодовая конструкция *canvas.create_объект (координаты)*.

Например, кодовая конструкция *canvas.create_oval(x, y, x+d, y+d, fill=colors)*позволяет изображать на холсте круги с центрами в точках x, y размером x+d, y+d, причем значение переменной d должно быть заранее задано в программе.

В таблице 4.2 представлены возможности canvas.*create* в Python.

Таблица 4.2 – Возможности *canvas.create* в Python

Конструкция *canvas.create*	Использование	Пример
create_line()	Изображение линии или фигур из линий	canvas.create_line(15, 25, 200, 25) – изображение линии, имеющей стартовую (15,25) и конечную (200,25) координаты
		canvas.create_line(300, 35, 300, 200, dash=(4, 2)) – изображение линии с теми же начальной и конечной координатой, при этом опция dash позволяет формировать прерывистую линию (в данном случае линия будет состоять из одинаковых сегментов тире длинной 4 пикселя и сегментов пустого пространства длиной 2 пикселя.
		canvas.create_line(55, 85, 155, 85, 105, 180, 55, 85) – три линии, замыкающиеся в треугольник

Продолжение таблицы 4.2

Конструкция *canvas.create*	Использование	Пример
create_rectangle()	Изображение квадрата или прямоугольника	canvas.create_rectangle(230, 10, 290, 60)
canvas.create_oval ()	Изображение круга	canvas.create_oval(10, 10, 80, 80, outline="red", fill="green") – круг, внешняя линия которого красного цвета, заливка внутри – зеленая
canvas.create_polygon ()	Изображение многоугольника	points = [150, 100, 200, 120, 240, 180, 210, 200, 150, 150, 100, 200] canvas.create_polygon(points, outline='red', fill='green', width=2) – создание многоугольника
canvas.create_text()	Рисование текста	canvas.create_text(20, 30, font="Purisa", text="Most relationships seem so transitory") – рисование заданного текста заданным шрифтом от заданной координатной точки

Для выбора цветов при работе с графическими изображениями потребуется конструкция: *colors = choice(['цвет1'...'цветn']).*

Например, кодовая конструкция *colors = choice(['aqua', 'blue', 'fuchsia', 'green', 'maroon', 'orange', 'pink', 'purple', 'red','yellow', 'violet', 'indigo', 'chartreuse', 'lime'])* позволяет выбирать цвета из представленных в списке.

Примечание 1. При работе с графикой иногда требуется использование функций для работы с очередью задач. В Python предусмотрены две функции *update* и *update_idletasks* для работы с очередью задач. Их выполнение вызывает обработку отложенных задач. Функция *update_idletasks* выполняет задачи, обычно откладываемые «на потом», когда приложение будет простаивать. Это приводит к прорисовке всех виджетов, расчёту их расположения и прочее. Обычно эта функция используется если были внесены изменения в состояние приложения, и пользователь хочет, чтобы эти изменения были отображены на экране немедленно, не дожидаясь завершения сценария. Функция *update* обрабатывает все задачи, стоящие в очереди. Обычно эта функция используется во время сложных расчётов или громоздкой графики, когда необходимо чтобы приложение оставалось отзывчивым на действия пользователя.

Примечание 2: Для организации изображений на холсте иногда требуется организация циклов. Например, выполнять рисование объектов, пока значение переменной не превысит некоторого заданного значения.

С этой целью (а также для решения множества других задач) могут потребоваться операторы присваивания.

Например, кодовая конструкция

```
diapason = 0
while diapason < 2000:
    colors = choice(['aqua', 'blue', 'fuchsia', 'green', 'maroon', 'orange', 'pink', 'purple', 'red','yellow', 'violet', 'indigo', 'chartreuse', 'lime'])
```

x = randint(0, size)
y = randint(0, size)
d = randint(0, size/3)
canvas.create_oval(x, y, x+d, y+d, fill=colors)
diapason += 1

позволит формировать на холсте круги разного цвета пока переменная *diapason* не превысит значение 2000, при этом переменная *diapason* каждый раз обновляется на 1 9к предыдущему значению переменной *diapason* прибавляется 1).

В таблице 4.3 представлены операторы присваивания в Python.

Таблица 4.3 – Операторы присваивания в Python

Оператор	Описание	Примеры
=	Присваивает значение правого операнда левому	c = 23 присвоит переменной c значение 23
+=	Прибавит значение правого операнда к левому и присвоит эту сумму левому операнду	c = 5, a = 2, c += a равносильно: c = c + a. c будет равно 7
-=	Отнимает значение правого операнда от левого и присваивает результат левому операнду	c = 5, a = 2, c -= a равносильно: c = c - a. c будет равно 3
*=	Умножает правый операнд с левым и присваивает результат левому операнду	c = 5, a = 2, c *= a равносильно: c = c * a. c будет равно 10
/=	Делит левый операнд на правый и присваивает результат левому операнду	c = 10, a = 2 , c /= a равносильно: c = c / a. c будет равно 5
%=	Делит по модулю операнды и присваивает результат левому	c = 5, a = 2, c %= a равносильно: c = c % a. c будет равно 1
**=	Возводит в левый операнд в степень правого и присваивает результат левому операнду	c = 3, a = 2, c **= a равносильно: c = c ** a. c будет равно 9
//=	Производит целочисленное деление левого операнда на правый и присваивает результат левому операнду	c = 11, a = 2, c //= a равносильно: c = c // a. c будет равно 5

Пример 10

Разработать программу, использующую методы графического модуля *tkinter*. Программа должна на холсте формировать изображения окружностей в течение бесконечного интервала времени. Размеры и положение окружностей – произвольные.

Выполнение примера 10

На рисунке 4.1 показан код программы к примеру 10.

```
grafica.py - C:\Users\an111\Documents\Все для Питона\К лабораторной 4\grafica.py (3.7.1)
File  Edit  Format  Run  Options  Window  Help
from random import * # импорт всех методов модуля random
from tkinter import * # импорт библиотеки, вернее всех ее методов, на что указывает звездочка (*)

size = 600  # Переменная размера окна
root = Tk()  # Создание окно
canvas = Canvas(root, width=size, height=size) # Создание холст, используя значение переменной size
canvas.pack() # указание расположить холст внутри окна
diapason = 0  # переменная diapason для расчета цикла
while diapason < 2000:   # с предусловием, которое говорит о том, что цикл будет повторятся пока переменная diapason не дойдет до 2000
    colors = choice(['aqua', 'blue', 'fuchsia', 'green', 'maroon', 'orange',
                     'pink', 'purple', 'red','yellow', 'violet', 'indigo', 'chartreuse', 'lime'])
    x = randint(0, size)   # случайный выбор координат x и y в рамках холста размером size
    y = randint(0, size)
    d = randint(0, size/3)  # произвольный выбор размера круга, ограниченный size/5
    canvas.create_oval(x, y, x+d, y+d, fill=colors) # рисуем круги, в точках с координатами x и y,
    # размерами по вертикали и горизонтали x0+d и y0+d,
    # заливкой цветом, который выбирается случайным образом из списка colors
    root.update() # обрабатывает все задачи, стоящие в очереди.
    # Обычно эта функция используется во время «тяжёлых» расчётов,
    # когда необходимо чтобы приложение оставалось отзывчивым на действия пользователя
    diapason += 1 # шаг цикла, счетчик
while True:
    colors = choicecolors = choice(['aqua', 'blue', 'fuchsia', 'green', 'maroon', 'orange',
                     'pink', 'purple', 'red','yellow', 'violet', 'indigo', 'chartreuse', 'lime'])
    x0 = randint(0, size)
    y0 = randint(0, size)
    d = randint(0, size/3)
    canvas.create_oval(x0, y0, x0+d, y0+d, fill=colors )
    root.update()
```

Рисунок 4.1 – Код программы к примеру 10

Примечание 3: Размер окна должен быть кратен числу 3 (соотношение размера окна и размера окружностей) или размер окружностей должен быть делителем размера окна, так как программа работает с целыми числами (центры окружностей).

Примечание 4: Программа работает с двумя циклами. Первый цикл работает до тех пор, пока переменная *diapason* не превысит значение 2000. Второй цикл работает бесконечно, так как использование кодовой конструкции *root.update()* приводит к регулярному обновлению окна программы.

На рисунке 4.2 представлен результат работы программы к примеру 10.

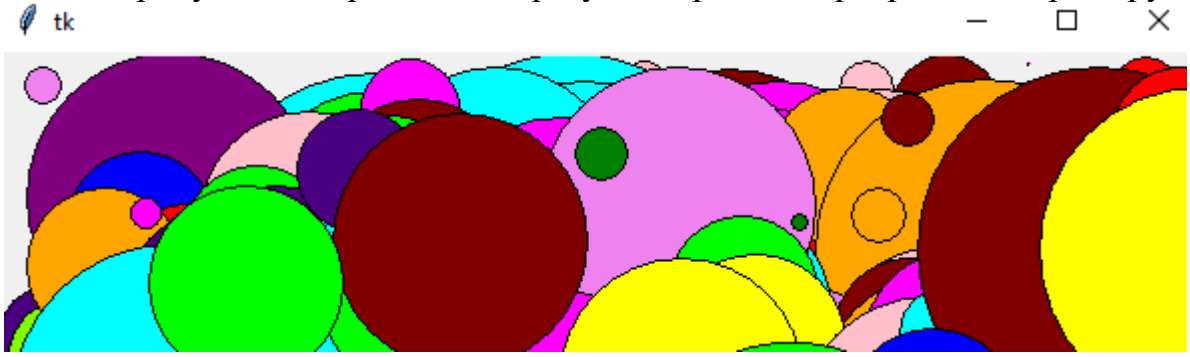


Рисунок 4.2 – Результат работы программы к примеру 10

4.3. Оборудование и программное обеспечение

1. Персональная ЭВМ.
2. Операционная система Windows/Linux.
3. Среда программирования IDLE или другая среда, содержащая язык программирования Python.

4.4. Задание на работу

1. Изучить теоретические положения лабораторной работы.
2. Получить результаты работы программ в соответствии с примером 10.

3. Для примера 10:

а) произвольно изменить размер окна;

б) произвольно изменить размеры окружностей;

в) произвольно задать контуры окружностей (сделать их другим цветом в сравнении с заливкой окружностей);

г) произвольно изменить границы цикла *while diapason < 2000* и описать, что произойдет с работой программы.

4. Разработать программу для изображения линий произвольной длины и цвета.

5. Разработать программу для изображения треугольников произвольного размера и цвета.

6. Разработать программу для изображения квадратов (прямоугольников) произвольного размера и цвета.

7. Разработать программу для изображения многоугольников произвольного размера и цвета.

8. Разработать программу для изображения (рисования) произвольного текста.

9. Изобразить схемы работы программ в соответствии с ГОСТ 19.701-90 (использовать графический редактор).

10. Показать результаты работы преподавателю.

11. Ответить на контрольные вопросы.

12. Оформить отчет.

4.5. Контрольные вопросы

1. Какая библиотека используется для реализации графических возможностей на языке Python?

2. Какие способы существуют для импорта библиотеки *tkinter* в среду разработки Python?

3. Как создать окно и задать его размеры в Python?

4. Что такое холст и как его создать в Python?

5. Как вписать холст в окно?

6. Какие возможности предусмотрены в Python при работе с модулем *random*?

7. Какие графические объекты можно создать на холсте в Python?

8. Как создаются графические объекты в Python?

9. Какие операторы присваивания можно использовать при работе с графическими объектами в Python?

10. Какую работу в Python организуют функции *update* и *update_idletasks?*

4.6. Правила оформления отчета

Отчет должен содержать:

1. Номер, название и цель лабораторной работы.

2. Описание процесса выполнения примеров (коды, схемы и результаты работы программ).

3. Вывод по работе.

Лабораторная работа №5
Работа с файлами на языке Python
5.1. Цель лабораторной работы

Научиться работать с файлами с помощью языка Python

5.2 Теоретические положения

Работа с файлами на языке Python позволяет пользователям следующие возможности:

- создание файлов различных типов (с различным расширением);
- открытие файлов;
- запись информации в файлах после обработки ее приложением;
- чтение информации, записанной в файле.

В Python предусмотрен ряд функций, которые позволяют достаточно легко организовать указанные действия.

Создание файлов

Для создания файлов формата .txt в Python (средствами языка, а не средствами операционной системы) используется метод open. Методу open передаются такие параметры как название файла и режим его обработки.

Например, код программы

```
file = open("test.txt")
file.close()
```

позволяет создать файл с расширением .txt в папке, где хранится исходный код программы (предусмотрен, так называемый, относительный путь к файлу).

В представленном фрагменте кода программы методу open передан только один параметр – имя файла (второй параметр – режим работы с файлом не передан).

Если же файл с указанным именем в текущей папке (в каталоге с кодом) уже существует, программа продолжит работу с ним, не создавая новый файл.

Следует помнить, что в Python после выполнения любых манипуляций над файлом, его обязательно следует закрыть, чтобы избежать потери информации. Закрытие файла осуществляется с помощью функции close (), что и продемонстрировано в представленном выше фрагменте кода программы.

Для указания полного пути к файлу, первый параметр метода open должен содержать полный путь с указанием диска и каталогов.

Например, код программы

```
file = open("C:\test.txt")
file.close()
```

позволит создать файл на диске С.

Открытие файлов

В Python предусмотрены специальные символы, которые используются при открытии файла. Специальные символы – это второй аргумент метода open. Специальные символы задают режим открытия файла, то есть указывают программе, как именно нужно открыть файл.

В таблице 5.1 представлены специальные символы, используемые при открытии файлов в Python.

Таблица 5.1 – Специальные символы, используемые при открытии файлов в Python

Специальный символ	Предназначение символа
r	открытие для чтения (по умолчанию)
w	открытие для записи, а если его не существует по заданному пути, то создается новый
x	открытие для записи, но только если его еще не существует, иначе будет выдано исключение (ошибка)
a	открытие на дополнительную запись, чтобы информация добавлялась в конец документа
b	открытие в двоичном режиме
t	открытие в текстовом режиме (по умолчанию)
+	открытие одновременно на чтение и запись

В Python можно комбинировать различные режимы работы с файлами. Например:

- "rb" – чтение записанные данных в двоичном режиме;
- "r+" – открытие файла на чтение и на запись (создается новый файл, если такого нет);
- "w+" – открытие файла на запись и чтение (создается новый файл, если такого нет).

Запись информации в файл

В Python запись в файл осуществляется с помощью метода write. Аргументы метода write – это данные, которые нужно поместить в текстовый файл. При использовании метода write предварительно следует открыть документ с помощью функции open и указать режим записи символом «w» (открытие для записи).

Например, код программы

```
file = open("test.txt", "w")
file.write("privet")
file.close()
```

позволяет записать в файл слово privet.

Если файл предварительно создан, то новый файл не создается.

На рисунке 5.1 показан результат работы программы для записи в текстовый файл слова privet.

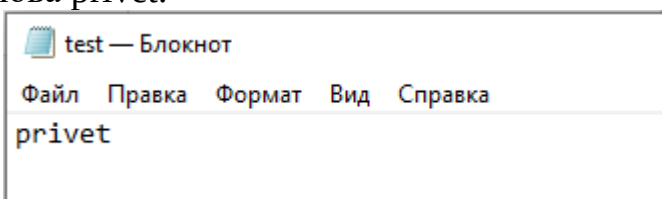


Рисунок 5.1 – Результат работы программы для записи в текстовый файл слова privet

Если необходимо добавить новую информацию к записанным ранее данным, то следует заново вызвать функцию open, указав ей в качестве режима работы символ "a". В противном случае все сведения из файла test.txt будут полностью удалены.

Например, код программы

```
file = open("test.txt", "a")
file.write(" student")
file.close()
```

добавит в ранее созданный файл, содержащий слово privet, слово student, разделив слова пробелом.

Результат работы программы по добавлению слова student представлен на рисунке 5.2.

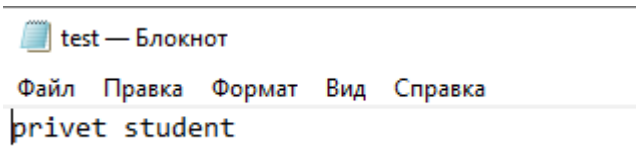


Рисунок 5.2 – Результат работы программы по добавлению слова student

Совмещение кодовых конструкций в одну, например, так

```
file = open("test.txt", "w")
file.write("privet")
file.close()
file = open("test.txt", "a")
file.write(" student")
file.close()
```

приведет к такому же результату (рисунок 29).

Чтение информации из файла

В Python для чтения информации из файла следует вызывать метод read, при этом второй параметр метода open при открытии файла будет представлять собой литеру "r".

Например, следующий код программы

```
file = open("test.txt", "r")
print(file.read())
file.close()
```

позволяет вывести информацию из файла.

Однако здесь часто возникают исключения (ошибки), так как файл может быть не найден. Поэтому при чтении информации из файла требуется обработка исключения, связанная с поиском требуемого файла.

Например, следующий код программы

```
try:
    file = open("test.txt", "r")
    print(file.read())
    file.close()
except FileNotFoundError:
    print('Not found')
```

except IOError:

print('Something else')

позволяет вывести информацию из ранее созданного и обработанного путем записи в него информации файла.

Если файл не будет найден, то на экран будет выведена информация: *'Not found', 'Something else' (Не найден, что-то еще?).*

В представленном коде конструкция *try-except* – это конструкция, обрабатывающая исключение (здесь конструкция *try-except* необходима на случай, если запрашиваемый файл не будет найден).

Результат работы кода программы для чтения из файла информации представлен на рисунке 5.3.

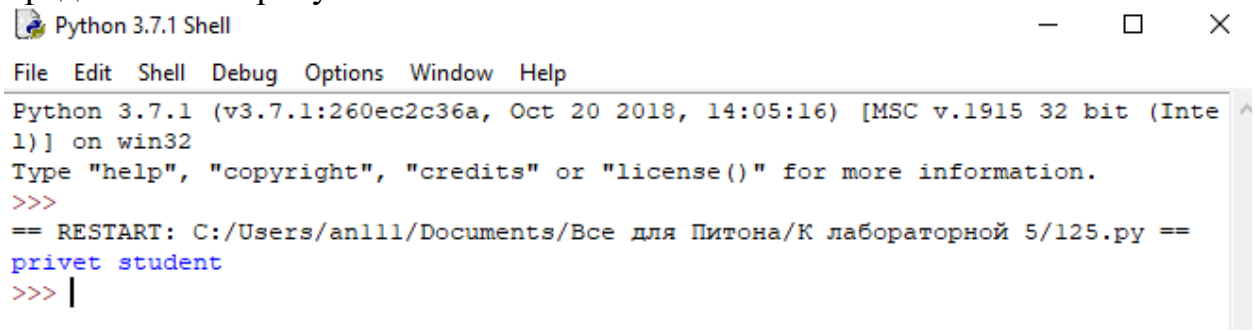


Рисунок 5.3 – Результат работы кода программы для чтения из файла информации

Дополнительные методы для работы с файлами

Объект, который возвращает функция open, содержит ссылку на существующий файл. В объекте метода open имеется информация о созданном документе, которая представлена четырьмя основными полями:

- имя файла (первый параметр метода open);
- режим открытия файла (второй параметр метода open);
- информация о закрытии/открытии файла;
- информация о требовании/не требовании пробела при выводе информации из файла.

В таблице 5.2 представлено описание полей, сопровождающих процесс работы с файлами.

Таблица 5.2 – Описание полей, сопровождающих процесс работы с файлами

Поле	Принцип работы
name	возвращает имя файла
mode	возвращает режим, в котором был открыт
closed	возвращает true, если файл закрыт
softspace	возвращает true, если при выводе данных из файла не следует отдельно добавлять символ пробела

Например, код программы

```
f = open("test.txt", "w")
print(f.name)
f.close()
```

позволяет вывести полное имя файла.

В представленном коде программы используется оператор доступа, то есть точка, записанная после переменной, которая отвечает за обрабатываемый файл (здесь переменная f). Чтобы вывести на экран свойства файла, доступ указывается в качестве параметра функции print.

Результат работы кода программы для вывода имени файла представлен на рисунке 5.4.

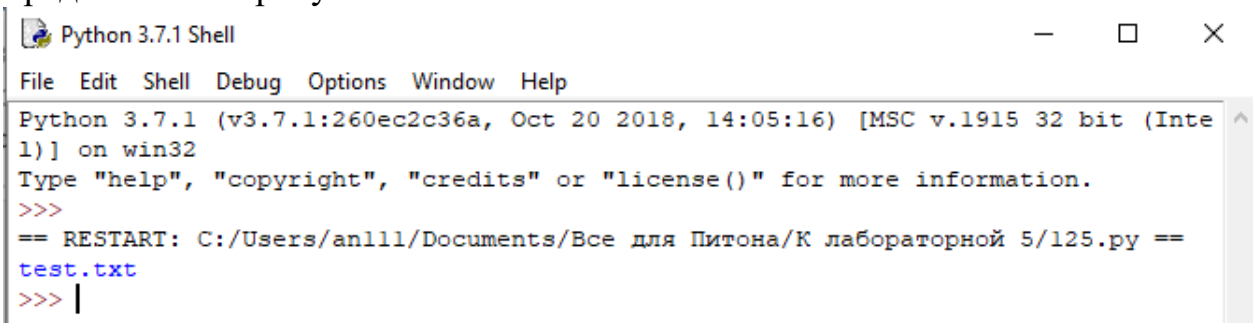


Рисунок 5.4 – Результат работы кода программы для вывода имени файла

Работа с бинарными данными

Бинарные файлы в отличие от текстовых хранят информацию в виде набора байт. При записи бинарных данных, следует использовать режим "wb".

Например, код программы

```
f = open('laba5.dat', 'wb')
f.write(bytes('программирование', 'utf8'))
f.close()
```

позволяет записать слово «программирование» в файл с расширением .dat (бинарный нетипизированный файл).

В представленном коде программы – utf8 – кодировка данных, которая позволит прочитать файл, как текстовый.

UTF-8 (Unicode Transformation Format, 8-bit – «формат преобразования Юникода, 8-бит») – стандарт кодирования текста, позволяющий более компактно хранить и передавать символы Юникода, совместим с ASCII.

После запуска программы для создания бинарного файла, получены соответствующий файл (рисунок 5.5).

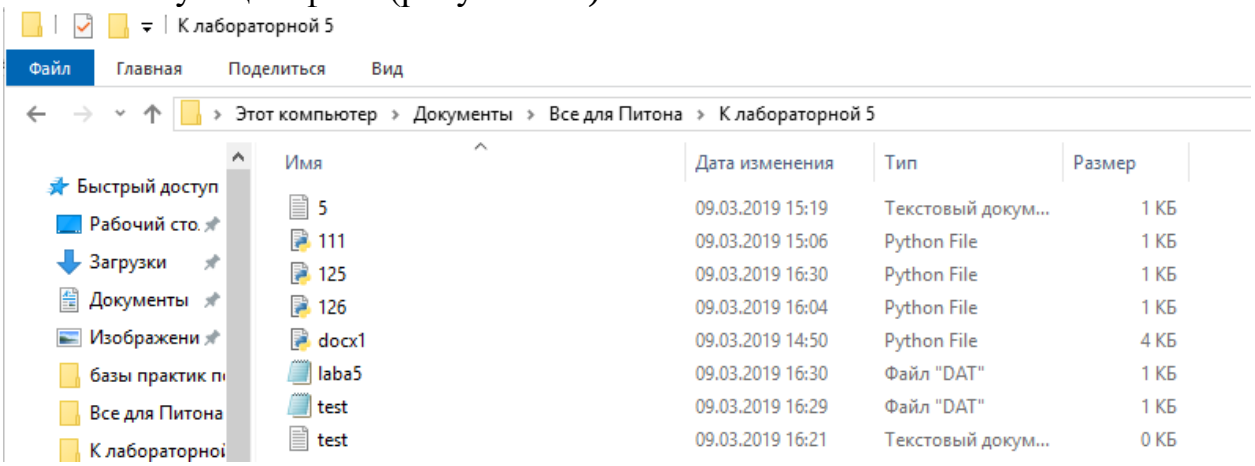


Рисунок 5.5 – Результат работы программы для создания бинарного файла

Открытие бинарного файла в программе Блокнот представлено на рисунке 5.6.

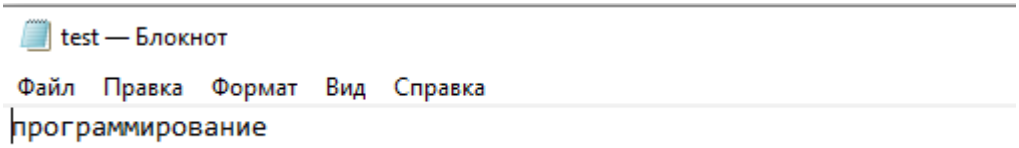


Рисунок 5.6 – Открытие бинарного файла в программе Блокнот

Чтение бинарных данных осуществляется с использованием литер "rb" в методе open.

Например, код программы

```
try:
    f = open("laba5.dat", "rb")
    b = f.read(1)
    str = ""
    while True:
        b = f.read(1)
        if b == b'':
            break
        str += b.hex()
    print(str)
    f.close()
except IOError:
    print('error')
```

позволяет побайтно считать информацию из файла laba5.dat.

Результат работы программы по считыванию информации из бинарного файла представлен на рисунке 5.7.

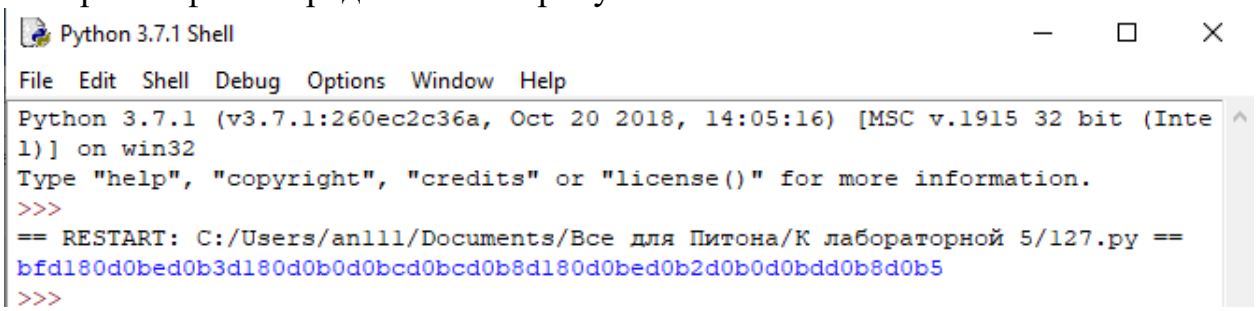


Рисунок 5.7 – Результат работы программы по считыванию информации из бинарного файла

Здесь файл читается побайтно в шестнадцатеричном представлении.

Конструкция with as

Для упрощения работы текстовых файлов рекомендуется использовать связку операторов через конструкцию with as. Это позволит не использовать метод close.

Например, код программы

```
with open('test1.txt', 'r') as file:
    print(file.read())
```

осуществляет считывание данных из test.txt.

Результат работы программы для считывания информации из текстового файла представлен на рисунке 5.8.

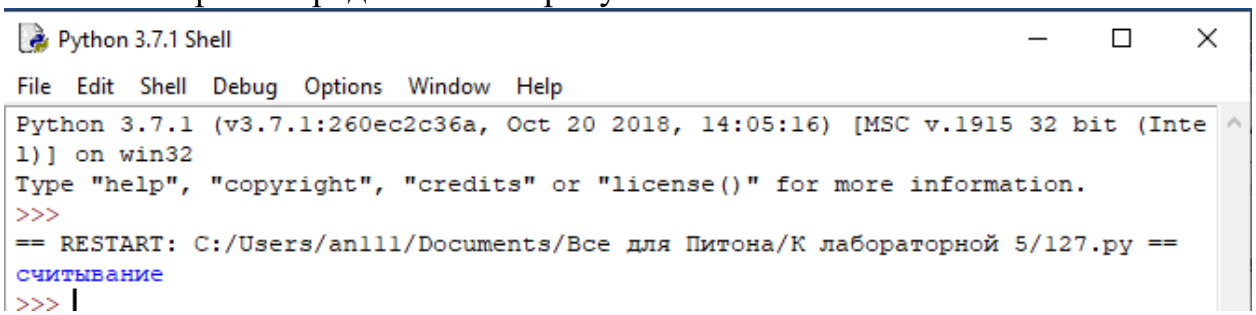


Рисунок 5.8 – Результат работы программы для считывания информации из текстового файла

Пример 11

Разработать программу, которая определяет результат сложения двух переменных, записывает этот результат в текстовый файл и считывает свойства этого файла (имя и режим работы с ним).

Выполнение примера 11

На рисунке 5.9 показан код программы к примеру 11.

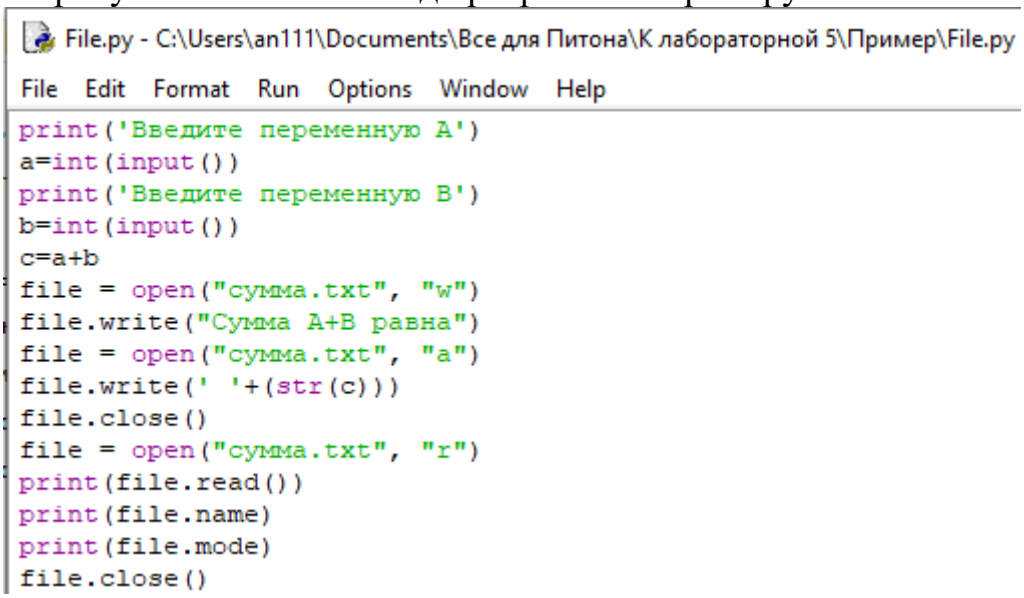


Рисунок 5.9 – Код программы к примеру 11

Примечание 1: Вначале файл открыт для записи строки «Сумма A+B равна». Далее в файл добавляется результат сложения переменных A, B, то есть данные, хранящиеся в переменной C. Затем из файла считывается информация относительно имени файла (name) и режима работы с ним (mode).

На рисунке 5.10 представлен результат работы программы к примеру 11.

На рисунке 5.11 представлен файл, обработанный в соответствии с примером 11.

На рисунке 5.12 представлен открытый файл в соответствии с примером 11.

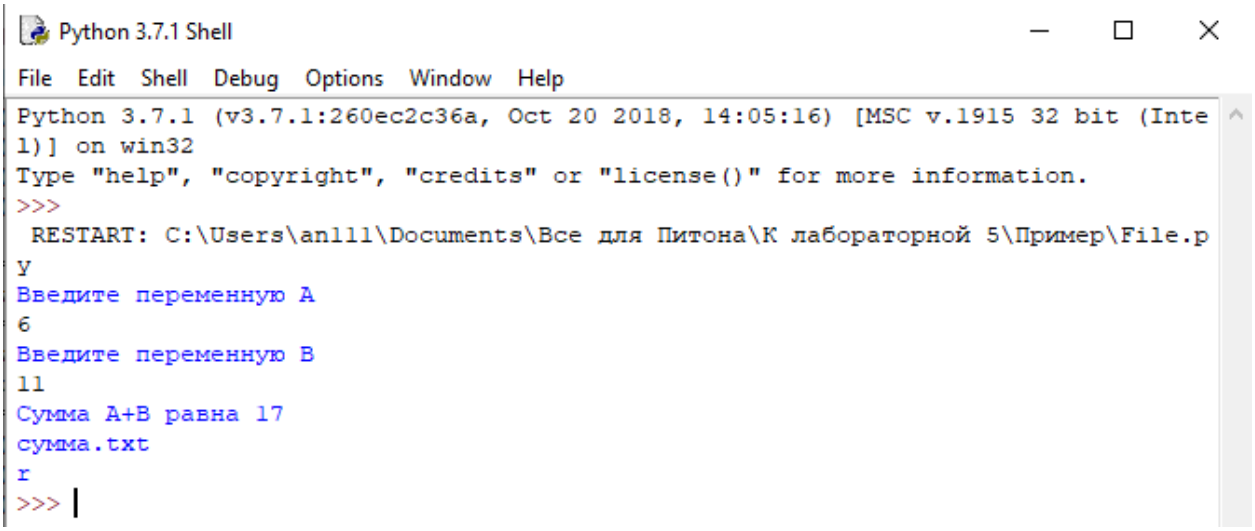


Рисунок 5.10 – Результат работы программы к примеру 11

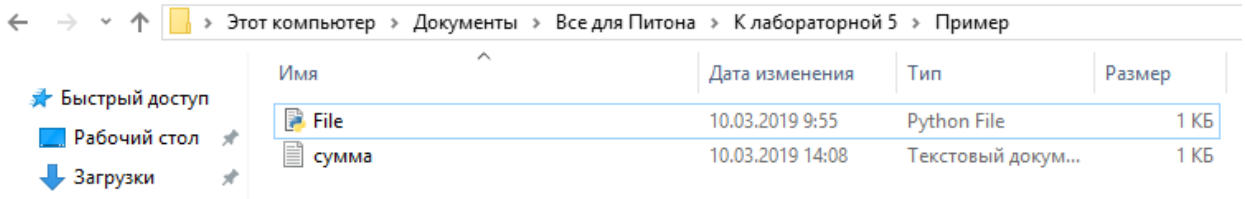


Рисунок 5.11 – Файл, обработанный в соответствии с примером 11

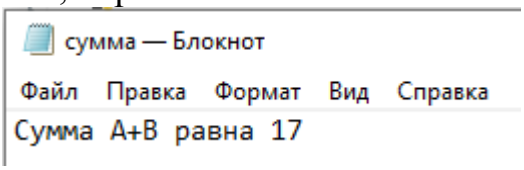


Рисунок 5.12 – Открытый файл в соответствии с примером 11

5.3. Оборудование и программное обеспечение

1. Персональная ЭВМ.
2. Операционная система Windows/Linux.
3. Среда программирования IDLE или другая среда, содержащая язык программирования Python.

5.4. Задание на работу

1. Изучить теоретические положения лабораторной работы.
2. Получить результаты работы программы в соответствии с примером 11.
3. Разработать программу, которая:

- для двух вводимых с клавиатуры переменных получает результаты алгебраических операций (разности, сложения, умножения и частного). Результаты записываются в текстовый файл.

4. Изменить программу так, чтобы результаты алгебраических операций записывались в различные четыре текстовых файла.
5. Для каждого из четырех файлов вывести информацию:

- об имени файла (поле name);
- о режиме работы с файлом (поле mode);
- об открытии/закрытии файла (поле closed).

6. Дополнить каждый файл тестовой информацией (ввести в файл информацию, комментирующую результат операции для чего использовать литеру "a", то есть открытие на дополнительную запись, чтобы информация добавлялась в конец документа).

7. Считать информацию из четырех файлов побайтно.

8. Разработать программу, которая вычисляет значение функции вида $f = \begin{cases} \frac{x^2}{3}, \text{при x} > 0 \\ 3x^2, \text{при x} \leq 0 \end{cases}$, если x$\in$ [−К; К], шаг – 1, где К – номер студента в списке группы. Результаты работы программы записать в текстовый файл. Применить метод для считывания информации из файла.

9. Изобразить схемы работы программ в соответствии с ГОСТ 19.701-90 (использовать графический редактор).

10. Показать результаты работы преподавателю.

11. Ответить на контрольные вопросы.

12. Оформить отчет.

5.5. Контрольные вопросы

1. Как в Python создать, открыть файл? Какие для этого используются методы?

2. Какие в Python параметры сопровождают метод open?

3. Какими литерами в Python представлен второй параметр метода open?

4. Как в Python записать информацию в файл (текстовый, бинарный)?

5. Как в Python считать информацию из файла?

6. Как упростить процесс работы с файлами в Python? Какие для этого используются кодовые конструкции?

7. Как вывести на экран информацию, которая сопровождает процесс обработки файлов в Python?

8. Как в Python организована работа с бинарными файлами?

9. Как в Python считать побайтно информацию из файла?

10. Как в Python управлять расстановкой пробелов при считывании информации из файла?

5.6. Правила оформления отчета

Отчет должен содержать:

1. Номер, название и цель лабораторной работы.

2. Описание процесса выполнения примеров (коды, схемы и результаты работы программ).

3. Вывод по работе.

Лабораторная работа №6
Работа в Python с документами MS Office, Libre Office (Open Office)
6.1. Цель лабораторной работы

Научиться в Python работать с документами
MS Office, Libre Office (Open Office)

6.2 Теоретические положения

Для работы в Python с документами предусмотрены следующие библиотеки:

1. Для работы с документом MS Word - библиотеки .docx или python-docx-template;

2. Для работы с документом MS Excel – библиотеки xlrd, xlwt, xlutils или openpyxl.

Цель указанных библиотек – создавать документы форматов doc docx xsl, xslx, xml и управлять работой с названными документами.

Для того, чтобы можно было работать с документами MS Office (MS Word, MS Excel) необходимо установить названные библиотеки.

Чтобы установить библиотеку .docx, необходимо в папку, где хранится модуль pip, добавить соответствующую библиотеку.

Примечание: pip – это система управления пакетами, которая используется для установки и управления программными пакетами, написанными на Python. Для открытия папки, содержащей модуль pip, необходимо вызвать командную строку (поиск, cmd). Затем ввести команду cd (смена директории), ввести пробел и вставить путь к модулю pip (путь к модулю pip необходимо предварительно скопировать, например, из программы Total Commander).

Затем после перехода в папку модуля pip необходимо указать команду install, пробел, docx.

На рисунке 6.1 показан путь к модулю pip, который хранится в папке Scripts.

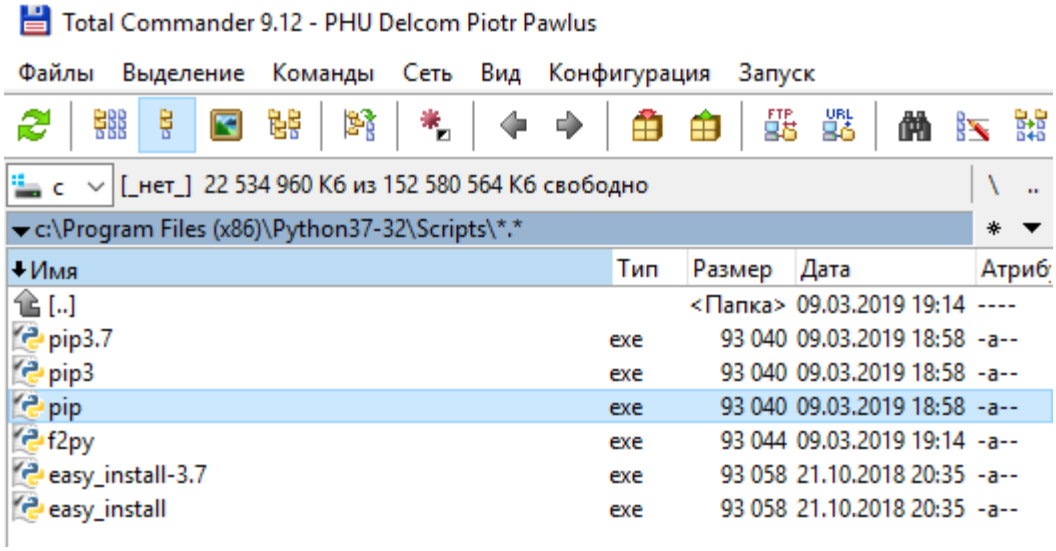


Рисунок 6.1 – Путь к модулю pip, который хранится в папке Scripts

На рисунке 6.2 показаны команды для установки библиотеки docx (в командной строке).

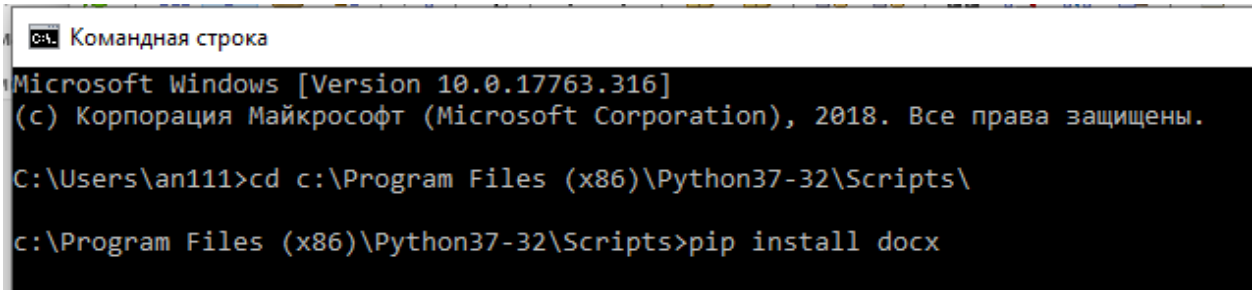


Рисунок 6.2 – Команды для установки модуля docx (в командной строке)

Аналогичным образом устанавливаются другие библиотеки.

Иногда возникает необходимость программно работать с документами Libre Office (Open Office). Для этих целей в Python есть инструмент – Python-UNO.

Если Libre Office (Open Office) уже установлен, нужно установить соответствующую библиотеку для работы с документами. В командной строке вводится команда install python-uno (для операционных систем семейства Linux - apt-get install python-uno).

Работа с документами Open Office осуществляется следующим образом:

- запускается Libre Office (Open Office) так, чтобы он слушал порт на локальном хосте;
- пишется команда на Python, которая посылает команды на запущенный LibreOffice;
- запускается эта программа и осуществляется работа с документом.

Например, скрипт (сценарий) для запуска LibreOffice может иметь вид:

```
#! /bin/bash
/usr/bin/libreoffice - accept="socket,host=localhost,port=2002;
urp;StarOffice.ServiceManager" \-norestore-nofirstwizard-nologo
exit 0
```

После написания скрипта и его запуска можно начинать работу в Python по созданию документов в Libre Office (Open Office).

Например, код программы, написанной в Python, для создания нового документа, записи в него строки и закрытия документа может иметь вид:

```
1 #! /usr/bin/python
 2 # -*- coding: utf-8 -*-
 3
 4 import sys,uno
 5
 6 local = uno.getComponentContext()
 7 resolver =
local.ServiceManager.createInstanceWithContext("com.sun.star.bridge.UnoUrlRe
solver", local)
 8 context =
resolver.resolve("uno:socket,host=localhost,port=2002;urp;StarOffice.Component
Context")
```

```
 9 desktop =
context.ServiceManager.createInstanceWithContext("com.sun.star.frame.Desktop"
, context)
10
11 url = "private:factory/swriter"
12 document = desktop.loadComponentFromURL(url, "_blank", 0, ())
13 cursor = document.Text.createTextCursor()
14
15 text = "This text is being added to openoffice using python and uno
package.\n\n"
16 document.Text.insertString(cursor, text, 0)
17
18 document.storeAsURL("file:///home/evgeny/tmp/002.odt",())
19 document.dispose()
20
21 sys.exit()
```

Так же, например, код программы, написанной в Python, для открытия файла Word, вывода на консоль количества символов, записанных в этот документ, и закрытия документа, может иметь вид:

```
 1 #! /usr/bin/python
 2 # -*- coding: utf-8 -*-
 3 # Copyright Python
 4 """
 5 """
 6
 7 import sys,uno
 8
 9 local = uno.getComponentContext()
10 resolver =
local.ServiceManager.createInstanceWithContext("com.sun.star.bridge.UnoUrlRe
solver", local)
11 context =
resolver.resolve("uno:socket,host=localhost,port=2002;urp;StarOffice.Component
Context")
12 desktop =
context.ServiceManager.createInstanceWithContext("com.sun.star.frame.Desktop"
, context)
13 url = "file:////home/evgeny/tmp//001.doc"
14 document = desktop.loadComponentFromURL(url,"_blank", 0, ())
15 print document.CharacterCount
16 document.dispose()
17 sys.exit()
```

Примечание: для удобства считывания кода, строки пронумерованы.

Пример 12

Разработать программу, которая позволяет управлять работой с документом MS Word (формат docx), а именно:

- формирует заголовки различного уровня;
- позволяет использовать полужирное выделение текста и выделение текста курсивом;
- формирует нумерованный список;
- формирует маркированный список;
- позволяет вставить таблицу;
- позволяет вставить рисунок;
- позволяет вставить разрыв страницы.

Выполнение примера 12

На рисунке 6.3 показан код программы к примеру 12.

docx1.py - C:\Users\an111\Documents\Все для Питона\К лабораторной 6\docx1.py (3.7.1)

File Edit Format Run Options Window Help

```
from docx import Document
from docx.shared import Inches

document = Document() #присваиваем переменной разрешение работы с документом
document.add_heading('Начало работы с документами',0) #Создаем заголовок самого верхнего уровня
text=document.add_paragraph("Этот текст раздела дополняется словами ")   #Создаем текст под заголовком (абзац)
text.add_run('полужирным').bold = True
text.add_run(' шрифтом и ')
text.add_run('курсивом.').italic = True
tx=('После абзаца можно привести пример добавления текста через переменную, с использованием нового стиля.')
te=(' Например список с цифровыми или точечными маркерами.')
document.add_paragraph(tx,style='List Number')      #Новый абзац
document.add_paragraph(te,style='List Number')      #Новый абзац
document.add_paragraph('Далее приведем пример того же самого, но с точкой')
document.add_paragraph(tx,style='List Bullet')      #Новый абзац
document.add_paragraph(te,style='List Bullet')      #Новый абзац
document.add_heading('Заголовок первого уровня, например для таблицы', level=1)


records = (
    (3, '101', 'Компьютер'),
    (7, '422', 'Сканер'),
    (4, '631', 'Монитор')
)                                          #создаем словарь для записи в таблицу

table=document.add_table(rows=1, cols=3, style='Light Grid') #создаем таблицу с 1 строкой и 3 столбцами
hdr_cells = table.rows[0].cells  #присвоение переменной нулевой строки
hdr_cells[0].text = '№ п/п'
hdr_cells[1].text = 'Код товара'        #запись в соответствующие ячейки 0-й строки
hdr_cells[2].text = 'Наименование'
for qty, id, desc in records:
    row_cells = table.add_row().cells #добавляет строку в таблице
    row_cells[0].text = str(qty)
    row_cells[1].text = id            #запись в соответствующие ячейки данной строки
    row_cells[2].text = desc

document.add_paragraph()              #Вставка пустой строки для разрыва таблицы и рисунка
place=('IMG_20170710_194103.jpg') # Указываем название картинки картинки, файл должен располагаться в той же папке
document.add_picture(place, width=Inches(3))#Добавляем картинку в файл где width=Inches(3) это размер картинки
document.add_page_break()# разрыв страницы
document.save('DEMO1.docx') #Сохранение файла
```

Рисунок 6.3 – Код программы к примеру 12

Примечание: Рисунок в формате jpg должен храниться в папке, в которой формируется документ.

На рисунке 6.4 показан результат создания файла формата docx в соответствии с примером 12 (в папке так же хранится файл с кодом программы и рисунок в формате jpg).

На рисунке 6.5 показан открытый файл формата docx, обработанный при помощи инструментов Python (в соответствии с примером 12).

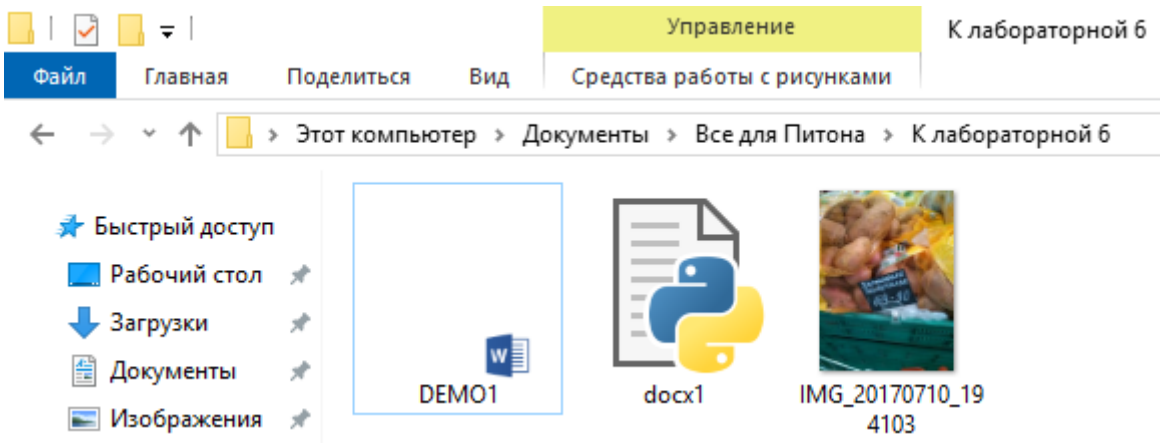


Рисунок 6.4 – Результат создания файла формата docx
в соответствии с примером 12



Рисунок 6.5 – Открытый файл формата docx, обработанный при помощи инструментов Python (в соответствии с примером 12)

Примечание: пустой второй лист иллюстрирует разрыв страницы.

6.3. Оборудование и программное обеспечение

1. Персональная ЭВМ.
2. Операционная система Windows/Linux.
3. Среда программирования IDLE или другая среда, содержащая язык программирования Python.
4. MS Office, Libre Office (Open Office).

6.4. Задание на работу

1. Изучить теоретические положения лабораторной работы.
2. Получить результаты работы программы в соответствии с примером 12.
3. Разработать программу, которая создает документ MS Word (формата doc, docx), в который средствами языка Python:

- вставляется заголовок – группа и ФИО студента (группа студента выделена полужирным шрифтом, ФИО – курсивом);

- вставляется заголовок таблицы (произвольно, в соответствии с содержимым таблицы);

- вставляется таблица, содержащая 5 столбцов и 7 строк, хранящая произвольные данные;

- список записей из полей таблицы (нумерованный или маркированный);

- произвольный рисунок;

- два разрыва страницы.

4. Разработать программу, аналогичную п. 3 задания для текстового документа Libre Office (Open Office).

5. Изобразить схемы работы программ в соответствии с ГОСТ 19.701-90 (использовать графический редактор).

6. Показать результаты работы преподавателю.

7. Ответить на контрольные вопросы.

8. Оформить отчет.

6.5. Контрольные вопросы

1. Какие библиотеки используются в Python для работы с документами MS Word?

2. Какие библиотеки используются в Python для работы с документами MS Excel?

3. Какая библиотека используются в Python для работы с документами Libre Office (Open Office)?

4. Как осуществляется управление документом Libre Office (Open Office) средствами Python?

5. К какому модулю подключаются библиотеки для работы с документами?

6. Как установить библиотеки?

7. Как создать в документе средствами Python заголовки различного уровня?

8. Как создать в документе средствами Python нумерованные и маркированные списки?

9. Как создать в документе средствами Python таблицу?

10. Как вставить в документ средствами Python изображение (рисунок)?

11. Как создать в документе средствами Python разрыв страницы?

12. Как форматировать в документе текст средствами Python?

6.6. Правила оформления отчета

Отчет должен содержать:

1. Номер, название и цель лабораторной работы.

2. Описание процесса выполнения примеров (коды, схемы и результаты работы программ).

3. Вывод по работе.

Лабораторная работа №7
Графические возможности языка Python (виджеты)
7.1. Цель лабораторной работы

Научиться создавать графический интерфейс программ на языке Python

7.2 Теоретические положения

Современный пользователь в основном взаимодействует с программой с помощью различных кнопок, меню, значков, вводя информацию в специальные поля, выбирая определенные значения в списках и т. д. Эти «изображения» в определенном смысле формируют графический интерфейс пользователя (GUI от английского graphical user interface). Изображения GUI принято называть виджетами (от англ. widget – «штучка»).

Для языка программирования Python такие виджеты включены в специальную библиотеку – tkinter. Если ее импортировать в программу (скрипт), то можно пользоваться ее компонентами, создавая графический интерфейс.

Последовательность шагов при создании графического приложения имеет свои особенности. Чтобы получить программу с GUI необходимо осуществить следующие действия:

1. Импорт библиотеки.
2. Создание окна программы.
3. Создание виджетов.
4. Установка свойств виджетов.
5. Определение событий, которые будут обрабатываться с помощью виджетов.
6. Определение обработчиков событий для виджетов.
7. Расположение виджетов в окне программы.
8. Отображение окна программы.

Импорт библиотеки (импорт модуля tkinter)

Как и любой модуль, tkinter в Python можно импортировать двумя способами: командами import tkinter или from tkinter import.

Удобнее пользоваться вторым способом, поскольку это позволяет не указывать каждый раз имя модуля при обращении к объектам, которые в нем содержатся. Следует обратить внимание, что в версии Python 3 имя модуля пишется со строчной буквы (tkinter), хотя в более ранних версиях Python использовалась прописная (Tkinter).

Создание окна программы

В современных операционных системах любое пользовательское приложение заключено в окно, которое можно назвать главным, т.к. в нем располагаются все остальные виджеты. Объект окна верхнего уровня создается при обращении к классу Tk модуля tkinter. Переменную, связанную с объектом-окном, принято называть root (можно назвать как угодно, но так принято среди разработчиков программ на языке Python). Строчка кода, отвечающая за создание окна имеет вид: *root = Tk()*.

Окна в tkinter создаются классами Tk и Toplevel. Класс Tk позволяет, как правило, создавать главное окно. Если создается многооконное приложение, то остальные окна создаются с использованием класса Toplevel.

Методы классов (виджетов) Tk и Toplevel

Методы классов (виджетов) Tk и Toplevel представлены в таблице 7.1.

Таблица 7.1 – Методы классов (виджетов) Tk и Toplevel

Методы классов (виджетов) Tk и Toplevel	Описание метода	Примечание
title	Заголовок окна	
overrideredirect	Указание оконному менеджеру игнорировать это окно. Аргументом является True или False. В случае, если аргумент не указан - получаем текущее значение. Если аргумент равен True, то такое окно будет показано оконным менеджером без обрамления	Может быть использовано, например, для создания splashscreen при старте программы
iconify / deiconify	Свернуть / развернуть окно	
withdraw	Спрятать (сделать невидимым) окно	Для того, чтобы снова показать его, надо использовать метод *deiconify*
minsize и maxsize	Минимальный / максимальный размер окна	Методы принимают два аргумента - ширина и высота окна. Если аргументы не указаны - возвращают текущее значение
state	Получить текущее значение состояния окна	Может возвращать следующие значения: normal (нормальное состояние), icon (показано в виде иконки), iconic (свёрнуто), withdrawn (не показано)
resizable	Может ли пользователь изменять размер окна	Принимает два аргумента - возможность изменения размера по горизонтали и по вертикали. Без аргументов возвращает текущее значение
geometry	Устанавливает геометрию окна в формате ширинахвысота+x+y	
transient	Сделать окно зависимым от другого окна, указанного в аргументе	Будет сворачиваться вместе с указанным окном. Без аргументов возвращает текущее значение
protocol	Получает два аргумента: название события и функцию	Функция будет вызываться при наступлении указанного события

Продолжение таблицы 7.1

Методы классов (виджетов) Tk и Toplevel	Описание метода	Примечание
tkraise и lower	Поднимает (размещает поверх всех других окон) или опускает окно	Методы могут принимать один необязательный аргумент: над/под каким окном разместить текущее
grab_set	Устанавливает фокус на окно, даже при наличии открытых других окон	
grab_release	Снимает монопольное владение фокусом ввода с окна	

Геометрия и положение окна

По умолчанию окно приложения появляется в верхнем левом углу экрана. Его размер (ширина и высота) определяется совокупностью размеров расположенных в нем виджетов. В случае если окно пустое, то класс tkinter устанавливает его размер в 200 на 200 пикселей.

С помощью метода geometry() можно изменить как размер окна, так и его положение. Метод принимает строку определенного формата.

Например, фрагмент кода

```
from tkinter import
root = Tk()
root.geometry('600x400+200+100')
root.mainloop()
```

задает установленную геометрию окна.

В представленном фрагменте кода программы первые два числа в строке-аргументе geometry() задают ширину и высоту окна. Вторая пара чисел обозначает смещение на экране по осям x и y. В примере окно размерностью 600 на 400 будет смещено от верхней левой точки экрана на 200 пикселей вправо и на 100 пикселей вниз.

Если перед обоими смещениями вместо плюса указывается минус, то расчет происходит от нижних правых углов экрана и окна. Например, кодовая конструкция

```
root.geometry('600x400-0-0')
```

позволяет создать окно появиться в нижнем правом углу.

В аргументе метода geometry() можно не указывать либо размер, либо смещение. Например, чтобы сместить окно, но не менять его размер, следует создать кодовую конструкцию:

```
root.geometry('+200+100').
```

Часто удобно, чтобы окно появлялось в центре экрана. Методы winfo_screenwidth() и winfo_screenheight() возвращают количество пикселей экрана, на котором появляется окно.

Например, фрагмент кода программы

```
w = root.winfo_screenwidth() # ширина экрана
h = root.winfo_screenheight() # высота экрана
```

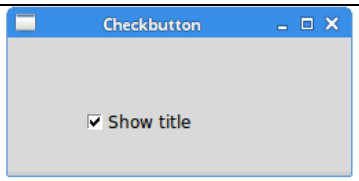
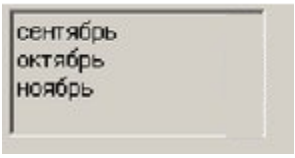
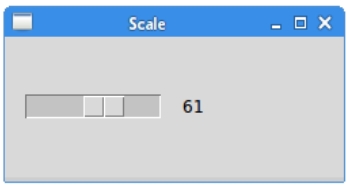
w = w//2 # середина экрана
h = h//2
w = w - 200 # смещение от середины
h = h - 200
root.geometry('400x400+{}+{}'.format(w, h))

помещает окно в центр экрана, если размер окна известен. В указанном фрагменте кода программы вычитается половина ширины и высоты окна (по 200 пикселей). В противном случае в центре экрана окажется верхний левый угол окна, а не его середина.

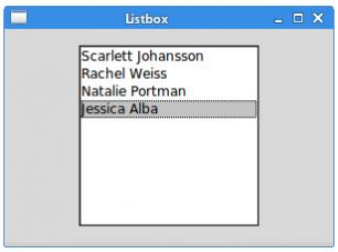
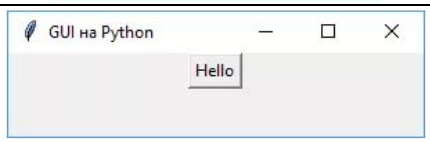
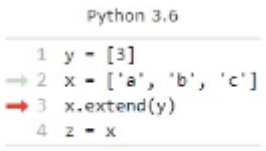
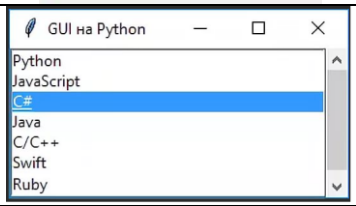
Создание виджетов

Виджеты, предусмотренные в Python, представлены в таблице 7.2.

Таблица 7.2 – Виджеты, предусмотренные в Python

Название виджета	Виджет в Python	Визуализация виджета	Комментарии
Включение/ выключение	Checkbutton		Имеет влажок (галочку) и текстовый ярлык
Ярлык	Label		Виджет Label может отображать ограниченное количество форматов изображений или текст
Однострочное текстовое поле	Entry (аналог поля Edit, например, в Object Pascal)		Вводит/отображает одну строку текста
Многострочное текстовое поле	Text (аналог поля Memo, например, в Object Pascal)		Вводит/отображает текст
Шкала	Scale		Метод onScale() получает текущее выбранное значение из виджета шкалы как параметр. Значение конвертируется в значение с плавающей точкой, после чего конвертируется в целое. В результате, значение передается держателю и отображается на виджете ярлыке.

Продолжение таблицы 7.2

Списки	Listbox		Можно извлекать фактическое значение, которое извлекается при помощи метода get(), получающего индекс объекта
Кнопка	Button		Для обработки, практически, любых событий
Вывод информации в виде окна	Frame		Окно в окне
Группа кнопок	Radiobutton		Несколько переключателей
Полоса прокрутки	Scrollbar		Для окна или формы

Установка свойств виджетов

Все виджеты в Tkinter обладают некоторыми общими свойствами. Виджеты создаются вызовом конструктора соответствующего класса. Первый аргумент (как правило неименованный, но можно использовать имя master) это родительский виджет, в который будет упакован (помещён) наш виджет. Родительский виджет можно не указывать, в таком случае будет использовано главное окно приложения. Далее следуют именованные аргументы, конфигурирующие виджет. Это может быть:

- используемый шрифт (font=...);
- цвет виджета (bg=...);
- команда, выполняющаяся при активации виджета (command=...) и прочее.

Определение событий, которые будут обрабатываться с помощью виджетов

Разработчик программы самостоятельно определяет события, которые будут обрабатываться при помощи того или иного виджета. Например, при нажатии на кнопку может быть осуществлено:

- выполнение расчетов в соответствии с кодом программы;

- вывод информации в поле, например, в поле Label;
- отображение другого окна программы;
- открытие внешнего приложения и прочее.

Определение событий, которые будут обрабатываться с помощью виджетов, должно осуществляться на стадии проекта разрабатываемого приложения (программы).

Событиям принято давать названия. Есть три формы названия событий В Python:

1. Символ кодировки ASCII.

При использовании символа кодировки ASCII описываются события нажатия клавиш на клавиатуре, например, фрагмент кода:

widget.bind("z", callback)

позволяет вызывать функцию callback каждый раз, когда будет нажата клавиша «z».

2. Использование конструкции <modifier-modifier-type-detail>.

Этот способ позволяет описать большее количество событий и связать их с виджетами.

Синтаксис <modifier-modifier-type-detail> говорит о том, что название события заключено в угловые скобки. Внутри имеются ноль или более модификаторов, тип события и дополнительная информация (номер нажатой клавиши мыши или символ клавиатуры) Поля разделяются дефисом или пробелом. Например, фрагмент кода

widget.bind("<Control-Shift-KeyPress-q>", callback)

позволяет вызывать функцию callback при одновременном нажатии клавиш клавиатуры Ctrl+Shift+q):

3. Привязка виртуальных событий, то есть событий, которые генерируются самим приложением (такие события определяются разработчиком программ самостоятельно). Имена таких событий помещаются в двойные угловые скобки: <<Paste>>. В Python есть некоторое количество уже определённых виртуальных событий (таблица 7.3).

Таблица 7.3 – Виртуальные события, определенные в Python

№	Определённое в Python виртуальное событие	Расшифровка события
1	Return	Enter
2	Escape	Esc
3	Control	Ctrl
4	Alt	Alt
5	Shift	Shift
6	Shift	Shift
7	Prior	PgUp
8	Next	PgDown
9	Button1, B1	Нажата первая (левая) кнопка мыши
10	Button2, B2	Нажата вторая (средняя) кнопка мыши
11	Button3, B3	Нажата третья (правая)
12	Double	Двойной щелчок мыши

Например, код программы

```
from Tkinter import *
root=Tk()
def leftclick(event):
   print u 'Вы нажали левую кнопку мыши'
def rightclick(event):
   print u 'Вы нажали правую кнопку мыши'
button1=Button(root, text=u'Нажми')
button1.pack()
button1.bind('<Button-1>', leftclick)
button1.bind('<Button-3>', rightclick)
root.mainloop()
```

позволяет увязать несколько обработчиков событий.

Определение обработчиков событий для виджетов

Привязка событий к виджету – это процесс связи запланированного разработчиком программы события к конкретному виджету.

В Python предусмотрены следующие опции привязки события к виджету:

1. Опция command

С использованием опции command можно привязать большинство событий к соответствующим виджетам, реагирующим на действия пользователя.

При помощи опции command можно осуществить привязку события к таким виджетам как: Button, Checkbutton, Radiobutton, Spinbox, Scrollbar, Scale.

Например, фрагмент кода

```
button = Button(command=callback)
```

позволяет вызвать callback принажатии на кнопку.

2. Метод bind()

Метод bind () привязывает событие к какому-либо действию (нажатие кнопки мыши, нажатие клавиши на клавиатуре и прочее). Метод bind принимает три аргумента (третий аргумент необязательный):

- название события;

- функцию, которая будет вызвана при наступлении события;

- строка «+», что означает процесс добавления привязки к уже существующим. Если третий аргумент опущен или равен пустой строке, то привязка замещает все другие привязки данного события к виджету.

Примечание: Если метод bind привязан к окну верхнего уровня, то Tkinter будет обрабатывать события всех виджетов этого окна.

Расположение виджетов в окне программы

Расположение виджетов в окне программы так же предусматривается разработчиком программы на стадии проекта.

Однако, в Python предусмотрены, так называемые, упаковщики (менеджеры геометрии или менеджеры расположения), то есть специальные механизмы, которые размещают (упаковывает) виджеты в окне программы.

Примечание: в Python в одном виджете можно использовать только один тип упаковки, при смешивании разных типов упаковки программа работать не будет.

В Python предусмотрены следующие упаковщики:

1. Упаковщик pack()

При использовании упаковщика pack() с помощью свойства side необходимо указать, к какой стороне родительского виджета текущий виджет должен примыкать. Как правило, этот упаковщик используют для размещения виджетов друг за другом, в частности, слева направо или сверху вниз.

Пример, код программы:

```
from tkinter import *
root=Tk()
button1 = Button(text="1")
button2 = Button(text="2")
button3 = Button(text="3")
button4 = Button(text="4")
button5 = Button(text="5")
button1.pack(side='left')
button2.pack(side='top')
button3.pack(side='left')
button4.pack(side='bottom')
button5.pack(side='right')
root.mainloop()
```

позволяет разместить кнопки в последовательности, как представлено на рисунке 7.1.



Рисунок 7.1 – Размещение виджетов (кнопок) при помощи упаковщика pack()

Аргументы упаковщика pack() представлены в таблице 7.4.

Таблица 7.4 – Аргументы упаковщика pack()

Аргументы упаковщика pack()	Расшифровка аргумента
side ("left"/"right"/"top"/"bottom")	Показывает, к какой стороне должен примыкать размещаемый виджет
fill (None/"x"/"y"/"both")	Показывает, необходимо ли расширять пространство, предоставляемое виджету
expand (True/False)	Показывает, необходимо ли расширять сам виджет, чтобы он занял всё предоставляемое ему пространство
in	Показывает, в какой родительский виджет должен быть помещён текущий виджет

В Python предусмотрены дополнительные функции для работы с упаковщиком pack(). Дополнительные функции позволяют вызвать (отобразить) информацию, связанную с упаковщиком виджета.

Дополнительные функции для упаковщика pack() представлены в таблице 7.5.

Таблица 7.5 – Дополнительные функции упаковщика pack()

Дополнительные функции упаковщика pack()	Предназначение функции
pack_slaves	Возвращает список всех дочерних упакованных виджетов
pack_info	Возвращает информацию о конфигурации упаковки
pack_propagate	Включает/отключает распространении информации о геометрии дочерних виджетов. По умолчанию виджет изменяет свой размер в соответствии с размером своих потомков. Этот метод может отключить такое поведение конструкцией вида (pack_propagate(False)). Это может быть полезно, если необходимо, чтобы виджет имел фиксированный размер и не изменял его под действием потомков
pack_forget	Удаляет виджет и всю информацию о его расположении из упаковщика. Позднее этот виджет может быть снова размещён.

2. Упаковщик grid()

При использовании упаковщика grid() образуется таблица с ячейками, куда помещаются виджеты.

Аргументы упаковщика grid () представлены в таблице 7.6.

Таблица 7.6 – Аргументы упаковщика grid ()

Аргументы упаковщика grid ()	Расшифровка аргумента
row	Показывает номер строки, в который помещен виджет
rowspan	Показывает, сколько строк занимает виджет
column	Показывает номер столбца, в который помещен виджет
columnspan	Показывает, сколько столбцов занимает виджет
padx / pady	Показывает размер внешней границы (бордюра) по горизонтали и вертикали
ipadx / ipady	Показывает размер внутренней границы (бордюра) по горизонтали и вертикали. Разница между pad и ipad в том, что при указании pad расширяется свободное пространство, а при ipad расширяется помещаемый виджет
sticky ("n", "s", "e", "w" или их комбинация)	Показывает, к какой границе следует привязывать виджет. Позволяет расширять виджет в указанном направлении. Границы названы в соответствии со сторонами света: - "n" (север) - верхняя граница; - "s" (юг) – нижняя; - "w" (запад) – левая; - "e" (восток) - правая.
in	Показывает, в какой родительский виджет должен быть помещён

Дополнительные функции упаковщика grid() представлены в таблице 7.7.

Таблица 7.7 – Дополнительные функции упаковщика grid()

Дополнительные функции упаковщика grid ()	Предназначение функции
grid_remove	Удаляет виджет из-под управления упаковщиком, но сохраняет информацию об упаковке. Этот метод удобно использовать для временного удаления виджета
grid_bbox	Возвращает координаты (в пикселях) указанных столбцов и строк
grid_location (синоним location)	Принимает два аргумента: x и y (в пикселях). Возвращает номер строки и столбца в которые попадают указанные координаты, либо -1 если координаты попали вне виджета.
grid_size (синоним size)	Возвращает размер таблицы в строках и столбцах
grid_columnconfigure и grid_rowconfigure	Важные функции для конфигурирования упаковщика. Методы принимают номер строки/столбца и аргументы конфигурации

Для каждого виджета необходимо указывать, в какой строке и в каком столбце он находится. Так же указывается, сколько ячеек занимает виджет. Пример, код программы

```
entry1.grid(row=0,column=0,columnspan=3)
button1.grid(row=1,column=0)
button2.grid(row=1,column=1)
button3.grid(row=1,column=2)
```

расположение кнопок при использовании упаковщика grid().

Ниже представлен пример кода программы для создания текстового виджета с двумя полосами прокрутки:

```
from tkinter import *
root=Tk()
text = Text(wrap=NONE)
vscrollbar = Scrollbar(orient='vert', command=text.yview)
text['yscrollcommand'] = vscrollbar.set
hscrollbar = Scrollbar(orient='hor', command=text.xview)
text['xscrollcommand'] = hscrollbar.set
# размещаем виджеты
text.grid(row=0, column=0, sticky='nsew')
vscrollbar.grid(row=0, column=1, sticky='ns')
hscrollbar.grid(row=1, column=0, sticky='ew')
# конфигурируем упаковщик, чтобы текстовый виджет расширялся
root.rowconfigure(0, weight=1)
root.columnconfigure(0, weight=1)
root.mainloop()
```

3. Упаковщик place()

Упаковщик place представляет собой простой упаковщик, позволяющий размещать виджет в фиксированном месте с фиксированным размером. При использовании этого упаковщика, необходимо указывать координаты каждого виджета. Например, строка кода программы

button1.place(x=0,y=0)

позволяет разместить кнопку, если заданы координаты (0,0) относительно окна программы (в пикселях).

Аргументы упаковщика place () представлены в таблице 7.8.

Таблица 7.8 – Аргументы упаковщика place ()

Аргументы упаковщика place ()	Расшифровка аргумента
anchor ("n", "s", "e", "w", "ne", "nw", "se", "sw" или "center")	Показывает, какой угол или сторона размещаемого виджета будет указана в аргументах x/y/relx/rely. По умолчанию "nw" - левый верхний
bordermode ("inside", "outside", "ignore")	Определяет, в какой степени будут учитываться границы при размещении виджета.
in	Показыает, в какой родительский виджет должен быть помещён
x и y	Показывает абсолютные координаты (в пикселях) размещения виджета
width и height	Показывает абсолютную ширину и высоту виджета
relx и rely	Показывает относительные координаты (от 0.0 до 1.0) размещения виджета
relwidth и relheight	Показывает относительную ширину и высоту виджета

Дополнительные функции упаковщика place() аналогичны дополнительным функциям упаковщика pack().

Отображение окна программы

Отображение окна программы рассмотрено при описании создания окон в Python.

Примеры создания виджетов

1. Код программы для создания окна:

```
from Tkinter import *
def window_deleted():
   print u'Окно закрыто'
   root.quit() # явное указание на выход из программы
root=Tk()
root.title(u'Пример приложения')
root.geometry('500x400+300+200') # ширина=500, высота=400, x=300, y=200
root.protocol('WM_DELETE_WINDOW', window_deleted) # обработчик закрытия окна
root.resizable(True, False) # размер окна может быть изменён только по горизонтали
```

root.mainloop()

2. Код программы для создания кнопки

```
from Tkinter import *
root=Tk()
button1=Button(root,text='ok',width=25,height=5,bg='black',fg='red',font
='arial 14')
button1.pack()
root.mainloop()
```

Примечание: В представленном коде программы создается окно (за создание окна, отвечает класс Tk). Далее создаётся кнопка, и указываются ее свойства:

- text (какой текст будет отображён на кнопке; в примере – ок);
- width,height (соответственно, ширина и длина кнопки).
- bg (цвет кнопки; сокращенно от background; в примере цвет – чёрный);
- fg (цвет текста на кнопке; сокращённо от foreground; в примере цвет – красный);
- font (шрифт и его размер; в примере – arial, размер – 14).

Далее кнопку необходимо разместить на окне. Для этого в Tkinter используются упаковщик pack(). В конце программы, необходимо использовать функцию mainloop, иначе окно не будет отображено.

3. Код программы для создания списка:

```
from Tkinter import *
root=Tk()
listbox1=Listbox(root,height=5,width=15,selectmode=EXTENDED)
listbox2=Listbox(root,height=5,width=15,selectmode=SINGLE)
list1=[u"Москва",u"Санкт-Петербург",u"Саратов",u"Омск"]
list2=[u"Канберра",u"Сидней",u"Мельбурн",u"Аделаида"]
for i in list1:
   listbox1.insert(END,i)
for i in list2:
   listbox2.insert(END,i)
listbox1.pack()
listbox2.pack()
root.mainloop()
```

4. Код программы для создания фрейма (рамки):

```
from tkinter import *
root=Tk()
frame1=Frame(root,bg='green',bd=5)
frame2=Frame(root,bg='red',bd=5)
button1=Button(frame1,text=u'Первая кнопка')
button2=Button(frame2,text=u'Вторая кнопка')
frame1.pack()
frame2.pack()
button1.pack()
```

```
button2.pack()
root.mainloop()
```

Примечание: свойство bd отвечает за толщину края рамки.

5. Код программы для создания «галочки»

```
from tkinter import *
root=Tk()
var1=IntVar()
var2=IntVar()
check1=Checkbutton(root,text=u'1
пункт',variable=var1,onvalue=1,offvalue=0)
check2=Checkbutton(root,text=u'2
пункт',variable=var2,onvalue=1,offvalue=0)
check1.pack()
check2.pack()
root.mainloop()
```

Примечание:

- IntVar() – специальный класс библиотеки для работы с целыми числами;

- variable – свойство, отвечающее за прикрепление к виджету переменной;

- onvalue, offvalue – свойства, которые присваивают прикреплённой к виджету переменной значение, которое зависит от состояния (onvalue – при выбранном пункте, offvalue – при невыбранном пункте).

6. Код программы для создания группы кнопок

```
from tkinter import *
root=Tk()
var=IntVar()
rbutton1=Radiobutton(root,text='1',variable=var,value=1)
rbutton2=Radiobutton(root,text='2',variable=var,value=2)
rbutton3=Radiobutton(root,text='3',variable=var,value=3)
rbutton1.pack()
rbutton2.pack()
rbutton3.pack()
root.mainloop()
```

Примечание: в созданном виджете используется уже одна переменная. В зависимости от того, какой пункт выбран, она меняет своё значение.

7. Код программы для создания шкалы

```
from tkinter import *
root = Tk()
def getV(root):
    a = scale1.get()
    print "Значение", a
scale1 =
Scale(root,orient=HORIZONTAL,length=300,from_=50,to=80,tickinterval=5,
```

```
            resolution=5)
        button1 = Button(root,text=u"Получить значение")
        scale1.pack()
        button1.pack()
        button1.bind("<Button-1>",getV)
        root.mainloop()
```

Примечание: в представленном коде используется специальный метод get(), который позволяет снять с виджета определенное значение, и используется не только в Scale.

8. Код программы для создания полосы прокрутки

```
        from tkinter import *
        root = Tk()
        text = Text(root, height=3, width=60)
        text.pack(side='left')
        scrollbar = Scrollbar(root)
        scrollbar.pack(side='left')
        # первая привязка
        scrollbar['command'] = text.yview
        # вторая привязка
        text['yscrollcommand'] = scrollbar.set
        root.mainloop()
```

Пример 13

Разработать программу, которая имеет графический интерфейс:

- главное окно, содержащее кнопку (кнопка подсвечена желтым цветом), при нажатии на которую открывается дополнительное окно программы;
- дополнительное окно программы содержит кнопку (кнопка подсвечена красным цветом), при нажатии на которую главное окно программы закрывается;
- текст на кнопке выделен цветом;

Примечание: главное и дополнительное окна программы содержат стандартные виджеты для:

- свертывания окна (размещения его на рабочем столе);
- развертывания окна на весь экран;
- закрытия окна программы.

Указанные виджеты устанавливаются при подключении библиотеки tkinter при создании окон.

Выполнение примера 13

На рисунке 7.2 показан код программы к примеру 13.

Примечание: на рисунке 7.2 примечания показаны не полностью.

На рисунке 7.3 показан полный текст комментариев к программе в соответствии с примером 13.

На рисунке 7.4 показаны дополнительные комментарии (показывают логику составления кода) к программе в соответствии с примером 13.

```
File  Edit  Format  Run  Options  Window  Help
from  tkinter import* # Импортируем все компоненты из м

root = Tk()      # Создаем окно (главное)
root.title("GUI на Python") # Прописываем заголовок окн
root.geometry("600x550+300+100")      # Задаем размеры о
def close():          # Создаем функцию нажатия на кноп
    root.destroy()    # Закрываем первое окно

clicks = 0      # Задаем начальное значение переменной
def click_button():   # Создаем функцию нажатия на кноп
    global clicks     # объявляем переменную глобальной
    clicks += 1    # с каждым вызовом функции увеличива
    root.title("Clicks {}".format(clicks))   # Изменяем
                                  #  .format(clicks) пер
    root1=Tk()                    # Создаем второре о
    root1.title("Новое окно")      # Прописываем заго
    root1.geometry("300x250+50+25") # Задаем размеры ок
    btn1 = Button(root1,                  # Указываем в
              text="Закрыть первое окно",       # текст
              background="#ff0000",     # фоновый цвет к
              foreground="#000000",     # цвет текста
              padx="20",                # отступ от границ
              pady="8",                 # отступ от границ
              font="16",                # высота шрифта
              command=close)  # команда обработчик событ
    btn1.pack() # Делаем кнопку видимой


btn = Button(root,                    # Указываем в каком
              text='Открыть новое окно и изменить Имя пе
              background="#ffff00",     # фоновый цвет к
              foreground="#00ff7f",     # цвет текста в
              padx="20",                # отступ от границ
              pady="8",                 # отступ от границ
              font="16",                # высота шрифта
              command=click_button)  # команда обработчи
btn.pack() # Делаем кнопку видимой (Метод размещения ви

root.mainloop() #Для отображения окна надо вызвать у н
```

Рисунок 7.2 – Код программы к примеру 13

```
r import* # Импортируем все компоненты из модуля tkinter

    # Создаем окно (главное)
'GUI на Python") # Прописываем заголовок окна
y("600x550+300+100")     # Задаем размеры окна "Ширина x Высота + координатаX + координатаY"
        # Создаем функцию нажатия на кнопку
troy()    # Закрываем первое окно

    # Задаем начальное значение переменной
tton():   # Создаем функцию нажатия на кнопку
licks     # объявляем переменную глобальной (для того, чтобы при случайном повторе в дальнейшем коде бралось именно это значение из функции)
= 1    # с каждым вызовом функции увеличиваем переменную на 1
le("Clicks {}".format(clicks))   # Изменяем заголовок окна
                  #  .format(clicks) переводит переменную в строковый тип
()                   # Создаем второре окно
tle("Новое окно")        # Прописываем заголовок окна
ometry("300x250+50+25") # Задаем размеры окна "Ширина x Высота + координатаX + координатаY"
utton(root1,                 # Указываем в каком окне создается вторая кнопка
  text="Закрыть первое окно",       # текст кнопки
  background="#ff0000",     # фоновый цвет кнопки
  foreground="#000000",     # цвет текста
  padx="20",             # отступ от границ до содержимого по горизонтали
  pady="8",              # отступ от границ до содержимого по вертикали
  font="16",             # высота шрифта
  command=close)  # команда обработчик события "Нажатие на кнопку"
k() # Делаем кнопку видимой
(root,                   # Указываем в каком окне создается кнопка
  text='Открыть новое окно и изменить Имя первого',       # текст кнопки
  background="#ffff00",     # фоновый цвет кнопки в 16-м формате
  foreground="#00ff7f",     # цвет текста в 16-м формате
  padx="20",             # отступ от границ до содержимого по горизонтали
  pady="8",              # отступ от границ до содержимого по вертикали
  font="16",             # высота шрифта
  command=click_button)  # команда обработчик события "Нажатие на кнопку"
 Делаем кнопку видимой (Метод размещения виджета)
p() #Для отображения окна надо вызвать у него метод mainloop(),который запускает цикл обработки событий окна для взаимодействия с пользователем.
```

Рисунок 7.3 – Полный текст комментариев к программе в соответствии с примером 13

```
# Создаем кнопку в первом окне btn = Button(root, options)
# Первый параметр содержит переменную с указанием окна, в котором создается кнопка
# Второй параметр options представляет набор на самом деле набор параметров,
# которые мы можем установить по их имени:
# activebackground: цвет кнопки, когда она находится в нажатом состоянии
# activeforeground: цвет текста кнопки, когда она в нажатом состоянии
# bd: толщина границы (по умолчанию 2)
# bg/background: фоновый цвет кнопки
# fg/foreground: цвет текста кнопки
# font: шрифт текста, например, font="Arial 14" - шрифт Arial высотой 14px, или font=("Verdana", 13, "bold")
# - шрифт Verdana высотой 13px с выделением жирным
# height: высота кнопки
# highlightcolor: цвет кнопки, когда она в фокусе
# image: изображение на кнопке
# justify: устанавливает выравнивание текста. Значение LEFT выравнивает текст по левому краю,
# CENTER - по центру, RIGHT - по правому краю
# padx: отступ от границ кнопки до ее текста справа и слева
# pady: отступ от границ кнопки до ее текста сверху и снизу
# relief: определяет тип границы, может принимать значения SUNKEN, RAISED, GROOVE, RIDGE
# state: устанавливает состояние кнопки, может принимать значения DISABLED, ACTIVE, NORMAL (по умолчанию)
# text: устанавливает текст кнопки
# textvariable: устанавливает привязку к элементу StringVar
# underline: указывает на номер символа в тексте кнопки, который подчеркивается.
# По умолчанию значение -1, то есть никакой символ не подчеркивается
# width: ширина кнопки
# wraplength: при положительном значении строки текста будут переносится для вмещения в пространство кнопки
```

Рисунок 7.4 – Дополнительные комментарии к программе в соответствии с примером 13

На рисунке 7.5 показан результат работы программы в соответствии с примером 13.

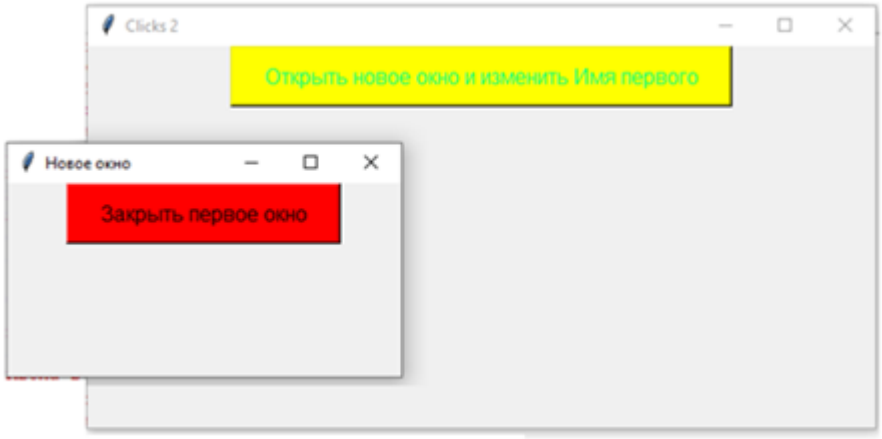


Рисунок 7.5 – Результат работы программы в соответствии с примером 13

Примечание: оба окна содержат инструменты для свертывания, развертывания и закрытия, которые содержатся в библиотеке tkinter и устанавливаются при создании окон.

Пример 14

Разработать программу, которая имеет графический интерфейс:

- главное окно, содержащее поля для ввода информации (два поля Entry) с их соответствующими подписями;
- кнопку для ввода текста, при нажатии на которую информация из полей выводится в Lable;
- кнопку для удаления содержимого из полей;
- кнопку для закрытия окна (кнопки подсвечены различными цветами и подписаны).

Выполнение примера 14

На рисунках 7.6, 7.7 показан код программы к примеру 14.

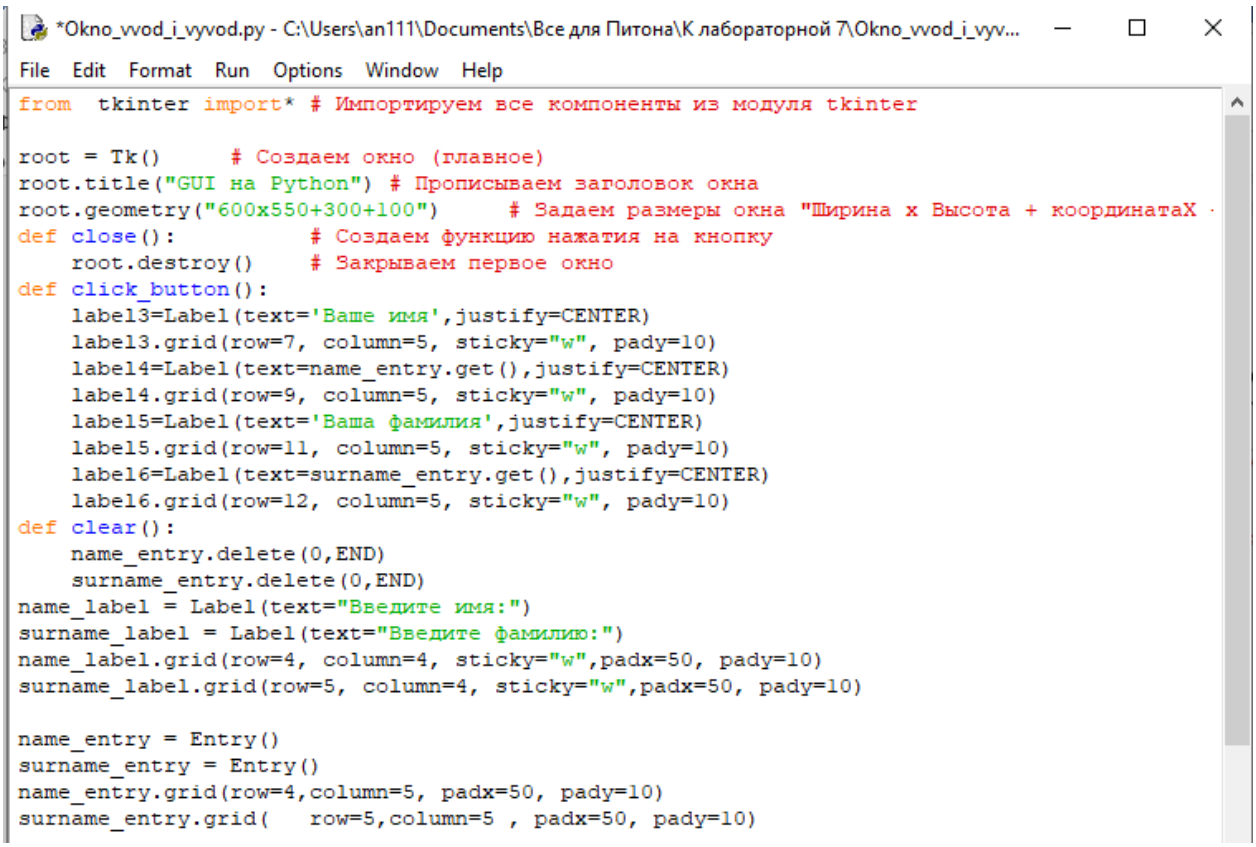


Рисунок 7.6 – Код программы к примеру 14 (создание полей и подписей)

```
btn = Button(root,                  # Указываем в каком окне создается кнопка
             text='Вывод текста',       # текст кнопки
             background="#ffff00",     # фоновый цвет кнопки в 16-м формате
             foreground="#00ff7f",     # цвет текста в 16-м формате
             padx="20",             # отступ от границ до содержимого по горизонтали
             pady="8",              # отступ от границ до содержимого по вертикали
             font="Arial 14",           # высота шрифта
             command=click_button)  # команда обработчик события "Нажатие на кнопку"
btn1 = Button(root,                  # Указываем в каком окне создается кнопка
             text='Закрыть окно',       # текст кнопки
             background="#ff0000",     # фоновый цвет кнопки в 16-м формате
             foreground="#000000",     # цвет текста в 16-м формате
             padx="20",             # отступ от границ до содержимого по горизонтали
             pady="8",              # отступ от границ до содержимого по вертикали
             font=("Times New Roman", 14, "bold"),          # высота шрифта
             command=close)  # команда обработчик события "Нажатие на кнопку"
btn2 = Button(root,                  # Указываем в каком окне создается кнопка
             text='Очистить поле ввода',       # текст кнопки
             background="#00ff00",     # фоновый цвет кнопки в 16-м формате
             foreground="#000000",     # цвет текста в 16-м формате
             padx="20",             # отступ от границ до содержимого по горизонтали
             pady="8",              # отступ от границ до содержимого по вертикали
             font=("Times New Roman", 14, "bold"),          # высота шрифта
             command=clear)  # команда обработчик события "Нажатие на кнопку"
btn.place(x=50,y=450)
btn1.place(x=370,y=450)
btn2.place(x=160,y=350)
root.mainloop()
```

Рисунок 7.7 – Код программы к примеру 14
(создание кнопок и отображение окна)

Дополнительные комментарии к примеру 14 представлены на рисунке 7.8.

```
# Метод place() позволяет более точно настроить параметры позиционирования.
# Он принимает следующие параметры:
# height и width: устанавливают соответственно высоту и ширину элемента в пикселях
# relheight и relwidth: также задают соответственно высоту и ширину элемента,
# но в качестве значения используется число float
# в промежутке между 0.0 и 1.0, которое указывает на долю от высоты и ширины родительского контейнера
# x и y: устанавливают смещение элемента по горизонтали и вертикали в пикселях соответственно
# относительно верхнего левого угла контейнера
# relx и rely: также задают смещение элемента по горизонтали и вертикали, но в качестве значения
# используется число float в промежутке между
# 0.0 и 1.0, которое указывает на долю от высоты и ширины родительского контейнера
# bordermode: задает формат границы элемента. Может принимать значение INSIDE (по умолчанию) и OUTSIDE
# anchor: устанавливает опции растяжения элемента. Может принимать значения n, e, s, w, ne, nw, se, sw, c,
# которые являются сокращениями
# от Noth(север - вверх), South (юг - низ), East (восток - правая сторона), West (запад - левая сторона)
# и Center (по центру).
# Например, значение nw указывает на верхний левый угол
```

Рисунок 7.8 – Дополнительные комментарии к примеру 14

Примечание: в Python поле для ввода текста представляет собой однострочное поле Entry.

На рисунке 7.9 показан результат работы программы в соответствии с рисунком 14.

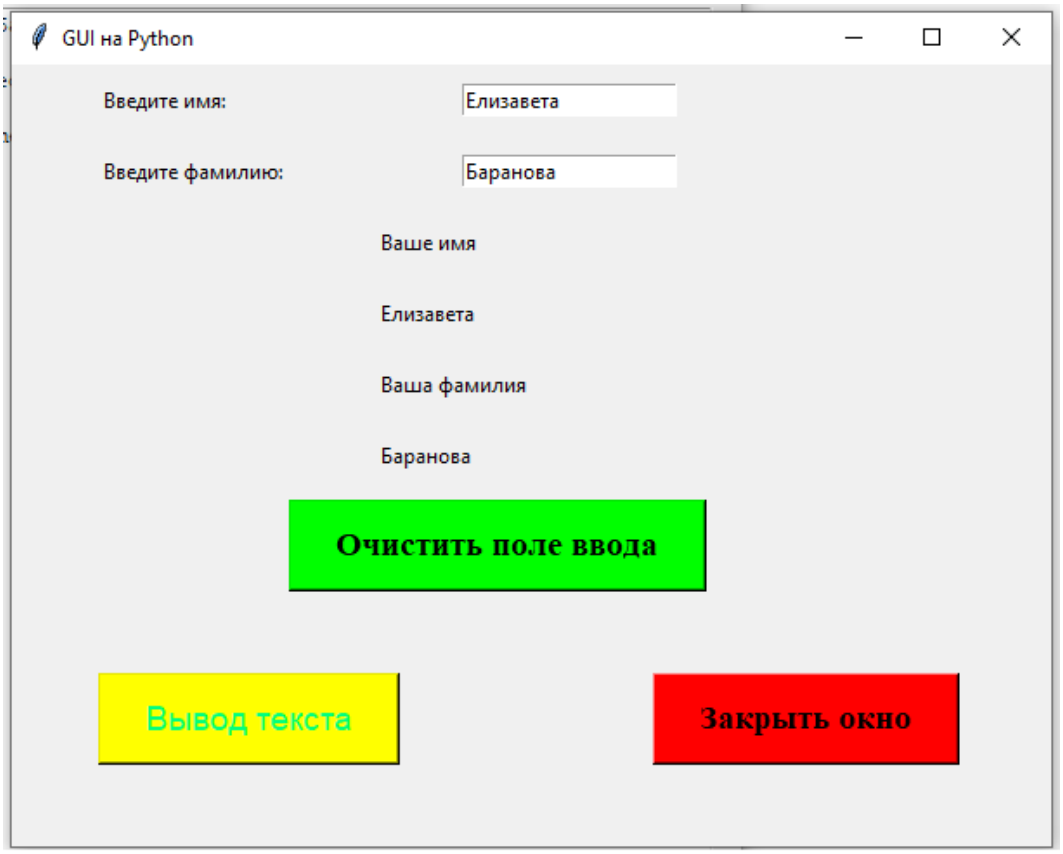


Рисунок 7.9 – Результат работы программы в соответствии с примером 14

7.3. Оборудование и программное обеспечение

1. Персональная ЭВМ.
2. Операционная система Windows/Linux.
3. Среда программирования IDLE или другая среда, содержащая язык программирования Python.
4. MS Office, Libre Office (Open Office).

7.4. Задание на работу

1. Изучить теоретические положения лабораторной работы.

2. Получить результаты работы программ в соответствии с примерами 13, 14.

3. Разработать программу, имеющую графический интерфейс, которая позволяет работать не менее чем с пятью различными виджетами.

4. Разработать программу, имеющую графический интерфейс, которая позволяет записывать в файл информацию, вводимую через соответствующие виджеты:

а) Label;

б) Entry;

в) Text.

5. Разработать программу, имеющую графический интерфейс, которая позволяет считывать информацию из файла в соответствующие виджеты:

а) Label;

б) Entry;

в) Text.

Примечание: проекты в соответствии с пунктами 3,4,5 можно объединить в одну программу.

6. Изобразить схемы работы программ в соответствии с ГОСТ 19.701-90 (использовать графический редактор).

7. Показать результаты работы преподавателю.

8. Ответить на контрольные вопросы.

9. Оформить отчет.

7.5. Контрольные вопросы

1. Что выражает понятие GUI?

2. В какую библиотеку включены виджеты в Python?

3. Какие способы предусмотрены в Python для создания окон программ?

4. Какие методы классов (виджетов) предусмотрены в Python для работы с окнами?

5. Какие виджеты предусмотрены в Python?

6. Как в Python установить свойство виджета? Какие предусмотрены свойства для виджетов в Python?

7. Как связать событие и виджет в Python?

8. Какие названия для событий предусмотрены в Python?

9. Что такое обработчик события?

10. Что такое упаковщики виджетов? Какие упаковщики для виджетов предусмотрены в Python?

7.6. Правила оформления отчета

Отчет должен содержать:

1. Номер, название и цель лабораторной работы.

2. Описание процесса выполнения примеров (коды, схемы и результаты работы программ).

3. Вывод по работе.

Лабораторная работа №8
Работа с базами данных в Python
8.1. Цель лабораторной работы

Научиться работать с базами данных в Python

8.2 Теоретические положения

В настоящее время в основу разработки программных продуктов закладываются технологии работы с базами данных, когда приложение ссылается на базу данных и позволяет обрабатывать данных, хранящиеся в базе, или записывать в базу новые (модифицированные) данные.

Одной из широко используемых баз данных (при создании профессиональных программных систем) сегодня является база MySQL (SQL – structured query language – язык структурированных запросов).

Более компактная база данных, созданная, примерно, по тем же принципам, что и MySQL – это база данных SQLite.

SQLite – это встраиваемая кроссплатформенная база данных (далее БД), которая поддерживает достаточно полный набор команд, вследствие чего позволяет создавать многофункциональные профессиональные приложения.

Принципиальное отличие SQLite от других БД в том, что вся база представляет собой один файл.

БД SQLite может иметь расширения sqlite, sqlite3, sl2, sl3 так и более универсальные db, db2, db3, sbd3.

Для создания баз данных SQLite можно воспользоваться, например, системой управления базами данных (далее СУБД) SQLiteStudio (http://freevi.net/besplatnaya-programma-sqlitestudio-dlya-redaktirovaniya-sqlite-baz-dannyx/).

Преимуществами работы SQLiteStudio являются:

- бесплатная установка;
- портабельность;
- русскоязычность;
- поддержка универсальной системы кодирования текстовой информации UNICODE;
- кросс-платформенность (работает под управлением операционных систем на Windows 9x / 2k / XP / 2003 / Vista / 7, Linux, MacOS X, Unix);
- возможность экспорта данных в другие программы, работающие с иными форматами (CSV, HTML, XML, PDF, JSON);
- возможность импорта данных из различных форматов (CSV, пользовательские текстовые файлы);
- дополнительные опции, сопровождающие процесс работы с данными (форматирование кода, история создания и редактирования запросов, проверка синтаксиса и прочее);
- возможность настройки интерфейса (цвета, шрифты и значки).

СУБД SQLiteStudio (иногда эту программу классифицируют как менеджер БД) имеет открытый исходный код, который опубликован под лицензией GPLv3. Официальный сайт SQLiteStudio – sqlitestudio.pl.

Если используется уже готовая база данных SQLite, то для ее просмотра можно воспользоваться:

- системами он-лайн просмотра баз данных SQLite, например, SQLite Browser (https://sqliteonline.com/);

- системами off-лайн просмотра баз данных SQLite, например, программа SQLite Viewer 2.6, которая представляет собой плагин для просмотра файлов баз данных SQLite3 (http://wincmd.ru/plugring/sqliteviewer.html).

Для работы с базами данных SQLite в Python предусмотрена библиотека SQLite3.

После того, как библиотека SQLite3 импортирована, можно начать работу по созданию и обработки баз данных формата SQLite.

1. Создание таблиц.

Создание таблиц БД SQLite в Python осуществляется следующим образом:

```
import sqlite3
 conn = sqlite3.connect("mydatabase.db") # или : memory: чтобы сохранить в RAM
cursor = conn.cursor()
# Создание таблицы
cursor.execute("""CREATE TABLE Nominalnie razmeri
                (title text, artist text, release_date text,
                 publisher text, media_type text)
               """)
```

В представленном коде программы:

- *import sqlite3* – осуществлен импорт компонентов соответствующей библиотеки;

- *conn = sqlite3.connect("mydatabase.db")* – позволяет установить связь с базой данных, в данном случае создается файл mydatabase.db на жестком диске (для сохранения базы данных в памяти можно использовать конструкцию *memory*);

- *cursor = conn.cursor()* – создается объект *cursor*, который позволяет взаимодействовать с базой данных, то есть вносить в нее записи;

- *cursor.execute("""CREATE TABLE Nominalnie razmeri*

```
                (title text, edinitsa_ism text, min real,
                 max real)
               """)
```

– используется синтаксис SQL для создания таблицы Nominalnie razmeri (Номинальные размеры), таблица будет содержать следующие поля:

а) *title (название)* – текстовое поле;

б) edinitsa_ism (единица измерения) – текстовое поле;

в) min (минимальное значение) вещественное поле;

г) max (максимальное значение) вещественное поле.

Примечание: SQLite поддерживает только пять типов данных:

- null (none) – логический тип данных (ничего);
- integer – целочисленный тип данных;
- real – вещественный тип данных;
- text – тестовый тип данных;
- blob – битовый тип данных (для описания больших битовых массивов).

Примечание: При внесении записей в таблицы БД SQLite через Python следует помнить, что каждый вставляемый объект записи заключается в одинарные кавычки.

2. Запись данных в поля таблицы.

а) Добавление записи (данных) в поля таблицы

Добавление записи (данных) в поля таблицы в Python осуществляется следующим образом:

```
# Вставка данных в таблицу
cursor.execute("""INSERT INTO Nominalnie razmeri
                VALUES ('Dlina', 'mkm', '12,25',
                14,65)"""
            )
 # Сохранение изменений
conn.commit()
```

В представленном коде программы:

- *cursor.execute("""INSERT INTO Nominalnie razmeri VALUES ('Dlina', 'mkm', '12,25', 14,65)"""* – вставка записи Длина, мкм (микрометры), 12,25 (минимальное значение), 12,65 (максимальное значение) в ранее созданную таблицу Nominalnie razmeri;
- *conn.commit()* – сохранение изменений.

б) Добавление множества записей (множества данных) в поля таблицы

Добавление множества записей (множества данных) в поля таблицы в Python осуществляется следующим образом:

```
# Вставка множества данных в таблицу при использовании безопасного метода "?"
Nominalnie razmeri = [(' Dlina ', ' mkm ', '12,25', '14,65'),
(' Shirina ', ' mkm ', '40,28', '45,88'), (' Ugol ', ' gradus ', '10,3', '11,7'),
(' Massa ', ' gramm ', '10,112', '12,112'),]
 cursor.executemany("INSERT INTO Nominalnie razmeri VALUES (?,?,?,?)", Nominalnie razmeri)
conn.commit()
```

В представленном коде программы одновременно вносятся 4 записи в поля ранее созданной таблицы *Nominalnie razmeri*.

Примечание: Для внесения множества записей в таблицу БД SQLite через Python используется метод курсора executemany. Чтобы вставить записи

в таблицы потребуется использовать подход строк замещения, для чего необходимо использовать знаки вопроса (?) вместо символа процентов (%). Такой подход предусмотрен для исключения возможной атаки инъекций со стороны SQL, когда происходит автозамена символов (вброс вредоносного кода) с целью похищения информации из БД.

Примечание: Если запускается команда create table, а база данных уже существует, то программа выдаст сообщение об ошибке.

3. Редактирование записей.

Редактирование записей происходит следующим образом:

```
import sqlite3
 conn = sqlite3.connect("mydatabase.db")
cursor = conn.cursor()
 sql = """
UPDATE Nominalnie razmeri
SET Dlina = '15,65'
WHERE Dlina = '14,65'
"""
 cursor.execute(sql)
conn.commit()
```

Использована команда *SQL UPDATE*, чтобы обновить таблицу *Nominalnie razmeri*. Чтобы изменить поле, используется команда *SET*. Для поиска записи используется команда *WHERE*. В данном случае изменена запись *'14,65'* на запись *'15,65'* в поле *Dlina*.

Для подтверждения изменений записей в БД используется кодовая конструкция *cursor.execute(sql)*.

4. Удаление записей.

Удаление записей осуществляется следующим образом:

```
mport sqlite3
conn = sqlite3.connect("mydatabase.db")
cursor = conn.cursor()
sql = "DELETE FROM Nominalnie razmeri WHERE Dlina = '14,65'"
cursor.execute(sql)
conn.commit()
```

Для удаления записей требуется указать SQLite, из какой таблицы какую запись требуется удалить. В данном случае запись '14,65' из поля Dlina будет удалена.

5. Формирование запросов.

Формирование запросов осуществляется следующим образом:

```
import sqlite3
conn = sqlite3.connect("mydatabase.db")
#conn.row_factory = sqlite3.Row
cursor = conn.cursor()
sql = "SELECT * FROM Nominalnie razmeri WHERE Dlina=?"
cursor.execute(sql, [("14,65")])
```

print(cursor.fetchall()) # or use fetchone()

print("Here's a listing of all the records in the table:")

*for row in cursor.execute("SELECT rowid, * FROM Nominalnie razmeri ORDER BY Dlina"):*

print(row)

print("Results from a LIKE query:")

*sql = "SELECT * FROM Nominalnie razmeri WHERE Min LIKE '1%'"*

cursor.execute(sql)

print(cursor.fetchall())

В представленном коде программы имеются два запроса.

Формирование запросов в данном случае начинается со структуры SELECT * (синтаксис языка SQL).

Первый запрос позволяет выбрать все записи, подходящие под переданное имя исполнителя, в нашем случае это *'14,65'*.

Для получения результатов SQL-запроса используется функция *fetchall()* (аналогично для получения результата SQL-запроса можно использовать функцию *fetchone()*).

Использование кодовой конструкции *row_factory* позволяет сформировать прокомментированный раздел. Поля БД *SQLite* подобны словарям, поэтому требуется комментирование доступа к ним.

Второй запрос возвращает каждую запись поля *Dlina* БД *Nominalnie razmeri*, которая начинается с 1 и упорядочивает по возрастанию найденные записи. Здесь знак процентов (%) является подстановочным оператором, показывающим, что после символа 1 может быть расположены любые символы.

Примечание: в данном случае рассмотрены запросы на выборку информации из таблиц БД SQLite.

Пример 15

Требуется разработать приложение (оконное приложение) для работы с ранее созданной базой данных формата SQLite.

Приложение должно иметь:

- главное меню (команду Open (Открыть БД));
- кнопку для отображения таблиц БД;
- переключатели для выбора способа отображения таблиц БД (по количеству отображаемых строк);
- кнопку добавить запись;
- поля и кнопки для добавления записей в таблицы БД.

Примерный интерфейс приложения в соответствии с примером 15

На рисунке 8.1 показан интерфейс разрабатываемого приложения в соответствии с примером 15 (окно для отображения таблиц БД).

На рисунке 8.2 так же показан интерфейс разрабатываемого приложения в соответствии с примером 15 (окно для внесения записей в таблицы БД).

Примечание: в примере 15 БД не создаётся, файл с БД предварительно создан (скачан) и сохранён на жестком диске компьютера.

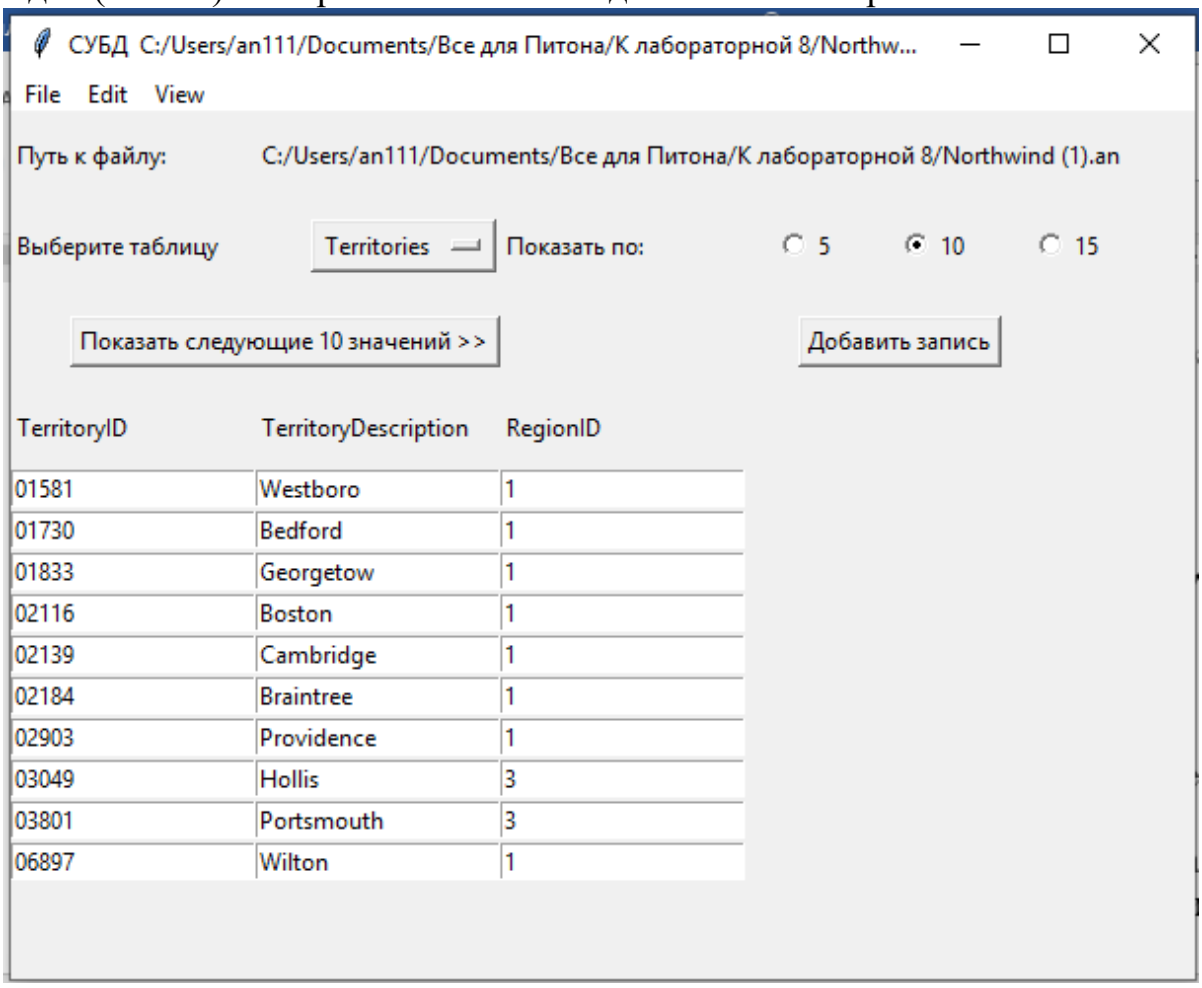


Рисунок 8.1 – Интерфейс разрабатываемого приложения в соответствии с примером 15 (окно для отображения таблиц БД)

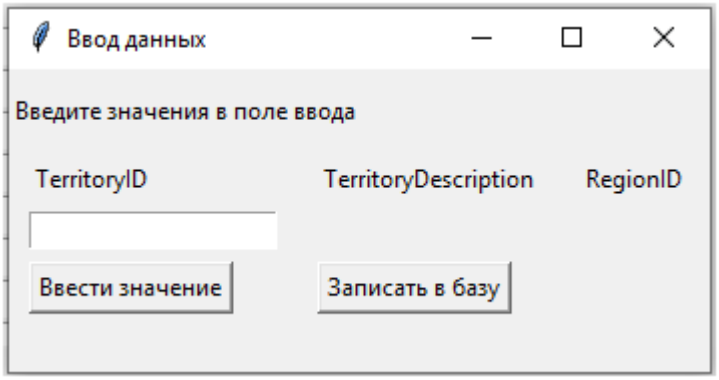


Рисунок 8.2 – Интерфейс разрабатываемого приложения в соответствии с примером 15 (окно для внесения записей в таблицы БД)

Примечание: промежуточные изменения в графическом интерфейсе приложения будут показаны ниже, в процессе демонстрации результатов работы приложения.

Выполнение примера 15

Пример 15 выполняется при помощи функциональной технологии программирования.

Функциональная технология программирования – это парадигма программирования, при котором созданная функция содержит процесс обработки данных, причем функция может быть вызвана в теле программы в любой необходимый момент.

1. Вызов необходимых библиотек и компонентов.

Разработка приложения в соответствии с примером 15 начинается с вызова необходимых библиотек и их компонентов:

- sqlite3 (библиотека для подключения БД формата SQLite);
- tkinter (библиотека для подключения библиотеки виджетов).

Для работы приложения потребуются компоненты библиотеки tkinter:

- компоненты filedialog и askdirectory, которые необходимы для связи с операционной системой, так как при создании или выборе файла БД должно быть открыто окно, в котором имеются папки и файлы, структурированные файловой системой компьютера (filedialog – работа с файлами операционной системы, askdirectory – работа с каталогами (папками) файловой системы);
- компонент messagebox, который необходим для формирования сообщений приложением.

2. Создание файла БД.

После вызова указанных библиотек и компонентов, необходимо создать функцию для выбора пути и имени БД, с которой будет происходить работа. Так как создаваемое приложение будет дорабатываться, предусматривается возможность создания БД формата SQLite.

Примечание: далее рассматривается работа с уже имеющейся БД, файл которой предварительно сохранен на жестком диске.

На рисунке 8.3 показан фрагмент кода разрабатываемого приложения (вызов библиотек и создание функции для выбора пути и имени БД).

SUBD1 (1).py - C:\Users\an111\Documents\Все для Питона\К лабораторной 8\БД\SUBD1 (1).py (3.7.1)

File Edit Format Run Options Window Help

```
import sqlite3
from tkinter import*
from tkinter import filedialog as fd
from tkinter.filedialog import askdirectory
from tkinter import messagebox

def NewDir():                          #Функуция выбора пути и имени создания базы
    global dir_name
    clear()
    # Выбор директории
    dir_name = askdirectory()
    dirl=Label(text='Путь к файлу:',justify=CENTER)
    dirl.grid(row=0, column=0, sticky="w",pady=10)
    dir3.set(dir_name)
    dir2=Label(textvariable=dir3,justify=CENTER)
    dir2.grid(row=0, column=1, sticky="w",pady=10)
    #Создание меток и окон ввода
    base=Label(text='База данных:(*.an)',justify=CENTER)
    base.grid(row=1, column=0, sticky="w",pady=10)
    base_name=Entry(textvariable=fn)
    base_name.grid(row=1, column=1, sticky="w",pady=10)
    #Создание кнопки "Создать"
    bl = Button(text="Создать", command=Create)
    bl.grid(row=2,column=0, sticky=E,pady=10)
```

Рисунок 8.3 – Фрагмент кода разрабатываемого приложения (вызов библиотек и создание функции для выбора пути и имени БД)

При создании функции выбора пути и имени БД необходимо создать новую директорию, где будут предусмотрены метки (надписи) и поля для ввода имени и пути БД, а также кнопка Создать после того, как путь и имя БД будут указаны.

На рисунке 8.4 показан, какой результат будет отображен после запуска указанного фрагмента кода программы при условии вызова в конце кода советующих функций (на рисунке показан этап выбора папки для сохранения файла создаваемой БД).

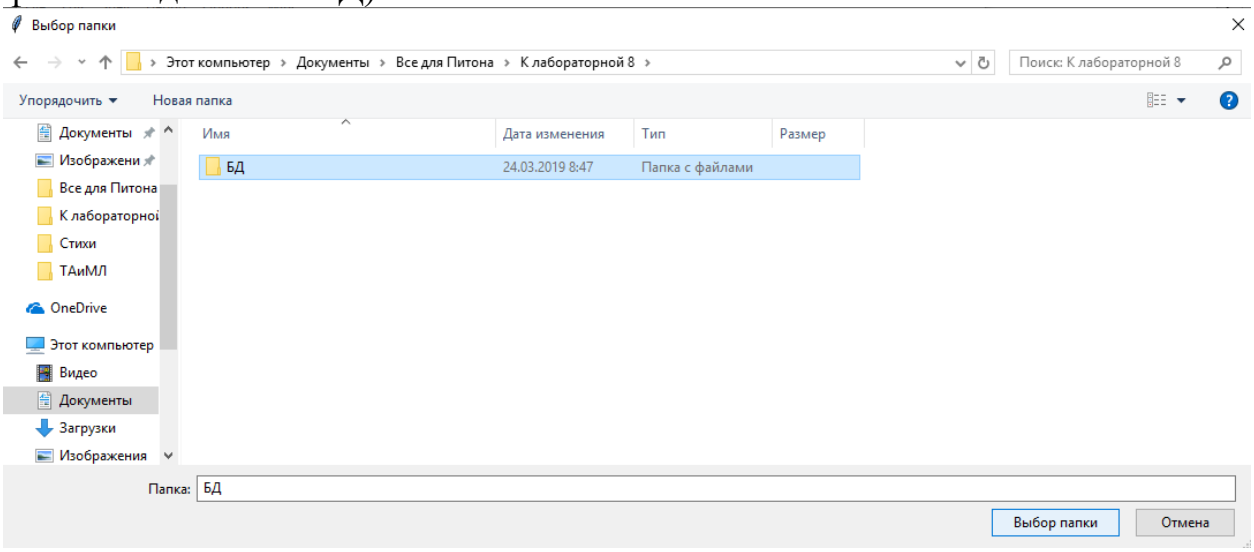


Рисунок 8.4 – Выбор папки для сохранения файла создаваемой БД

На рисунке 8.5 показано окно для создания новой БД, которое запускается после нажатия на кнопку Выбор папки.

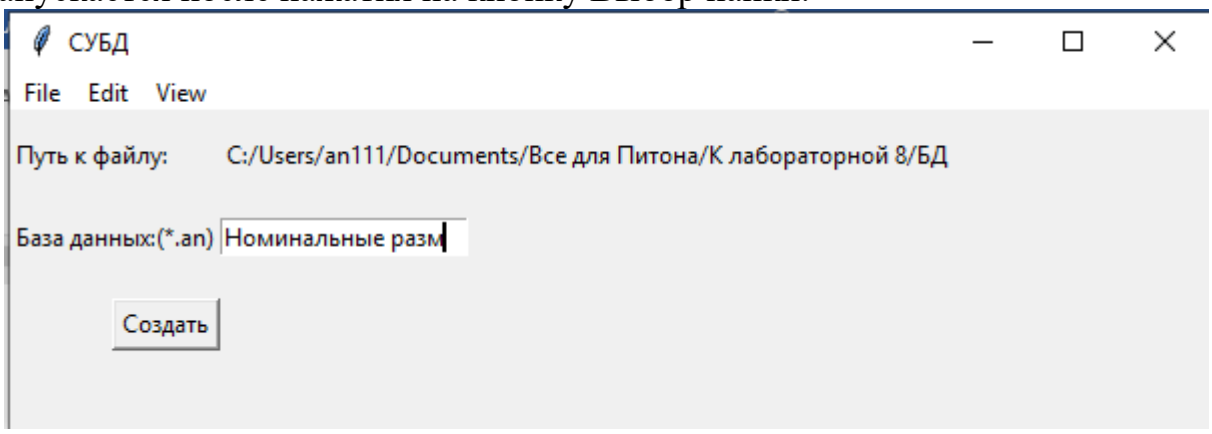


Рисунок 8.5 – Окно для создания новой БД, которое запускается после нажатия на кнопку Выбор папки

Примечание: расширение БД может быть любым, однако тип БД будет относится к типу SQLite, так как для работы с БД предусмотрена соответствующая библиотека sqlite3.

После нажатия на кнопку Создать будет создана база данных с указанным именем и расширением .an, которая будет располагаться по указанному в приложении пути.

Примечание: после создания БД в окне приложения появляются соответствующая запись, указывающая путь к созданной БД.

На рисунке 8.6 показан фрагмент кода разрабатываемого приложения (функция для создания БД).

```
def Create():                                          #Функция созадния базы
    global file_name
    file_name=dir_name+'/'+fn.get()+'.an'
    conn = sqlite3.connect(file_name)
    cursor = conn.cursor()
    base5=Label(text=file_name,justify=CENTER)
    base5.grid(row=5, column=0, sticky="w",pady=10,columnspan=22)
```

Рисунок 8.6 – Фрагмент кода разрабатываемого приложения (функция для создания БД)

На рисунке 8.7 показан файл созданной БД (имя файла – Номинальные размеры.an, тип файла – SQL).

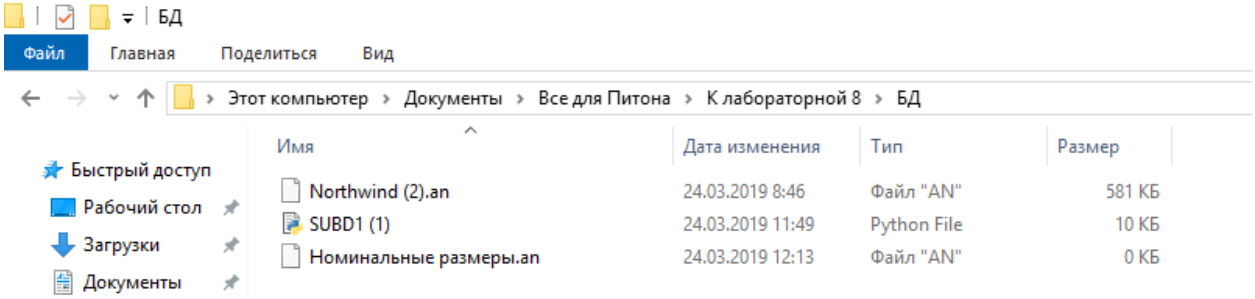


Рисунок 8.7 – Файл созданной БД (тип SQLite, расширение .an)

3. Создание кнопок (переключатели для выбора способа отображения таблиц БД по количеству отображаемых строк)

Для создания соответствующих кнопок потребуется следующий фрагмент кода программы, отвечающий за описание функции создание кнопок и переключателей для выбора количества отображаемых строк (записей) БД (рисунок 8.8).

```
# Функция создания кнопок
def But():
        global nex
        s = sel.get()
        list = root.grid_slaves(row=6)
        for l in list:
            l.destroy()
        nex=Button(command=Table1)
        nex.grid(row=6,column=0, sticky=E,pady=10,columnspan=2)
        nex.config(text='Показать следующие '+str(s)+' значений >>')
        ins=Button(text='Добавить запись',command=child1)
        ins.grid(row=6,column=3, sticky=E,pady=10,columnspan=2)
        Table()
```

Рисунок 8.8 – Фрагмент кода программы, отвечающий за описание функции создание кнопок и переключателей для выбора количества отображаемых строк (записей) БД

Вызов соответствующей функции отвечает за элементы интерфейса приложения, показанные на рисунке 8.9 (выделены цветом).

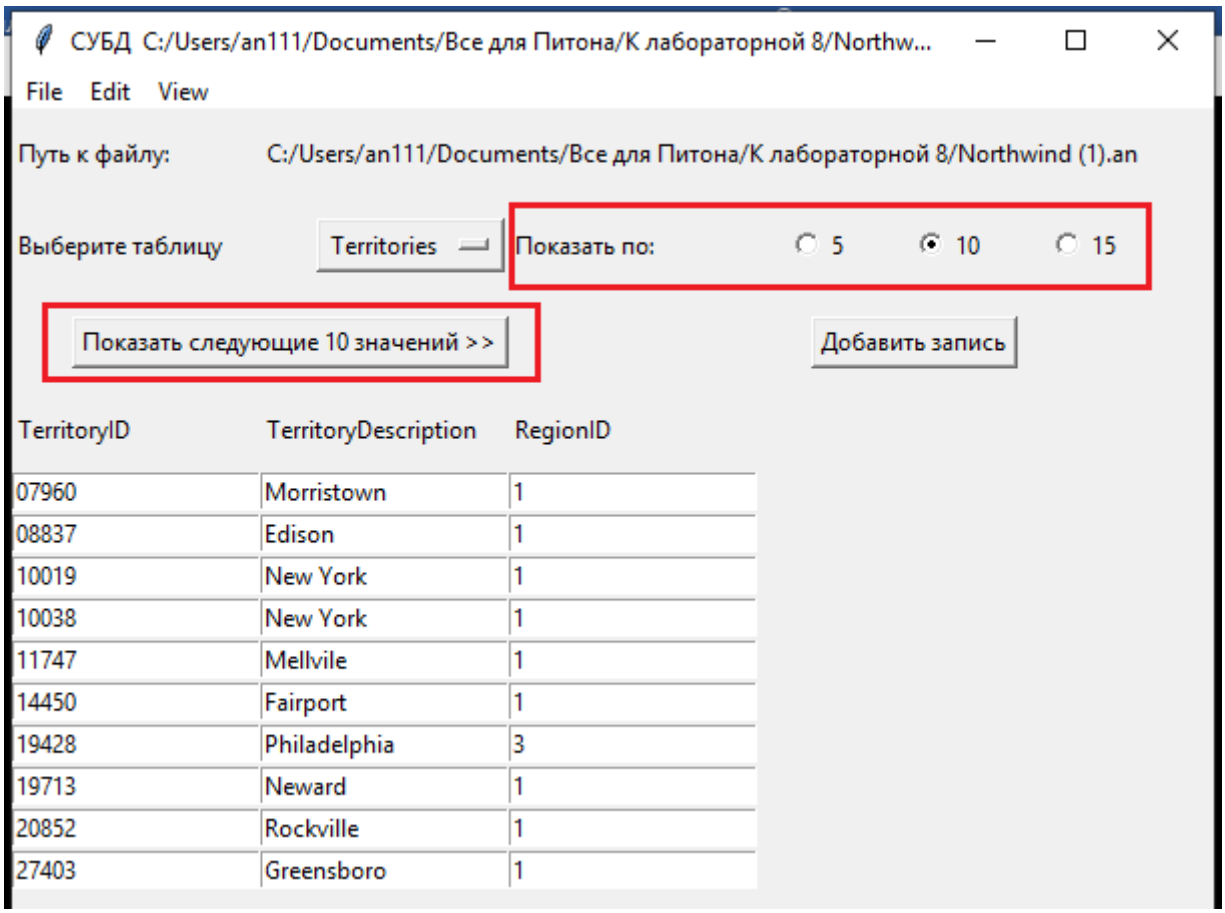


Рисунок 8.9 – Элементы интерфейса при вызове функции создания кнопок и переключателей для выбора количества отображаемых строк (записей) БД

4. Создание функции выбора пути и имени БД, которую следует открыть.

По аналогии с тем, как создавалась функция для выбора пути и имени создания базы, необходимо создать функцию для выбора пути и имени БД, которую следует открыть.

На рисунке 8.10 показан фрагмент кода программы, отвечающий за описание функции для выбора пути и имени БД, которую следует открыть.

В представленном фрагменте кода:

- вызываются элементы интерфейса для указания имени файла с целью его открытия (в рассматриваемом примере открывается БД Northwind(1).an.

- устанавливается связь БД с приложением (строка кода *conn = sqlite3.connect(file_name)*, причем переменная *file_name* описана выше при помощи кодовой конструкции *file_name = fd.askopenfilename(filetypes=((("base files","*.an"),("All files", "*.*")))))*;

- обрабатывается событие открытия таблицы БД путем нажатия на кнопку, подписанную как Выберете таблицу (строка кода *tb= Label(text='Выберите таблицу',justify=CENTER)*);

- вызывается на экран поле со списком таблиц БД (строка кода *opt = OptionMenu(root,var,*tn,command=RB);*

- отображаются строки таблицы БД (строка кода *opt.grid(row=5,column=1, sticky=E,pady=10).*

```
def Open():                                 #Функция выбора пути и имени открываемой БД
    global file_name
    global opt
    global var
    global cursor

    global s
    global sell
    global conn
    clear()
    file_name = fd.askopenfilename(filetypes=(("base files","*.an"),("All files", "*.*")))
    fp=Label(text='Путь к файлу:',justify=CENTER)
    fp.grid(row=0, column=0, sticky="w",pady=10)
    fp1=Label(text=file_name,justify=CENTER)
    fp1.grid(row=0, column=1, sticky="w",pady=10,columnspan=43)
    root.title('СУБД  '+file_name)
    #messagebox.showinfo('СУБД',('База данных успешно открыта!!!'))
    conn = sqlite3.connect(file_name)
    cursor = conn.cursor()
    cursor.execute("SELECT tbl_name FROM sqlite_master WHERE type = 'table' ")
    tables = cursor.fetchall()
    tn=[]
    for table_name in tables:
        table_name = table_name[0]
        tn.append(table_name)
    #var=StringVar()
    tb= Label(text='Выберите таблицу',justify=CENTER)
    tb.grid(row=5, column=0, sticky="w",pady=10)
    opt = OptionMenu(root,var,*tn,command=RB)     # Вызывает на экран поле со списком.
                                                  #(*) разбивает список на отдельные строки
                                                  #(без нее получается одна строка)
    opt.grid(row=5,column=1, sticky=E,pady=10)
    #RB()
```

Рисунок 8.10 – Фрагмент кода программы, отвечающий за описание функции для выбора пути и имени БД, которую следует открыть

На рисунке 8.11 показан результат вызова функции для выбора пути и имени БД, которую следует открыть.

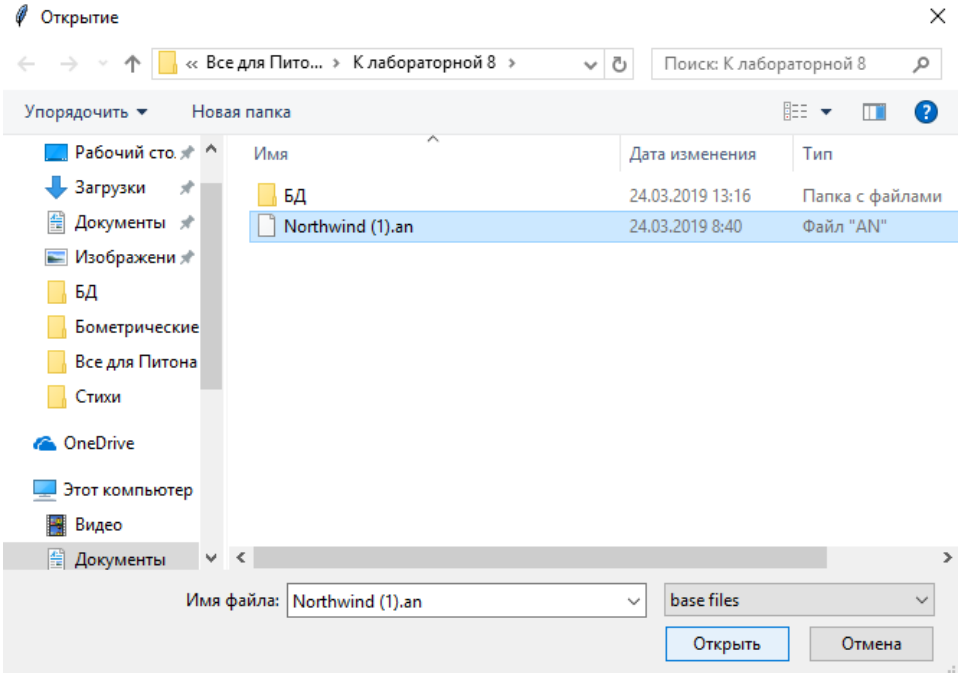


Рисунок 8.11 – Результат вызова функции для выбора пути и имени БД, которую следует открыть

После нажатия на кнопку Открыть, приложение открывает окно, в котором предложено путем нажатия на кнопку Выберете таблицу, выбрать таблицу из открытой БД.

На рисунке 8.12 показаны элементы интерфейса разрабатываемого приложения, появляющиеся после нажатия на кнопку Открыть.

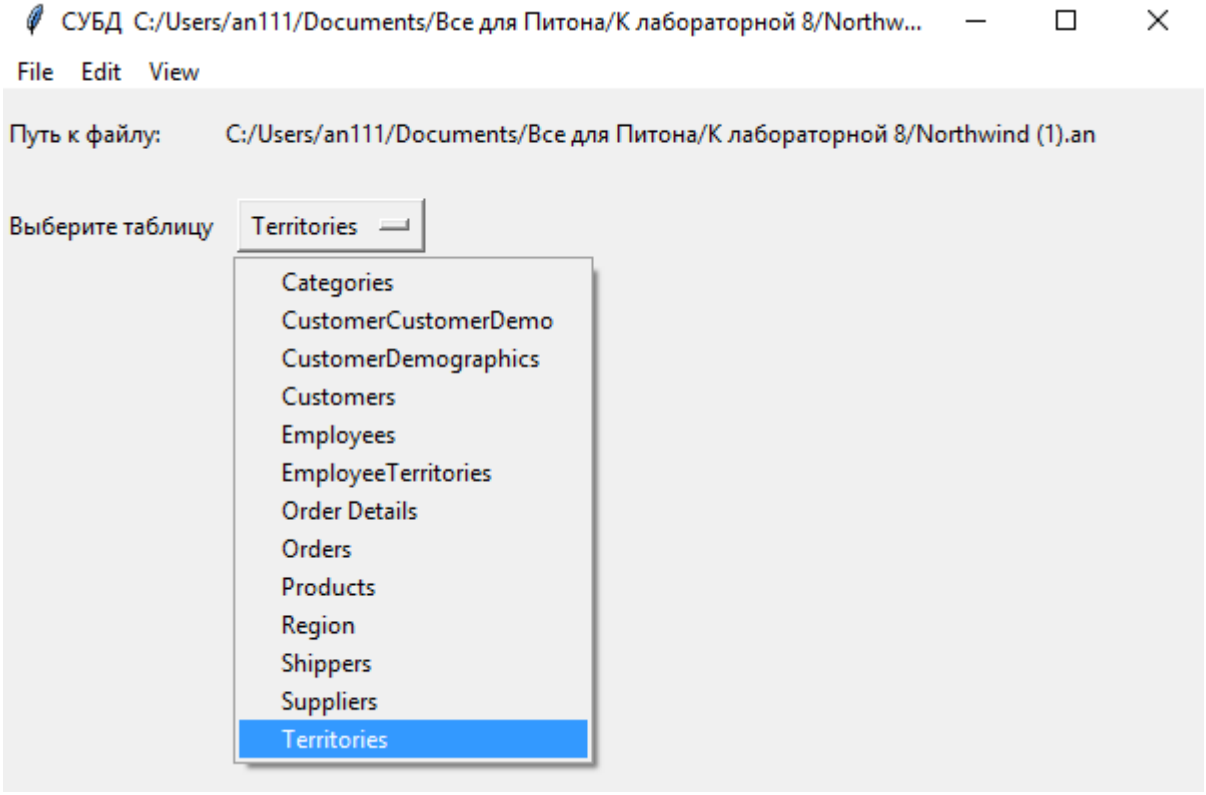


Рисунок 8.12 – Элементы интерфейса разрабатываемого приложения, появляющиеся после нажатия на кнопку Открыть

Примечание: приложение предлагает выбрать для открытия таблицу БД (демонстрируется поле со списком). В рассматриваемом примере открывается таблица Territories. Таблица будет отображена в зависимости от выбранного переключателя (5, 10, 15 строк таблицы).

5. Выбор количества отображаемых строк таблицы БД

Выбор количества отображаемых строк таблицы БД также описывается при помощи функции.

На рисунке 8.13 показан фрагмент кода, отвечающий за функцию создания выбора количества отображаемых строк таблицы БД.

```
# Функция создания выбора количества строк
def RB(event):
    global sel
    global nex
    sels = [("5", 5), ("10", 10), ("15", 15)]
    sel = IntVar()
    se = Label(text="Показать по: ")
    se.grid(row=5,column=2, sticky=W)
    col = 3
    for txt, val in sels:
        Radiobutton(text=txt, value=val, variable=sel, padx=15, pady=10,command=But)\
            .grid(row=5,column=col, sticky=W)
        col += 1
```

Рисунок 8.13 – Фрагмент кода, отвечающий за функцию создания выбора количества отображаемых строк таблицы БД

На рисунке 8.14 показаны элементы интерфейса, связанные с фрагментом кода, отвечающим за функцию создания выбора количества отображаемых строк таблицы БД.

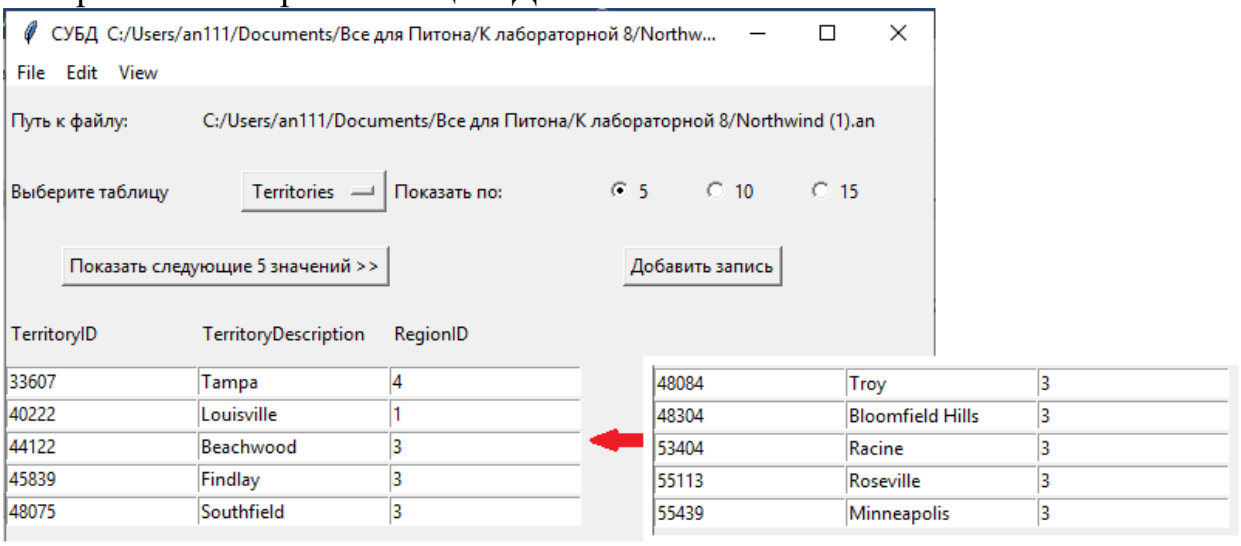


Рисунок 8.14 – Элементы интерфейса, связанные с фрагментом кода, отвечающим за функцию создания выбора количества отображаемых строк таблицы БД

Примечание: при нажатии на кнопку Показать следующие 5 значений отображаемые строки БД будут обновляться.

6. Для корректной работы приложения предусмотрены функции очистки окна и закрытия окна приложения (рисунок 8.15).

```
# Функция очистки окна
def clear():
    list = root.grid_slaves()
    for l in list:
        l.destroy()

# Функция закрытия окна
def Close():
    root.destroy()
```

Рисунок 8.15 – Функции очистки окна и закрытия окна приложения

Функция очистки окна необходима тогда, когда загружается новая БД. При использовании команды главного меню File, New Base открывается окно для выбора файла, при этом главное окно программы очищается для того, чтобы отобразить информацию в соответствии с новой открываемой БД.

Функция закрытия окна необходима для того, чтобы организовать в главном меню команду Exit (Выход).

Примечание: далее будет показан процесс создания команд меню, так, например, для организации команды Exit (Выход) применяется строка кода file_menu.add_command(label="Exit",command=Close).

На рисунке 8.16 показано очищенное окно, возникающее при открытии файла новой БД.

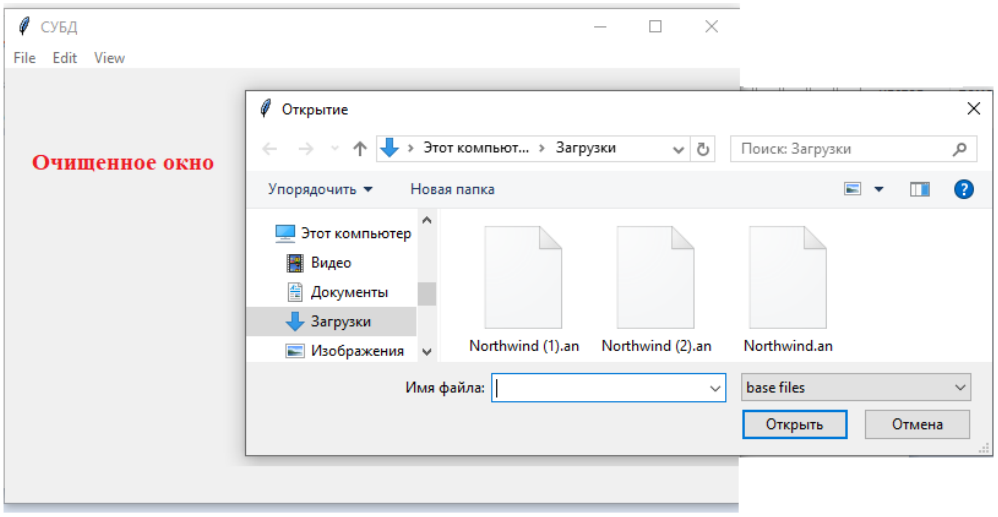


Рисунок 8.16 – Очищенное окно, возникающее при открытии файла новой БД

7. Обозначение имени полей БД

При отображении в окне приложения данных из таблиц БД необходимо, чтобы эти данные были подписаны, то есть необходимо в окне приложения отобразить названия полей (столбцов) таблиц БД. Отображение названия полей таблиц БД также осуществляется при помощи функции.

На рисунке 8.17 показан фрагмент кода, связанный с созданием функции обозначения имени полей таблиц БД.

```
# Функция обозначения имени полей таблиц БД
def Table():
    global nex
    global q
    global w
    s=sel.get()
    q=0
    for i in range(7,25):
        list = root.grid_slaves(row=i)
        for l in list:
            l.destroy()
    list = root.grid_slaves(row=7)
    for l in list:
        l.destroy()
    table_name=var.get()
    cursor.execute("PRAGMA table_info("+table_name+")")
    ciao = cursor.fetchall()
    w=0
    HeaderList=[]
    for i in ciao:
     counter=0
     for a in i:
      counter+=1
      if(counter==2):
        HeaderList.append(a)
        stolb=Label(text=a,justify=CENTER)
        stolb.grid(row=7, column=w, sticky="w",pady=10)
        w=w+1
    Table1()
```

Рисунок 8.17 – Фрагмент кода, связанный с созданием функции обозначения имени полей таблиц БД

На рисунке 8.18 показаны элементы интерфейса приложения, связанные с функцией обозначения имени полей таблиц БД.

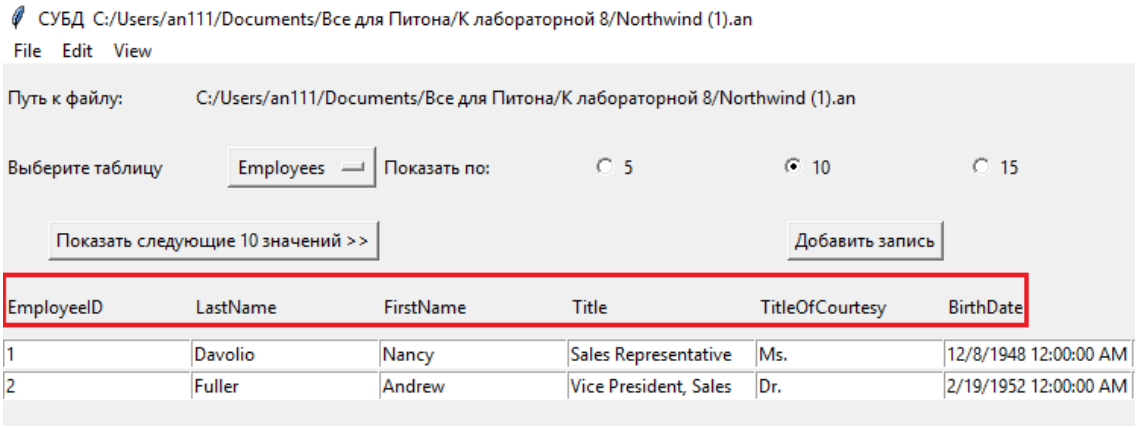


Рисунок 8.18 – Элементы интерфейса приложения, связанные с функцией обозначения имени полей таблиц БД

Примечание: выше отображена таблицы БД Emploees (соответствующие названия полей выделены цветом).

На рисунке 8.19 показаны названия полей таблицы БД Customers.

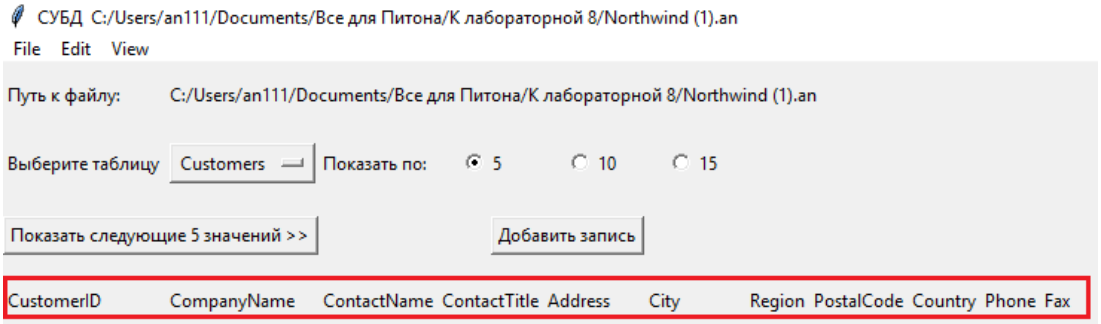


Рисунок 8.19 – Названия полей таблицы БД Customers

8. Отображение данных таблицы

Отображение данных выбранной таблицы также происходит при помощи функции (рисунок 8.20).

```
# Функция отображения данных таблицы
def Table1():
    global q
    global s
    s = sel.get()
    for i in range(8,25):
        list = root.grid_slaves(row=i)
        for l in list:
            l.destroy()
    x=8
    z=0
    table_name=var.get()
    cursor.execute("SELECT * FROM "+table_name)
    rows = cursor.fetchall()
    for row in rows:
        y=0
        for ro in row:
         if (z>=q):
            r=Entry()
            r.insert(0,ro)
            r.grid(row=x, column=y, sticky="w",pady=1)
            y=y+1
        z=z+1
        x=x+1
        if ((z%s)==0):
            x=8
        if (z==(q+s)):
            break
    q=q+s
    nex.bind(Table1)
```

Рисунок 8.20 – Отображение данных выбранной таблицы при помощи функции

Организованные циклы необходимы для отображения полей Entry, которыми изображается решетка для отображения записей таблицы БД, при этом поля Entry привязаны к количеству полей таблицы БД (чем больше полей БД, тем больше полей Entry потребуется для отображения реквизитов записей).

9. Добавление записи в таблицы БД

а) Создание дочернего окна приложения

Для того, чтобы добавить запись в таблицу БД необходимо предусмотреть поле в окне приложения для внесения этой записи. С этой целью предусмотрена функция создания дочернего окна приложения (рисунок 8.21).

```
# Функция создания дочернего окна
def childl():
    global child
    global sh
    child = Toplevel()
    child.title('Ввод данных')
    child.geometry('350x200+400+30')
    table_name=var.get()
    cursor.execute("PRAGMA table_info("+table_name+")")
    ciao = cursor.fetchall()
    w=0
    sh=0
    date=list()
    cu=Label(child,text="Введите значения в поле ввода",justify=CENTER)
    cu.grid(row=0, column=0, sticky="w",pady=10,columnspan=43)
    HeaderList=[]
    for i in ciao:
     counter=0
     for a in i:
      counter+=1
      if(counter==2):
        HeaderList.append(a)
        stolb=Label(child,text=a,justify=CENTER)
        stolb.grid(row=1, column=w, sticky="w",padx=10,pady=3)
        w=w+1
```

Рисунок 8.21 – Функция создания дочернего окна приложения

Дочернее окно приложения для внесения новой записи в таблицу БД показано на рисунке 8.22

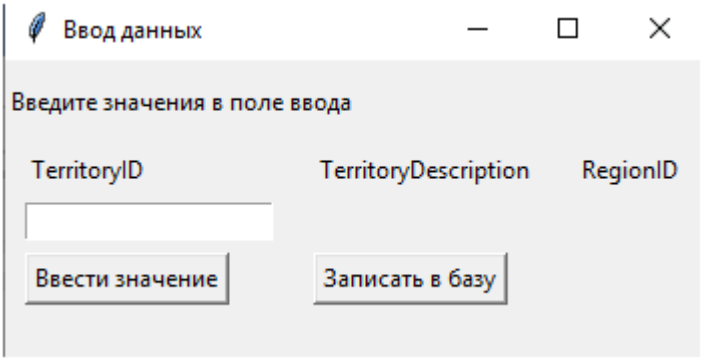


Рисунок 8.22 – Дочернее окно приложения для внесения новой записи в таблицу БД

Примечание: дочернее окно приложения, содержащее поле для внесения новой записи в таблицу БД, открывается при нажатии на кнопку Добавить запись главного окна приложения.

б) Добавление записи в таблицу БД по кнопке на форме

Для добавления записи в таблицу БД по кнопке на форме так же предусмотрена соответствующая функция (рисунок 8.23).

```
# Функция добавления записи в таблицу по кнопке на форме
def da():
    global a
    def dannie():
        global sh
        global a
        dann = Label(child,text=message.get())
        dann.grid(row=2, column=sh, sticky="w",padx=10,pady=3)
        d.append(message.get())
        sh = sh+1
        dannie_entry.delete(0,END)
        print(d,sh)
    def zapis():
        a="','".join(d)
        print(a)
        a="('"+a+"')"
        print(a)
        table_name=var.get()
        cursor.execute("INSERT INTO "+table_name+" VALUES "+a)
        conn.commit()
        child.destroy()

    d=[]
    message = StringVar()

    dannie_entry = Entry(child,text='',textvariable=message)
    dannie_entry.grid(row=3, column=sh, sticky="w",padx=10,pady=3)

    dannie_button = Button(child,text="Ввести значение", command=dannie)
    dannie_button.grid(row=4, column=0, sticky="w",padx=10,pady=3)

    zap_button = Button(child,text="Записать в базу", command=zapis)
    zap_button.grid(row=4, column=1, sticky="w",padx=10,pady=3)
```

Рисунок 8.23 – Фрагмент кода по созданию функции, отвечающей за добавление записи в таблицу БД по кнопке на форме

Представленный фрагмент кода организует внесение записи в таблицу БД следующим образом:

- в поле Entry проставляется значение, соответствующее первому полю таблицы БД, затем нажимается кнопка Ввести значение (в примере это поле Territory ID);

- в то же поле Entry проставляется значение, соответствующее второму полю таблицы БД, затем нажимается кнопка Ввести значение (в примере это поле Territory Description);

- далее процедура повторяется до тех пор, пока не будут внесены соответствующие реквизиты, составляющие запись (в соответствии с полями таблицы БД; в примере это третье поле Region ID);

- затем нажимается кнопка Записать в базу.

После этого можно просмотреть записи БД, не обновляя работу программы, то есть не перезапуская ее. Примечание: при отладке указанного фрагмента кода следует помнить, что операционная система защищает компьютер от записи файлов нелицензионных программных продуктов, поэтому запись в БД может быть внесена только если запущен программный код или с разрешения администратора операционной системы.

Процесс добавления записи в таблицу БД по кнопке на форме представлен на рисунке 8.24.

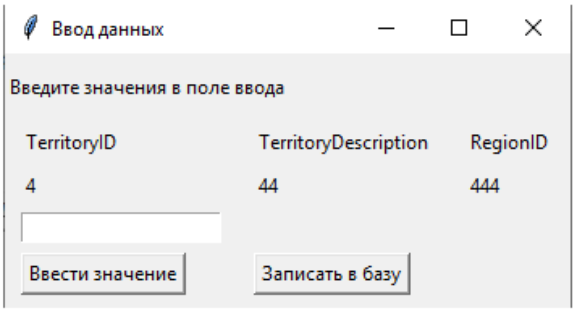


Рисунок 8.24 – Процесс добавления записи в таблицу БД по кнопке на форме

На рисунке 8.25 показана сохраненная в таблице Territories запись.

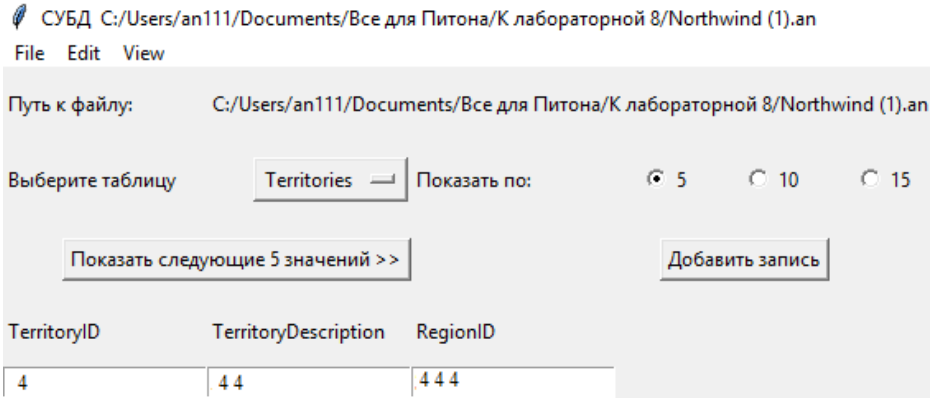


Рисунок 8.25 – Сохраненная в таблице Territories запись

в) Добавление записи в таблицу БД по команде главного меню

В разрабатываемом приложении учитывается возможность внесения новой записи в таблицу БД через команду главного меню, что также реализуется посредством создания функции (рисунок 8.26).

```
# функция добавления записи в таблицу по команде главного меню

def InsertDate ():
    a=var.get()
    if (a==''):
       messagebox.showinfo('СУБД',('Откройте базу и выберите таблицу!!!'))
    else:
       child1()

def InsertTable ():
    global file_name
    if(file_name==''):
        messagebox.showinfo('СУБД',('Откройте (создайте) базу!!!'))
    else:
        child2 = Toplevel()
        child2.title('Ввод данных')
        child2.geometry('350x200+400+30')
```

Рисунок 8.26 – Фрагмент кода, отвечающий за создание функции для добавления новой записи в таблицу БД через команду главного меню

10. Создание главной формы, главного меню и работа с переменными для виджетов Label и Entry

После того, как созданы все необходимые функции, для разрабатываемого приложения необходимо создать главное окно (главную форму), главное меню и определить работу с переменными для виджетов Label и Entry.

Соответствующий код программы представлен на рисунке 8.27.

```
#Создание главной формы
root = Tk()
root.title('СУБД')
root.geometry("600x600+370+40")

#Работа с переменными для Label и Entry
dir3 = StringVar()
fn=StringVar()
selec=IntVar()
var=StringVar()

file_name=''

#Создание Меню
main_menu = Menu()

file_menu = Menu()
file_menu.add_command(label="New base",command=NewDir)
#file_menu.add_command(label="Save base",command=Save)
file_menu.add_command(label="Open base",command=Open)
file_menu.add_separator()
file_menu.add_command(label="Exit",command=Close)

main_menu.add_cascade(label="File",menu=file_menu)

edit_menu = Menu()
edit_menu.add_command(label="Insrt table",command=InsertTable)
edit_menu.add_command(label="Insert date",command=InsertDate)
edit_menu.add_separator()
edit_menu.add_command(label="Exit")
main_menu.add_cascade(label="Edit",menu=edit_menu)
main_menu.add_cascade(label="View")
root.config(menu=main_menu)

root.mainloop()
```

Рисунок 8.27 – Фрагмент кода программы по созданию главной формы, главного меню и работа с переменными для виджетов Label и Entry

В представленном фрагменте кода программы:

- способом, характерным для Python, создается главное окно приложения;

- определяются типы переменных для виджетов Label и Entry;

- вызываются ранее созданные функции при использовании команд создаваемого главного меню.

Для отображения главного окна необходима кодовая конструкция *root.mainloop()*, которой завершается код для создания приложения с графическим интерфейсом (виджетами).

8.3. Оборудование и программное обеспечение

1. Персональная ЭВМ.

2. Операционная система Windows/Linux.

3. Среда программирования IDLE или другая среда, содержащая язык программирования Python.

4. MS Office, Libre Office (Open Office).

8.4. Задание на работу

1. Изучить теоретические положения лабораторной работы.

2. Получить результаты работы программ в соответствии с примером 15.

3. Модифицировать приложение, разработанное в соответствии с примером 15, следующим образом:

- количество записей таблицы БД отображалось по принципу 3,6,9,12 и более записей;

- при нажатии на кнопку для отображения записей предусмотрена возможность обновлять каждые 3 записи таблицы;

- дочернее окно приложении содержало кнопку для удаления записи из таблицы БД;

- дополнительное дочернее окно приложения отображало произвольный запрос на выборку данных из таблиц БД.

4. Разработать приложение, позволяющее работать с базой данных формата SQL (содержимое БД и интерфейс приложения на усмотрение разработчика).

5. Изобразить схемы работы программ в соответствии с ГОСТ 19.701-90 (использовать графический редактор).

6. Показать результаты работы преподавателю.

7. Ответить на контрольные вопросы.

8. Оформить отчет.

8.5. Контрольные вопросы

1. Какая библиотека используется в Python для работы с БД формата SQL?

2. Чем отличается формат БД от ее расширения?

3. Как в Python реализовать процесс соединения БД с приложением?

4. Как в Python реализовать внесение записей в таблицы БД?

5. Какие методы реализации запросов предусмотрены в Python?

6. Что такое функциональное программирование и как оно реализуется?

8.6. Правила оформления отчета

Отчет должен содержать:

1. Номер, название и цель лабораторной работы.

2. Описание процесса выполнения примеров (коды, схемы и результаты работы программ).

3. Вывод по работе.

Лабораторная работа №9
Создание баз данных формата SQLite и работа с ними в Python
9.1. Цель лабораторной работы
Научиться создавать базы данных формата SQLite и работать с ними в Python
9.2 Теоретические положения

Проект базы данных начинается с построения инфологической модели.

Реляционные БД предусматривают следующие типы связей:

- один к одному (1-1);
- один ко многим или многие к одному (1-n или n-1);
- многие ко многим (n-n).

Связь 1-1 в реальном мире встречается тогда, когда один объект содержит в соподчинении только один объект, например, государство – президент, ВУЗ – ректор, предприятие – генеральный директор и прочее. Связь 1-1 самая простая связь, все что представлено в таблицах, объединенных связью 1-1 можно представить в одной таблице. Связь 1-1 используется для более эффективной структуризации данных и более удобного прочтения массивов данных в БД.

Связь 1-n в реальном мире встречается тогда, когда один объект содержит в соподчинении много объектов (минимум два, так как в теории реляционных баз данных (РБД) все что не один – это много (несколько)), например, государство – граждане, ВУЗ – факультеты, предприятие – структурные подразделения и прочее.

Связь n-1 это связь 1-n, прочитанная справа налево (в соответствии со свойством реверсивности).

Связь n-n реальном мире встречается тогда, когда множество объектов содержат в соподчинении множество объектов, например, студенты – преподаватели, магазины – постоянные клиенты, сайты – посетители.

Связь n-n представляет собой две связи 1-n, читаемые слева направо и справа налево, поэтому этот тип связи читается с двух сторон, например, каждый студент взаимодействует со многими преподавателями, каждый преподаватель взаимодействует со многими студентами.

Связи в РБД устанавливаются благодаря наличию в таблицах ключевых полей (столбцов). Ключевое поле – это поле, значения которого не могут повторятся при внесении данных (записей) в таблицу.

В РБД связь 1-1 устанавливается по правилу: ключевое поле одной таблицы связывается с ключевым полем второй таблицы, причем в каждой из таблиц по одному ключевому полю.

В РБД связь 1-n устанавливается по правилам:

а) ключевое поле одной таблицы связывается с таким же не ключевым полем второй таблицы, причем в каждой из таблиц по одному ключевому полю;

б) ключевое поле одной таблицы связывается с таким же ключевым полем второй таблицы, причем в первой таблице одно ключевое поле, во второй таблице два ключевых поля – первичный и вторичный ключ.

Примечание: первичный ключ – ключ, встречающийся первый раз в системе двух связываемых таблиц. Вторичный ключ – ключ, пришедший во вторую таблицу из первой таблицы.

В РБД связь n-n устанавливается по правилу: ключевое поле первой таблицы связывается с таким же ключевым полем третьей промежуточной таблицы – с одной стороны; ключевое поле второй таблицы связывается с таким же ключевым полем третьей промежуточной таблицы – с другой стороны; причем первая и вторая связываемые таблицы имеют по одному ключевому полю, третья таблица является промежуточной и содержит два вторичных ключа, пришедших из первой и второй таблицы соответственно.

На рисунке 9.1 показан пример образования связи 1-1.

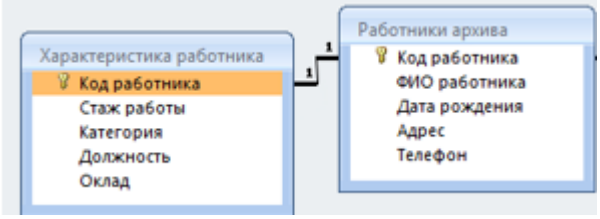


Рисунок 9.1 – Пример образования связи 1-1

Примечание: предполагается, что у каждого работника имеется только одна характеристика (на предприятии).

На рисунке 9.2 показан пример образования связи 1-n.

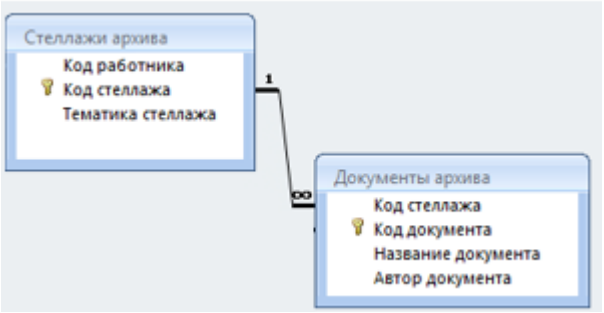


Рисунок 9.2 – Пример образования связи 1-n

Примечание: предполагается, что на каждом стеллаже, например, архива, хранится много документов.

На рисунке 9.3 показан пример образования связи n -n.

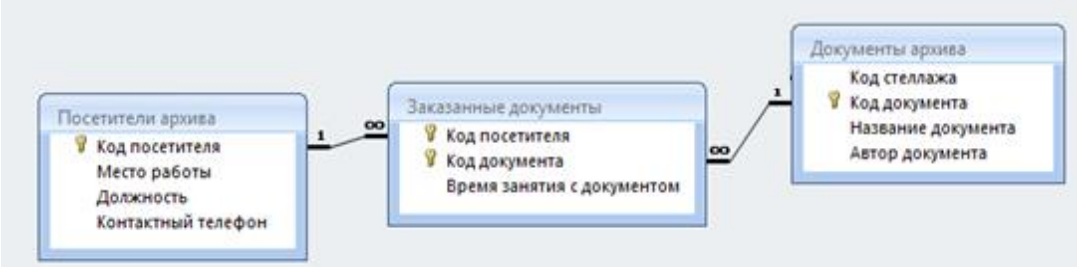


Рисунок 9.3 – Пример образования связи n-n

Примечание: предполагается, что каждый посетитель может взять несколько документов, но и каждый документ может быть взят несколькими посетителями.

Путем использования указанных правил получаются инфологические модели (схемы РБД), достаточно точно моделирующие информационные процессы.

В РБД таблицы заполняются данными в строгом порядке, с соблюдением иерархии связи. Порядок загрузки таблиц в РБД называются канонической моделью БД. Общий принцип построения канонической модели БД сводится к тому, что таблицы, стоящие на более высоком уровне («старшие» таблицы, из которых «сливаются» данные в другие таблицы) заполняются раньше, чем «младшие» таблицы.

Пример канонической модели РБД представлен на рисунке 9.4.

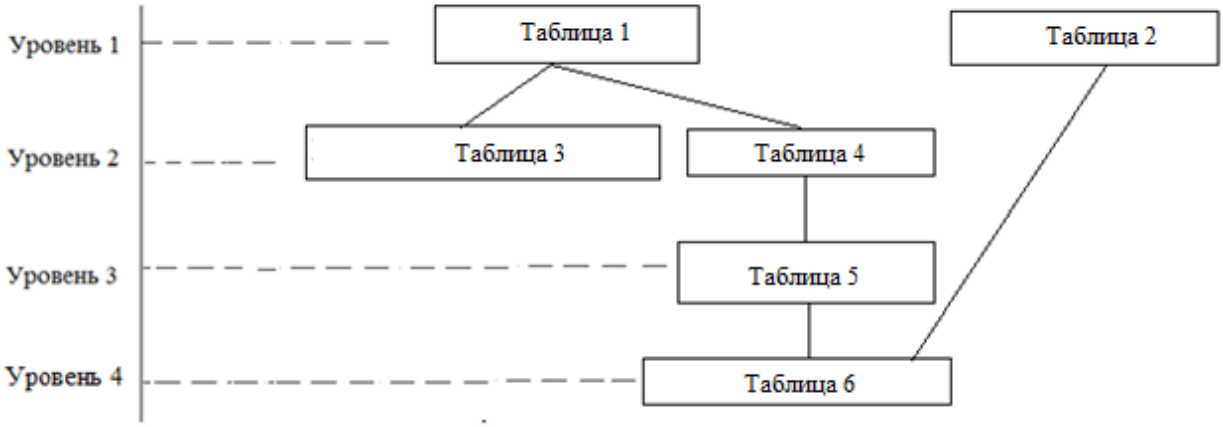


Рисунок 9.4 – Пример канонической модели РБД

Для работы с базой данной (БД) удобно использовать менеджер БД, с помощью которого можно управлять базой в визуальном режиме. Для СУБД SQLite самый удобный и распространенный 

менеджер – это SQLiteStudio. Далее рассмотрен менеджер SQLiteStudio 3.2.1.

Пример 16

Создать в SQLiteStudio РБД Знания экспертов, инфологическая модель которой представлена на рисунке 9.5.

РБД Знания экспертов имеет таблицы и структуру, представленные ниже. РБД Знания экспертов, состоит из следующих таблиц:

- дефекты;
- оценки причин 1 уровня;
- оценки причин 2 уровня;
- оценки причин 3 уровня;
- причины 1 уровня;
- причины 2 уровня;
- причины 3 уровня;
- эксперты;
- эксперты и дефекты.

Структуры таблиц РБД Знания экспертов представлены в таблицах 9.1 – 9.10.

Таблица 9.1 – Структура таблицы Дефекты

Имя поля	**Тип данных**
Код дефекта	Счетчик
Название дефекта	Текстовый

Таблица 9.2 – Структура таблицы Оценки причин 1 уровня

Имя поля	**Тип данных**
Номер записи	Счетчик
Код эксперта	Числовой
Код причины 1 уровня	Числовой
Оценка причины 1 уровня	Числовой

Таблица 9.3 – Структура таблицы Оценки причин 2 уровня

Имя поля	**Тип данных**
Номер записи	Счетчик
Код эксперта	Числовой
Код причины 2 уровня	Числовой
Оценка причины 2 уровня	Числовой

Таблица 9.4 – Структура таблицы Оценки причин 3 уровня

Имя поля	**Тип данных**
Номер записи	Счетчик
Код эксперта	Числовой
Код причины 3 уровня	Числовой
Оценка	Числовой

Таблица 9.5 – Структура таблицы Причины 1 уровня

Имя поля	**Тип данных**
Код дефекта	Числовой
Код причины 1 уровня	Числовой
Название причин 1 уровня	Текстовый

Таблица 9.6 – Структура таблицы Причины 2 уровня

Имя поля	**Тип данных**
Код причины 1 уровня	Числовой
Код причины 2 уровня	Числовой
Название причин 2 уровня	Текстовый

Таблица 9.7 – Структура таблицы Причины 3 уровня

Имя поля	**Тип данных**
Код причины 2 уровня	Числовой
Код причины 3 уровня	Числовой
Название причин 3 уровня	Текстовый

Таблица 9.8 – Структура таблицы Эксперты и дефекты

Имя поля	**Тип данных**
Код эксперта	Числовой
Код дефекта	Числовой

Таблица 9.9 – Структура таблицы Эксперты

Имя поля	Тип данных
Код эксперта	Счетчик
ФИО эксперта	Текстовый
Место работы	Текстовый
Должность	Текстовый
Стаж работы	Текстовый

Таблица 9.9 – Структура таблицы Эксперты и дефекты

Имя поля	Тип данных
Код эксперта	Числовой
Код дефекта	Числовой

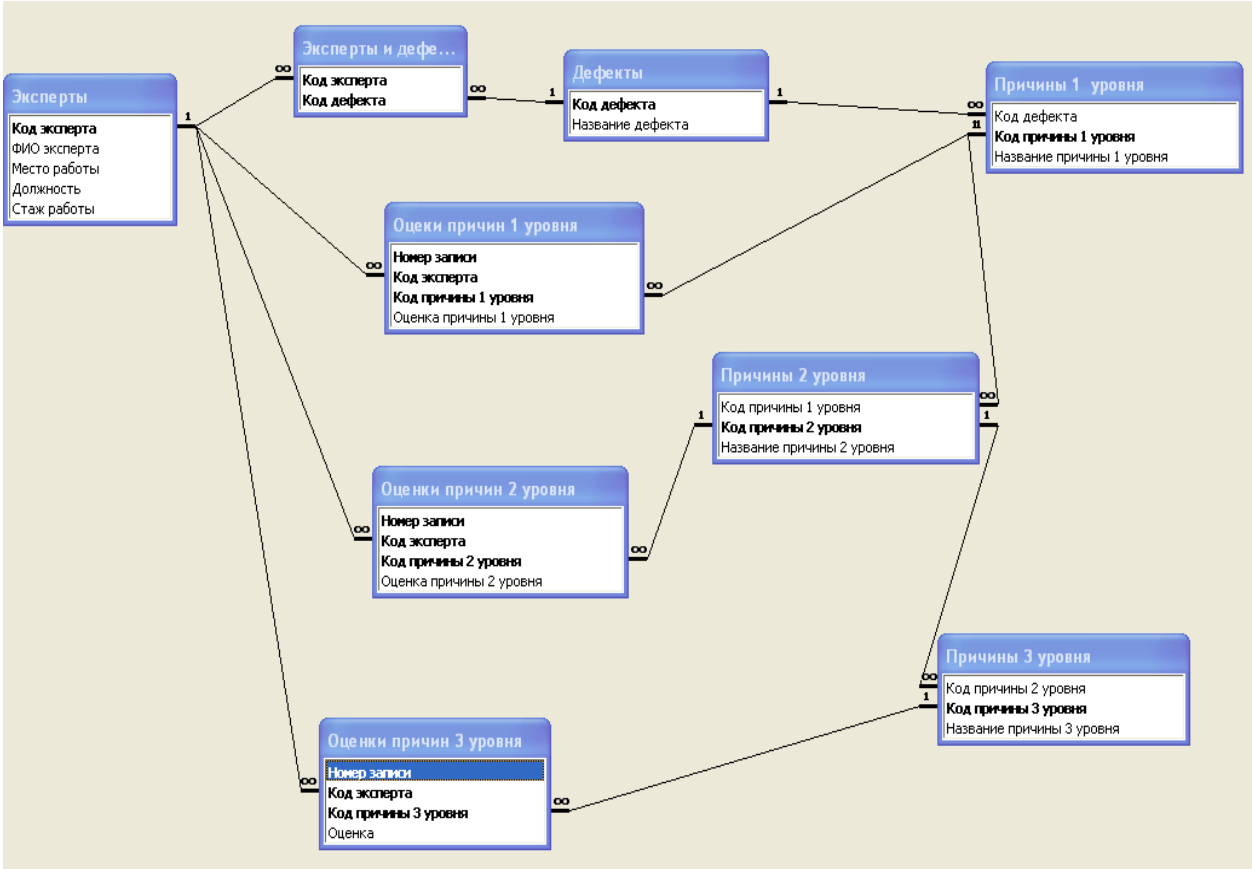


Рисунок 9.5 – Инфологическая модель РБД Знания экспертов

Примечание: Одна из таблиц в БД Знания экспертов играет роль буфера обмена информацией, куда временно загружаются данные после их обработки с целью выдачи на экран ПК информации по устранению указанных видов дефектов.

Технологии использования промежуточного буфера обмена информацией используются при проектировании и разработке информационных систем. Однако, не все технологии опираются на необходимость создания буфера обмена информацией, часто такая

информация может содержаться в правильно сконструированном запросе к РБД.

Таблица Буфер обмена не входит в инфологическую модель РБД Знания экспертов.

Таблица 9.10 – Структура таблицы Буфер обмена

Имя поля	**Тип данных**
Название дефекта	Текстовый
Название причины 1 уровня	Текстовый
Оценка причины 1 уровня	Числовой
Код причины	Числовой
Название причины 2 уровня	Текстовый
Оценка причины 2 уровня	Числовой
Название причины 3 уровня	Текстовый
Оценка причины 3 уровня	Числовой
Код	Числовой

Выполнение примера 16

Для открытия существующей базы или добавления новой надо нажать кнопку «&Add a database» или комбинацию клавиш Ctrl+O.

На рисунке 9.6 показан процесс открытия существующей (ранее сохраненной) БД.

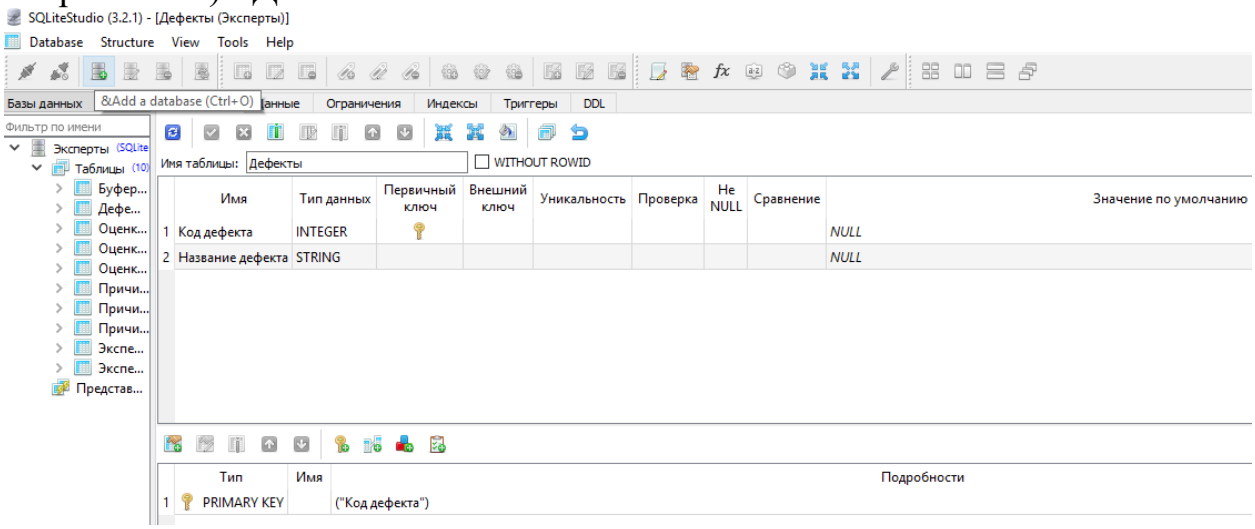


Рисунок 9.6 – Процесс открытия существующей (ранее сохраненной) БД

На рисунке 9.7 показан процесс открытия новой БД.

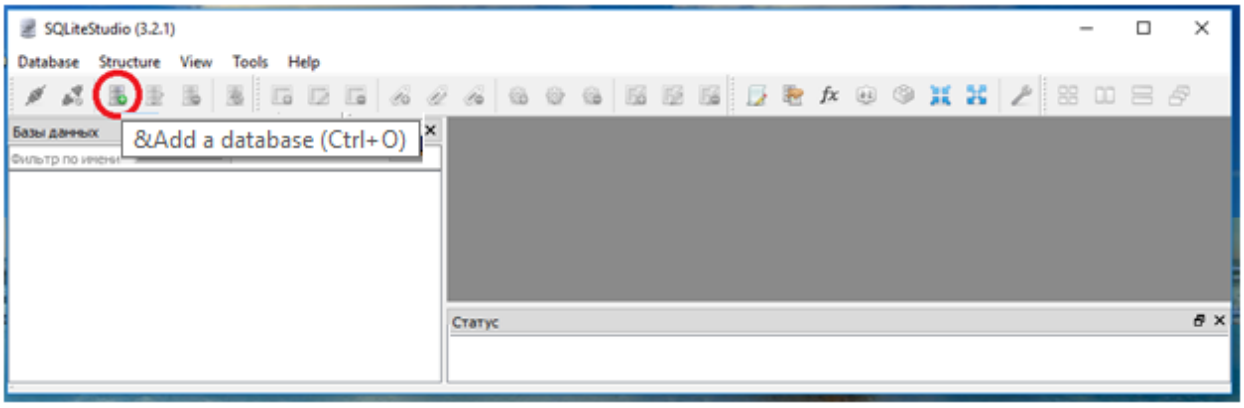


Рисунок 9.7 – Процесс открытия новой БД

После нажатия на кнопку «&Add a database», откроется окно База данных, в котором предусмотрены варианты создания/открытия БД (в соответствии с рисунком 9.8):

- 1 – необходимо сначала выбрать тип создаваемой или открываемой базы (в поле со списком);
- 2 – нажать кнопку для создания новой базы данных;
- 3 – нажать кнопку открытия существующей базы.

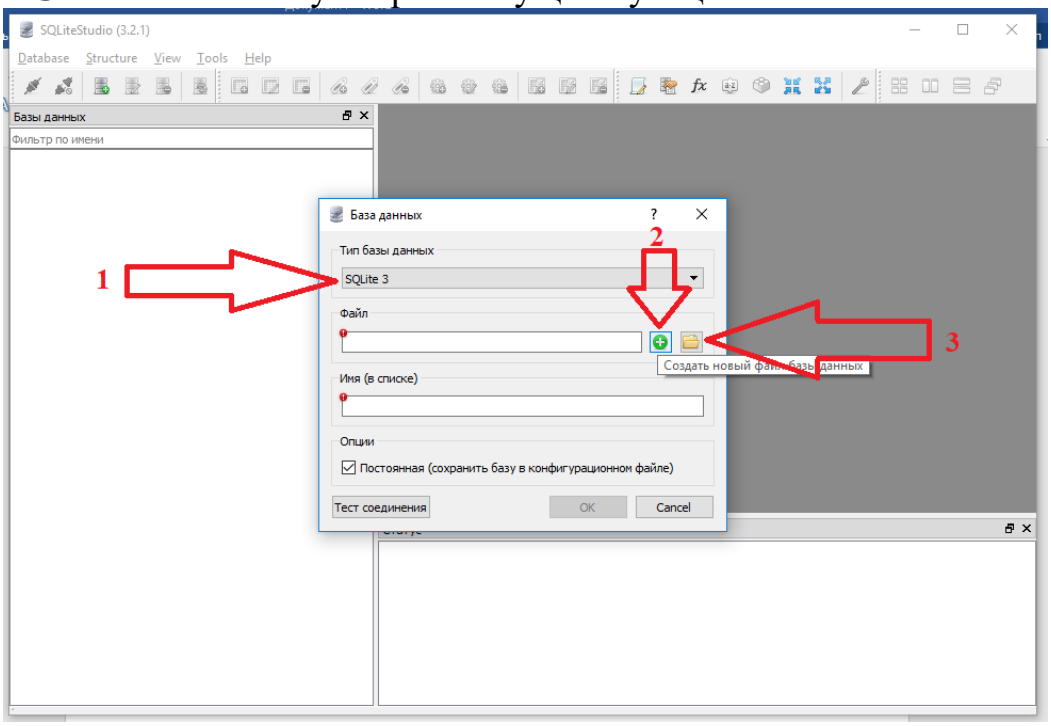


Рисунок 9.8 – Окно, в котором предусмотрены варианты работы с БД

Если выбрать вариант 2 (нажатие на кнопку ), то появится окно для сохранения новой БД выбранного формата (рисунок 9.9).

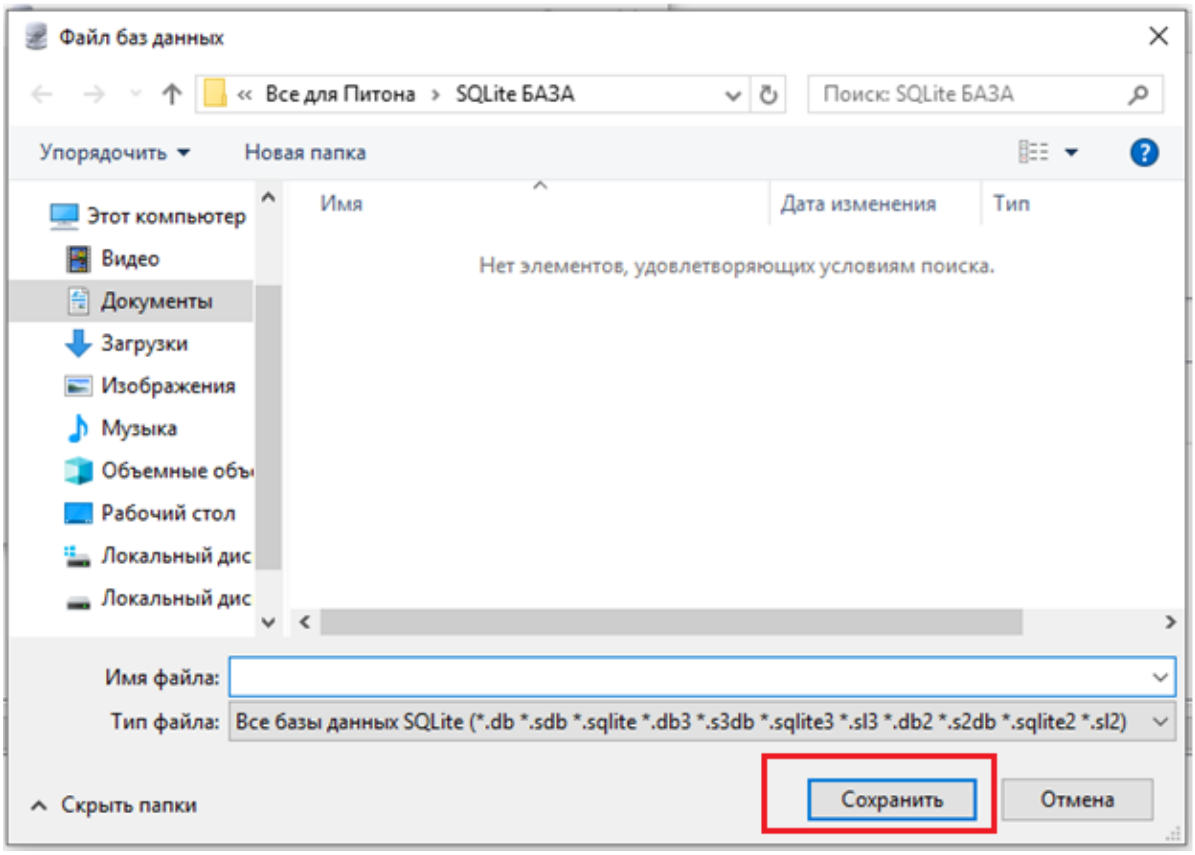


Рисунок 9.9 – Окно для сохранения новой БД

Если выбрать вариант 3 (нажатие на кнопку ), то появится окно для открытия БД выбранного формата (рисунок 9.10).

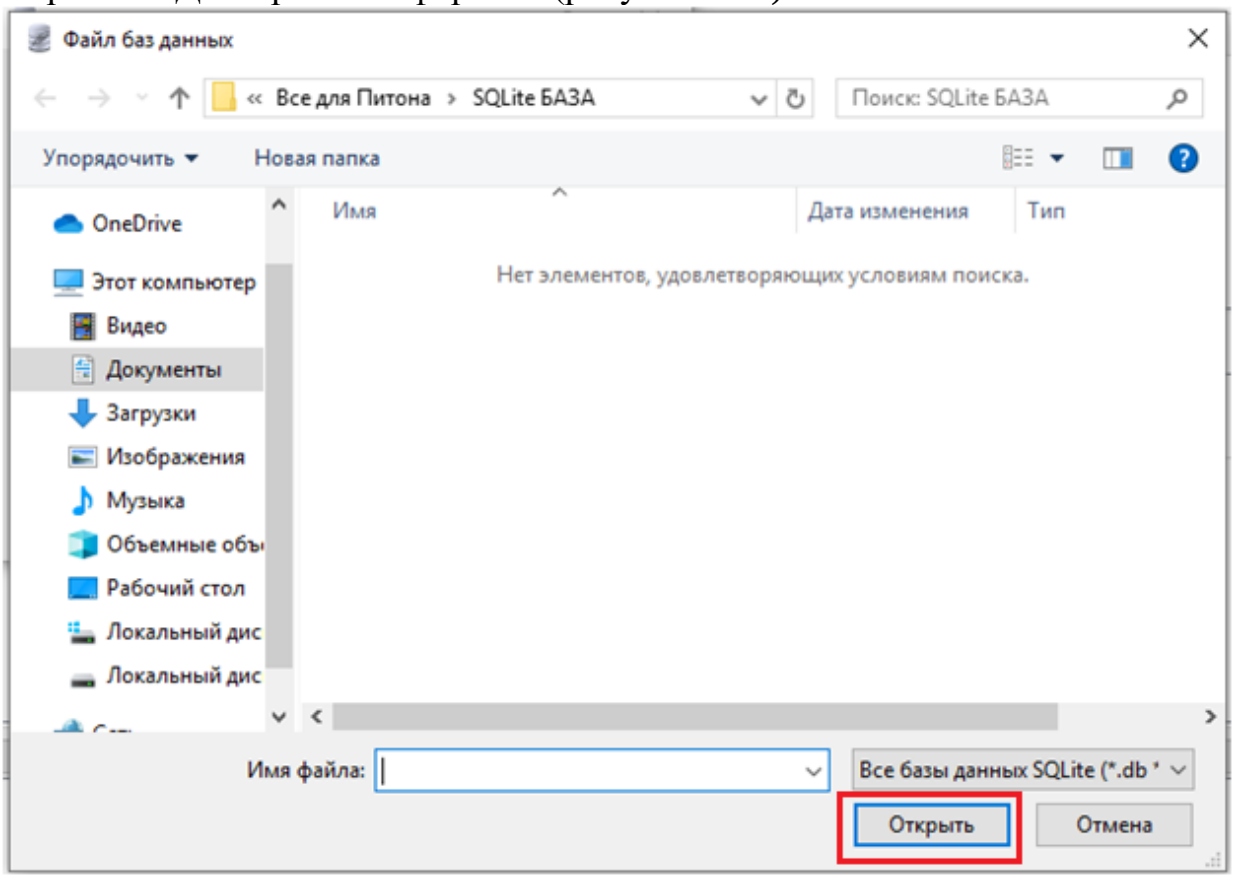


Рисунок 9.10 – Окно для открытия БД

Путь создаваемой или открываемой БД пропишется в советующем поле Файл окна База данных. После работы с диалогом окно База данных примет вид, как показано на рисунке 9.11.

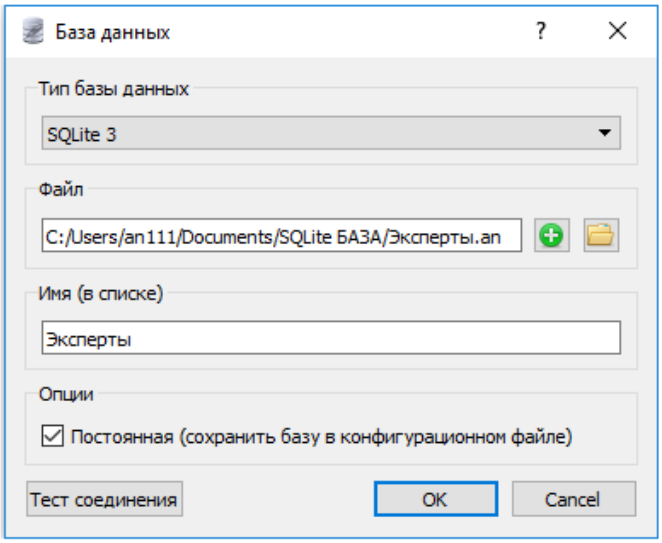


Рисунок 9.11 – Путь открываемой (в данном случае) БД

На рисунке 9.11:

- в поле Файл будет указан путь к создаваемой/открываемой БД;
- в поле Имя (в списке) будет указано имя, с которым БД будет отображаться в панели навигации Базы данных.

После нажатия кнопки Ок БД создана/открыта в указанном каталоге и отображается в панели навигации Базы данных.

На рисунке 9.12 в панели навигации Базы данных показана загруженная БД с именем Эксперты.

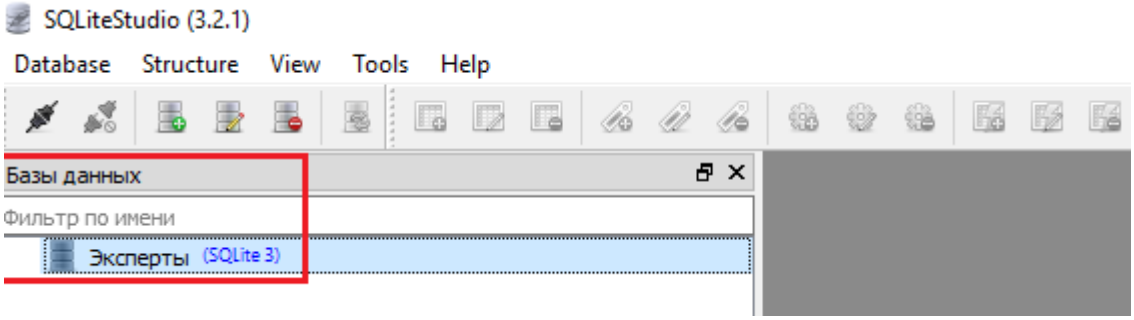


Рисунок 9.12 – Загруженная БД с именем Эксперты

Для дальнейшей работы с БД необходимо организовать к ней подключение. Для этого необходимо выбрать соответствующий пункт Connect to the database контекстного меню БД (правой кнопкой в панели навигации) или нажать соответствующую кнопку в панели управления.

На рисунке 9.13 показана команда контекстного меню Connect to the database.

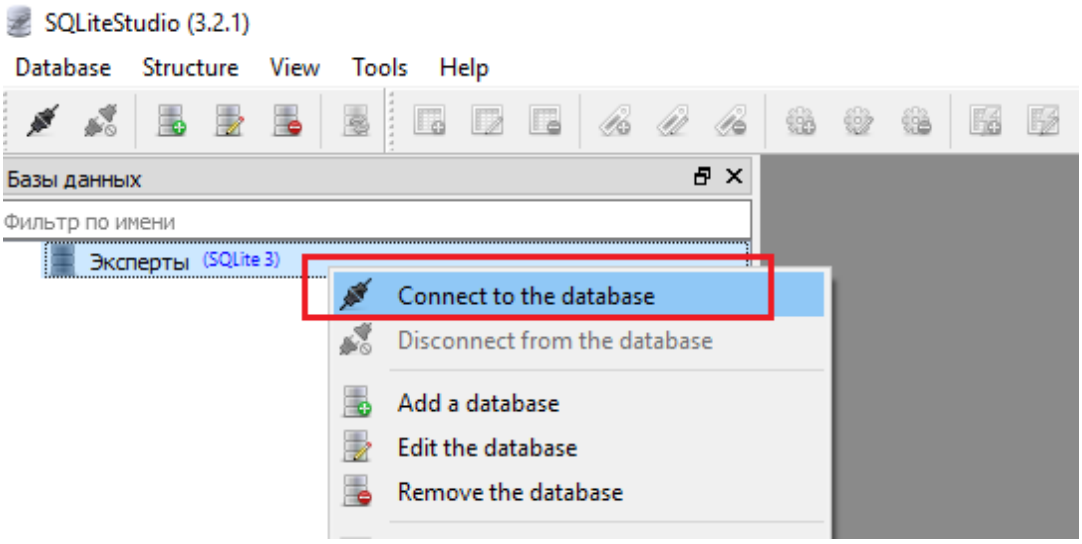


Рисунок 9.13 – Команда контекстного меню Connect to the database

На рисунке 9.14 показана кнопка Connect to the database.

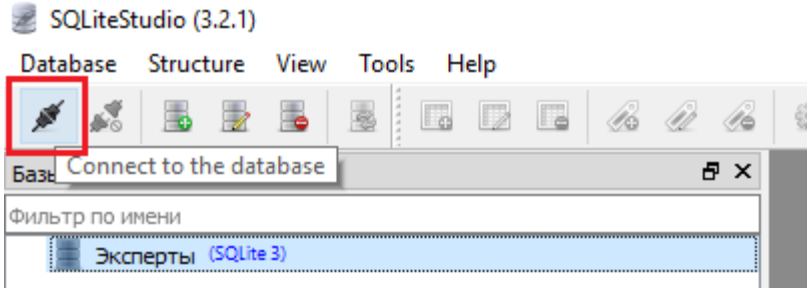


Рисунок 9.14 – Кнопка Connect to the database

Примечание: рядом располагается кнопка Disconnect from the database для прерывания соединения (рисунок 9.15).

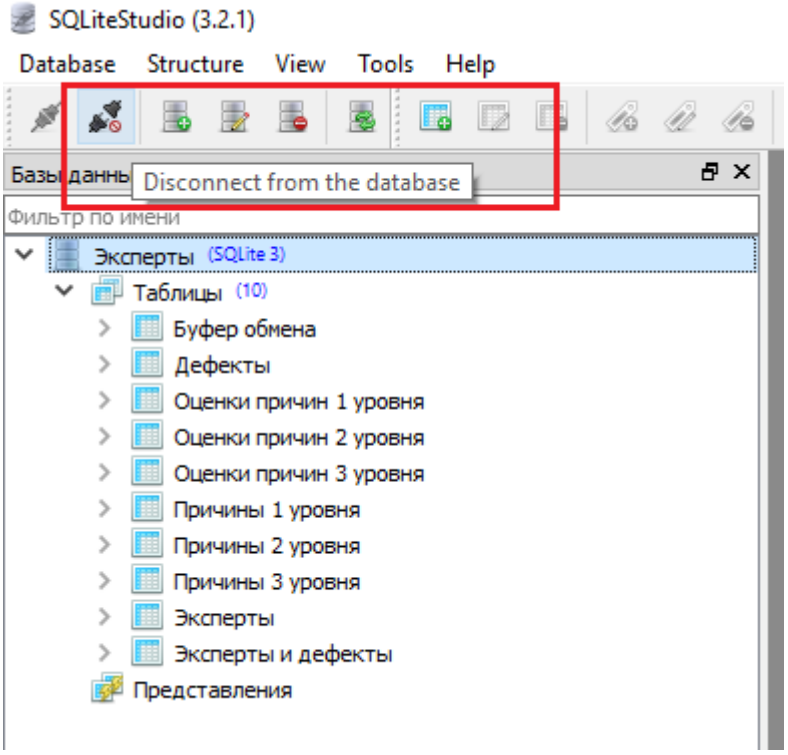


Рисунок 9.15 – Кнопка Disconnect from the database

Если открывалась ранее сохраненная БД, то после выбора команды Connect to the database отобразится структура БД (рисунок 9.16).

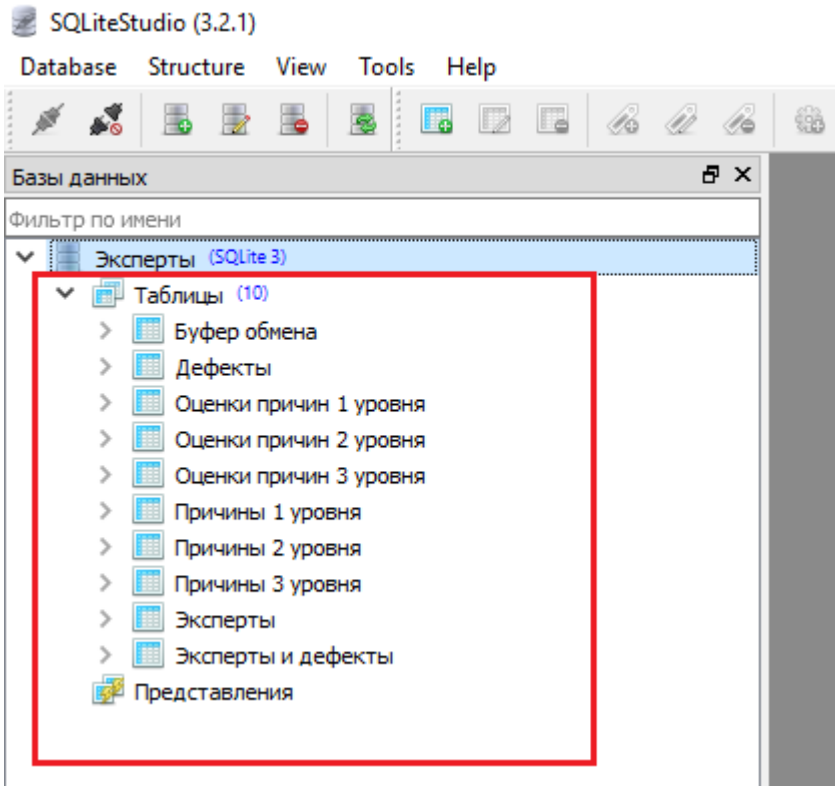


Рисунок 9.16 – Структура БД

Процесс работы с вновь созданной БД продолжается процессом создания таблиц базы данных. Для создания таблиц необходимо нажать соответствующую кнопку в панели управления Create a table (рисунок 9.17) либо выбрать соответствующий пункт контекстного меню Create a table в панели навигации (рисунок 9.18). После этого в правой части откроется модуль редактирования таблицы (рисунок 9.19).

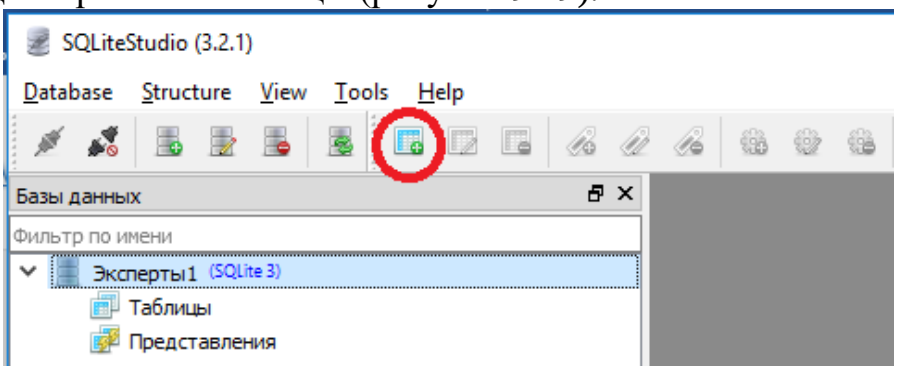


Рисунок 9.17 – Кнопка Create a table

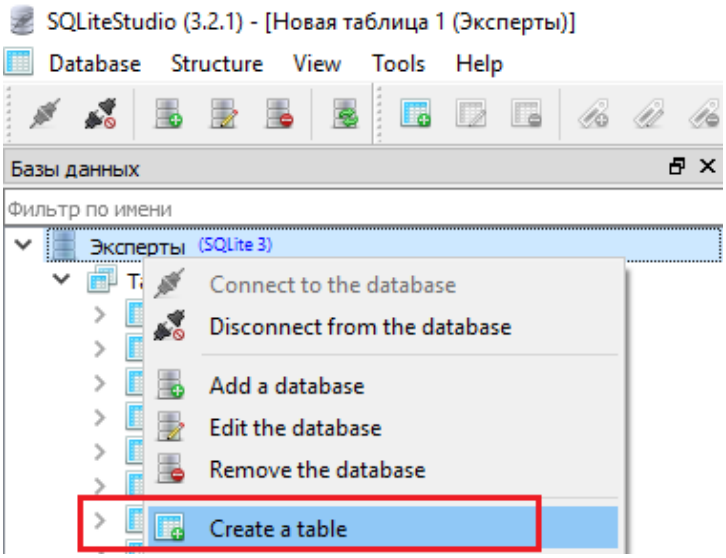


Рисунок 9.18 – Пункт контекстного меню Create a table

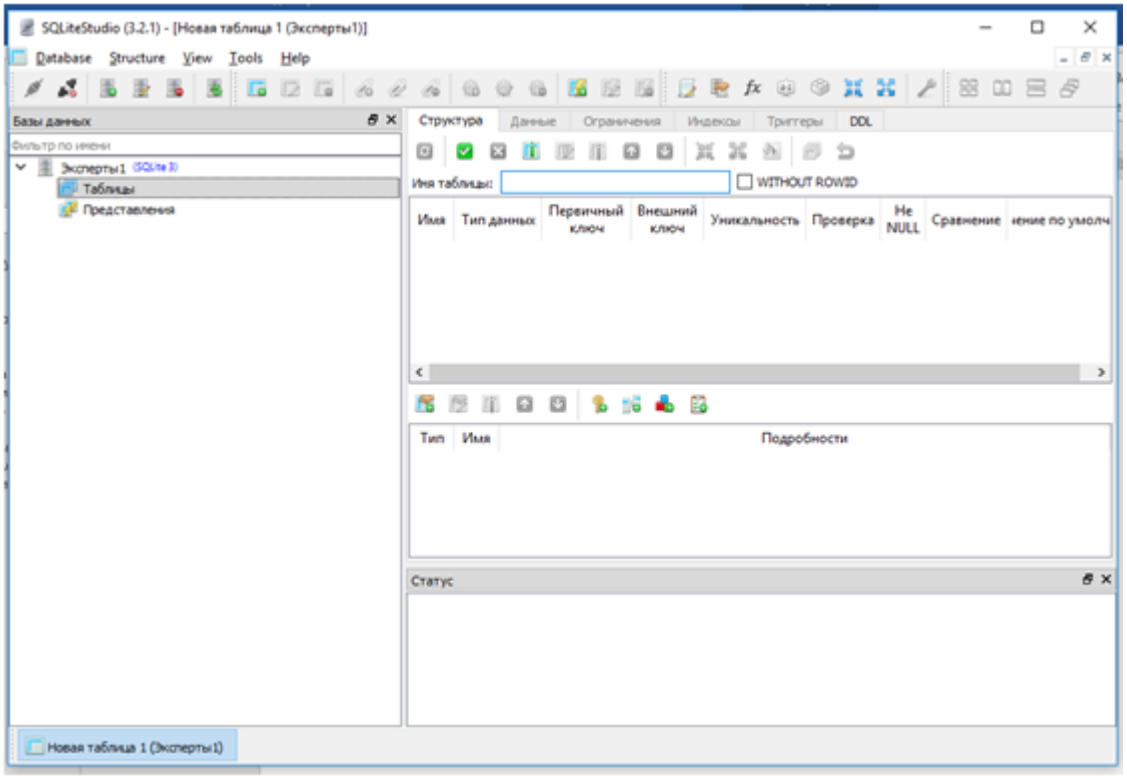


Рисунок 9.19 – Модуль редактирования таблицы

Модуль редактирования таблицы содержит несколько вкладок:

Вкладка Структура – Вкладка добавления столбцов (полей), которая, в свою очередь, состоит из трех блоков (как показано на рисунке 9.19):

- блок редактирования таблицы;
- блок подробности (блок добавления ограничений или связей между таблицами);
- блок статус (выполнено, не выполнено, наличие ошибок).

На рисунке 9.20 показана активированная вкладка Структура.

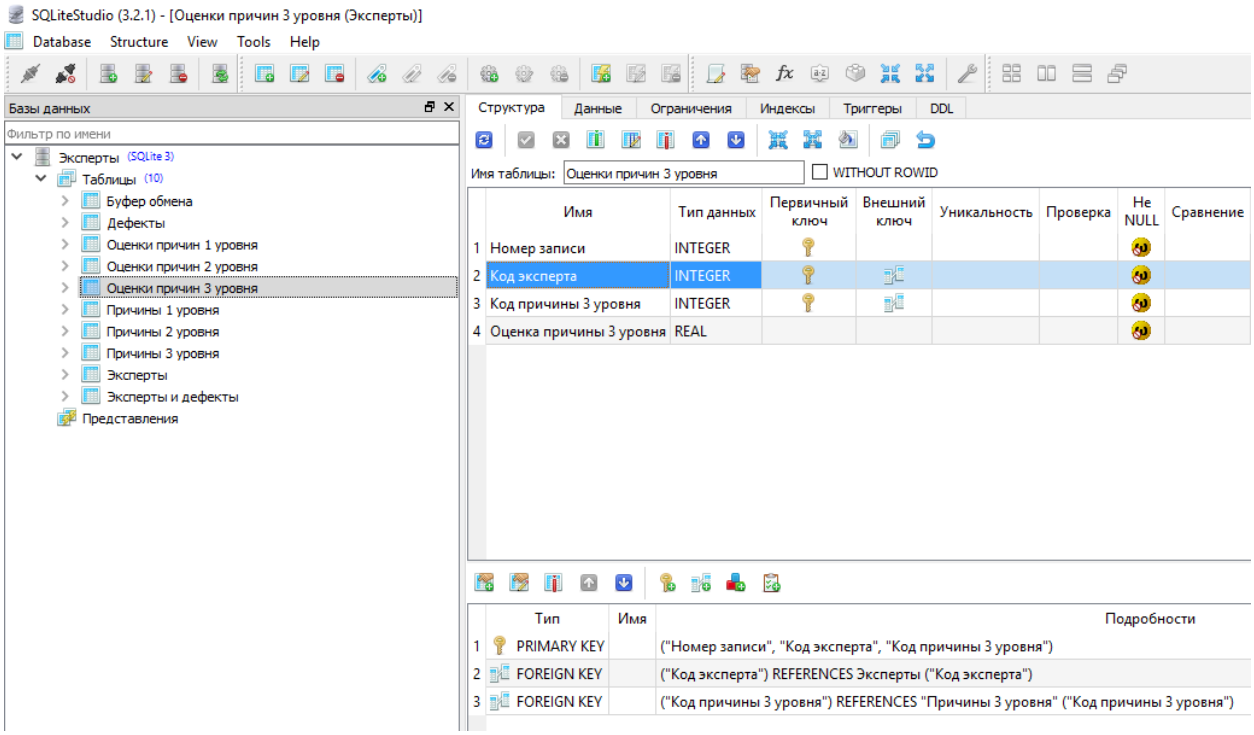


Рисунок 9.20 – Активированная вкладка Структура

Примечание: на рисунке 9.20 отображена структура таблицы Оценки причин 3 уровня (эта таблица выделена в панели навигации).

Блок редактирования таблицы вкладки Структура содержит ряд полезных кнопок для осуществления операций по редактированию таблиц (рисунок 9.21).



Рисунок 9.21 – Кнопки для осуществления операций по редактированию таблиц

Например, данные в таблицу можно импортировать или экспортировать. Также можно создать подобную таблицу, удалить переместить, добавить столбец и прочее.

Для редактирования столбцов (полей) таблиц необходимо вызвать окно редактирования (дважды щелкнуть мышью на поле Имя), после чего откроется окно Столбец (рисунок 9.22).

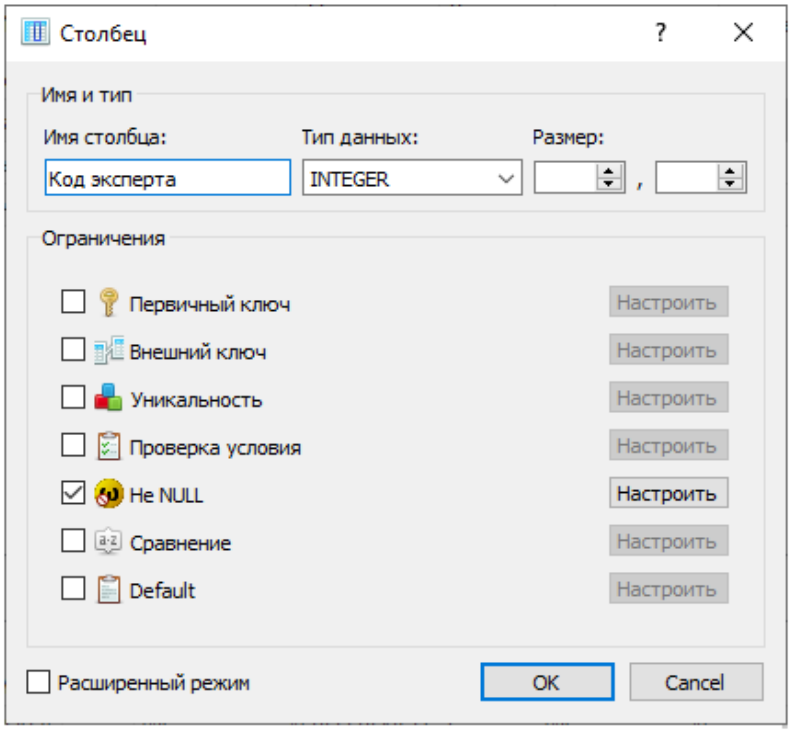


Рисунок 9.22 – Окно Столбец для редактирования столбцов (полей) таблиц БД

В окне Столбец задается имя столбца (поля), указывается тип поля, указывается размер поля. Также в окне Столбец задаются ограничения, к которым относятся:

- первичный ключ, устанавливаемый на поле;
- внешний ключ, указывающий на взаимосвязь с другими таблицами;
- уникальность (данные поля не могут повторяться);
- проверка условия, установленного на данные поля;
- необходимость установления ненулевого значения (Не NULL);
- сравнение (сравниваются между собой две строки одного столбца);
- ввод значения записи по умолчанию (Default).

В таблице 9.11 представлено описание ограничений.

Таблица 9.11 – Описание ограничений

Наименование ограничения	Описание необходимости установления ограничения	Примечание
Первичный ключ (Primary key)	Устанавливается в том случае если это столбец, значения которого должны быть уникальными, и SQLite никогда не нарушит правило уникальности первичного ключа. Так же первичный ключ является индексом таблицы в базе данных SQLite. Он не позволяет создавать одинаковых записей (строк) в таблице и обеспечивают логическую связь между таблицами одной базы данных (для реляционных БД)	При установке этого ограничения существуют настройки, (автоинкремент – обеспечивает автоматическое присвоение значения этому полю (счетчик), тип данных в этом случае должен быть INTEGER)

Продолжение таблицы 9.11

Наименование ограничения	Описание необходимости установления ограничения	Примечание
Внешний ключ Foreign key (кратко FK)	Обеспечивает однозначную логическую связь, между таблицами одной БД. Внешние ключи рекомендуется проставлять после того, как будут созданы все таблицы базы и будут продуманы связи между таблицами	
Уникальность	заставляет SQLite проверять уникальность значений в столбце таблицы	Если вы задали для столбца это ограничение, то SQLite не даст вам записать в него повторяющееся значение
Проверка условия	Данное ограничение задает диапазон значений, которые могут храниться в текущем столбце	Например, в столбце зарплата вы можете установить ограничение, что зарплата не может быть меньше МРОТ при помощи ЭТОГО ограничения
Не NULL	Контролирует, чтобы все поля, имеющие это свойство, были заполнены	
Сравнение	Способ сравнения двух строк между собой	Задается для столбца таблицы и определяет какой тип сравнения используется для значений этого столбца
Default	Задает значение записи по умолчанию	Значение добавляется автоматически в режиме добавления записей

Представленный список ограничений позволяет устанавливать ограничения как на таблицу, так и на отдельные поля таблицы. Для настройки ограничений предусмотрены соответствующие кнопки Настроить.

Вкладка Данные – вкладка добавления данных в таблицу.

На рисунке 9.23 показана активированная вкладка Данные.

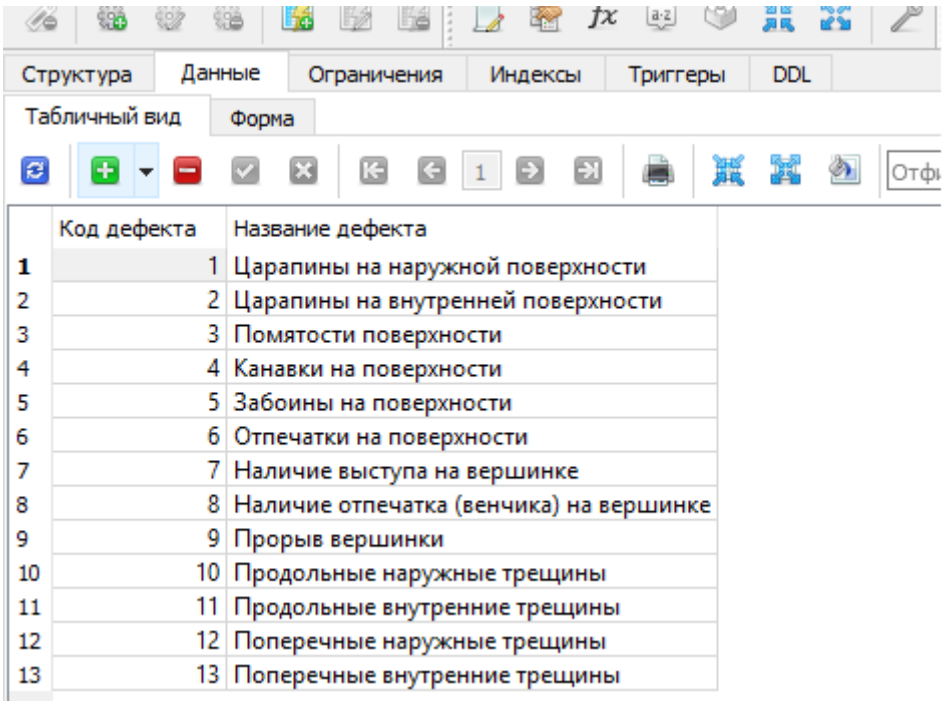


Рисунок 9.23 – Активированная вкладка Данные

Для добавления новой строки данных (новой записи) предусмотрена кнопка Вставить строку (рисунок 9.24).

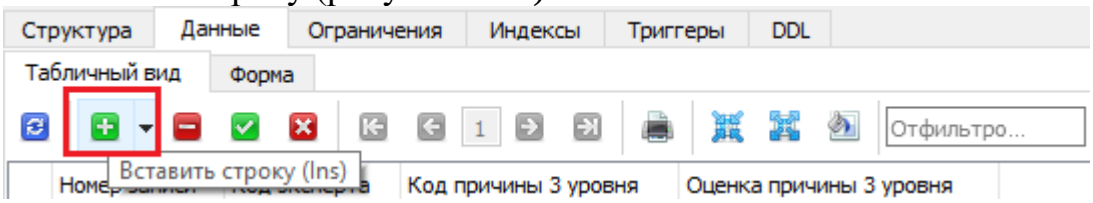


Рисунок 9.24 – Кнопка Вставить строку

Введенные данные необходимо подтвердить нажатием на кнопку Подтвердить (рисунок 9.25).

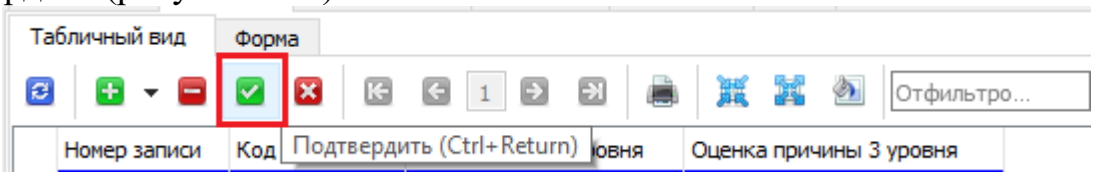


Рисунок 9.25 – Кнопка Подтвердить

Примечание: при быстром вводе записей целесообразно использовать горячие клавиши (комбинации горячих клавиш указаны рядом с соответствующими кнопками).

Вкладка Ограничения – вкладка просмотра и редактирования всех ограничений таблицы (ограничений всех полей таблицы).

На рисунке 9.26 показана активированная вкладка Ограничения для одной из таблиц.

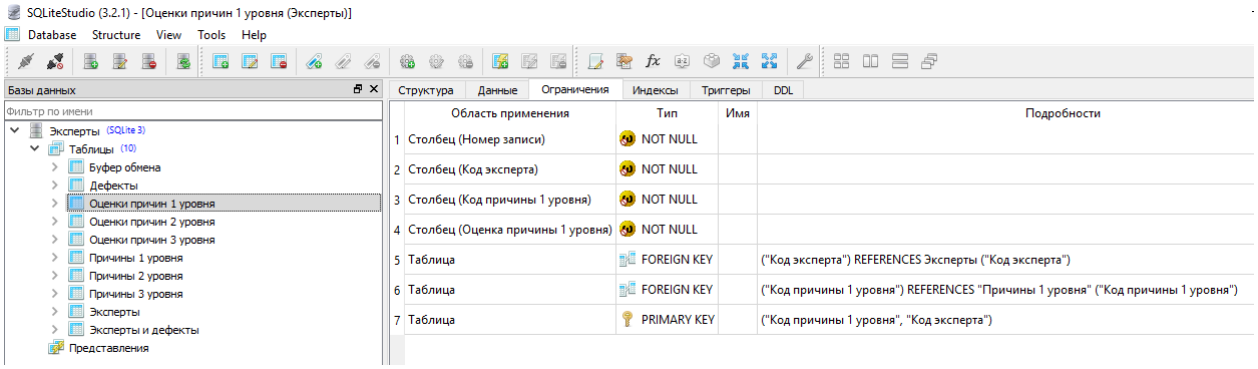


Рисунок 9.26 – Активированная вкладка Ограничения

Вкладка Индексы – вкладка создания индексов, основное назначение которых в БД заключается в том, чтобы ускорить операцию выборки данных из БД.

На рисунке 9.27 показана активированная вкладка Индексы.

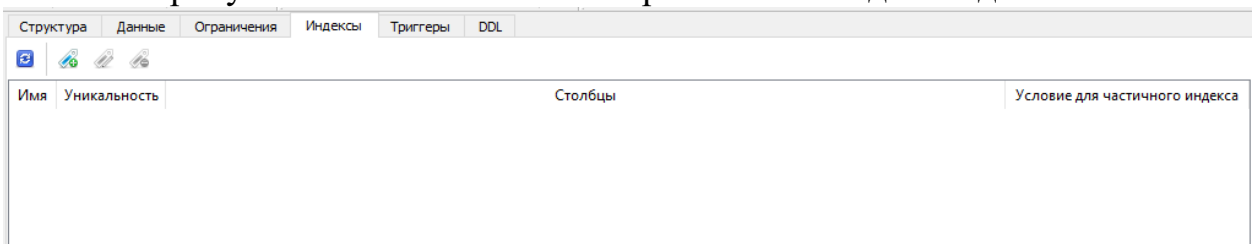


Рисунок 9.27 – Активированная вкладка Индексы

Вкладка Триггеры – вкладка создания триггеров.

Триггер – это особая разновидность хранимых процедур в БД.

Особенность триггеров заключается в том, что SQL код, написанный в теле триггера, будет исполнен после того, как в БД произойдет какое-либо событие. События в БД происходят в результате выполнения DML-команд или команд манипуляции данными.

Примечание: DML-команда – это команда, осуществляющая операции с записями БД.

На рисунке 9.28 показана активированная вкладка Триггеры.

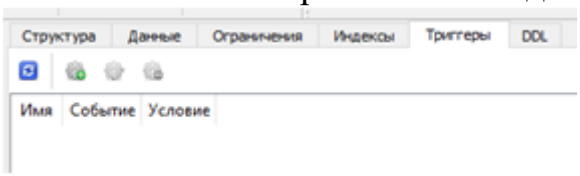


Рисунок 9.28 – Активированная вкладка Триггеры

Вкладка DDL – вкладка листинга основных операторов, возникающих в процессе создания таблиц БД.

На рисунке 9.29 показана активированная вкладка DDL.

Примечание: на рисунке 9.29 показан SQL-код, отображающий:

- процесс создания таблицы (с названием таблицы);
- процесс создания полей таблицы (с названием полей);
- процесс задания размерности полей таблицы;
- процесс ввода ограничений на поля таблицы или на всю таблицу в целом.

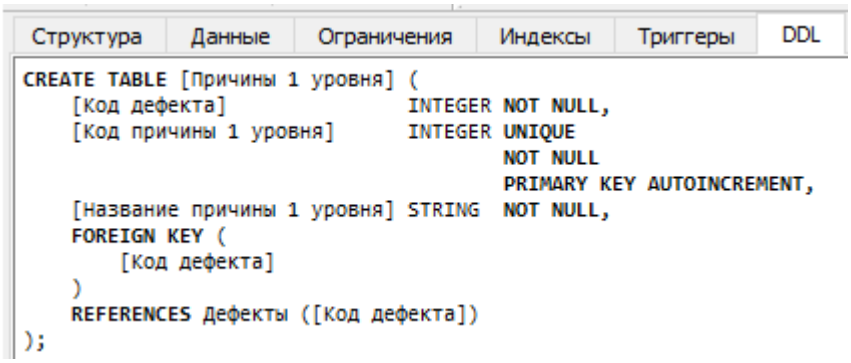


Рисунок 9.29 – Активированная вкладка DDL

Если создается новая БД, то во вкладке Структура в поле Имя таблицы необходимо ввести название таблицы (1). Следующим шагом следует добавить поля, которые будет содержать таблица. Для этого в панели управления таблицей необходимо использовать кнопку (2) Добавить столбец (Ins).

Соответствующие поля и кнопки представлены на рисунке 9.30

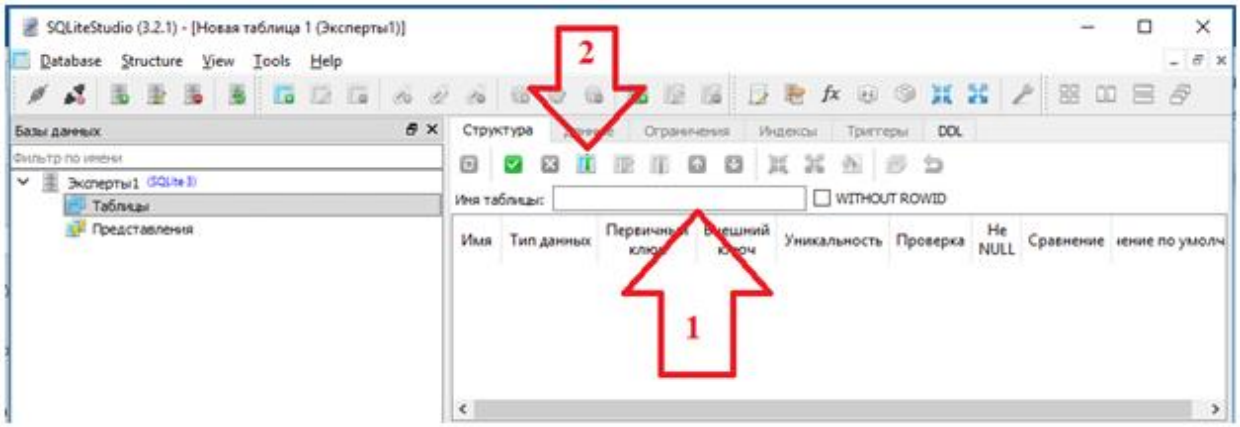


Рисунок 9.30 – Соответствующие поля и кнопки
(ввод имени таблицы и добавление полей)

Если нажать на кнопку Добавить столбец (Ins), откроется окно редактирования столбца – окно Столбец (рассмотрено выше).

Если требуется установить на поле первичный ключ, то напротив соответствующего ограничения требуется нажать кнопку Настроить, после чего откроется окно Редактировать ограничение (рисунок 9.31).

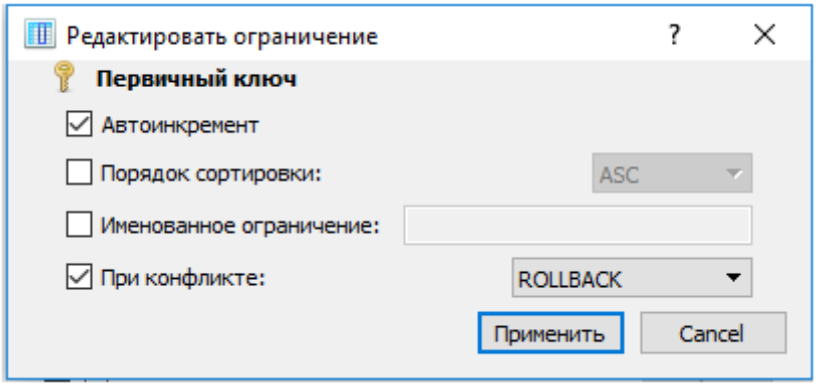


Рисунок 9.31 – Окно Редактировать ограничение (Первичный ключ)

В случае, если в таблице должно быть несколько ключей, то это ограничение необходимо задавать во вкладке Структура, для чего следует нажать кнопку Добавить первичный ключ таблицы .

На рисунке 9.32 показана вкладка Структура и место расположения кнопки Добавить первичный ключ таблицы.

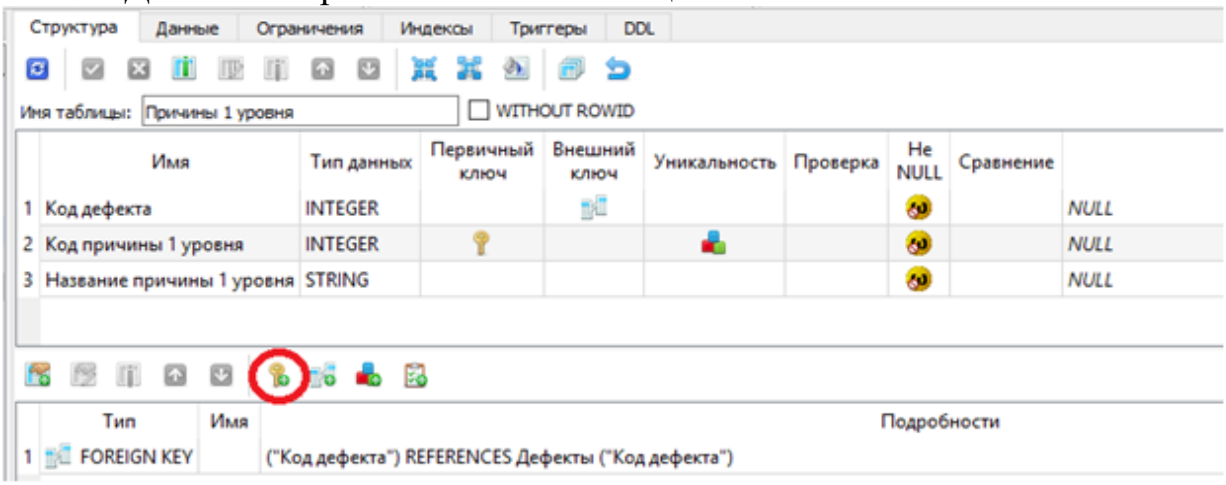


Рисунок 9.32 – Вкладка Структура
и место расположения кнопки Добавить первичный ключ таблицы

После нажатия на кнопку Добавить первичный ключ таблицы в открывшемся окне Редактировать ограничение необходимо выбрать поля, которые должны быть ключевыми (автоинкремент, то есть счетчик, в этом случае поставить нельзя).

На рисунке 9.33 показано окно Редактировать ограничение.

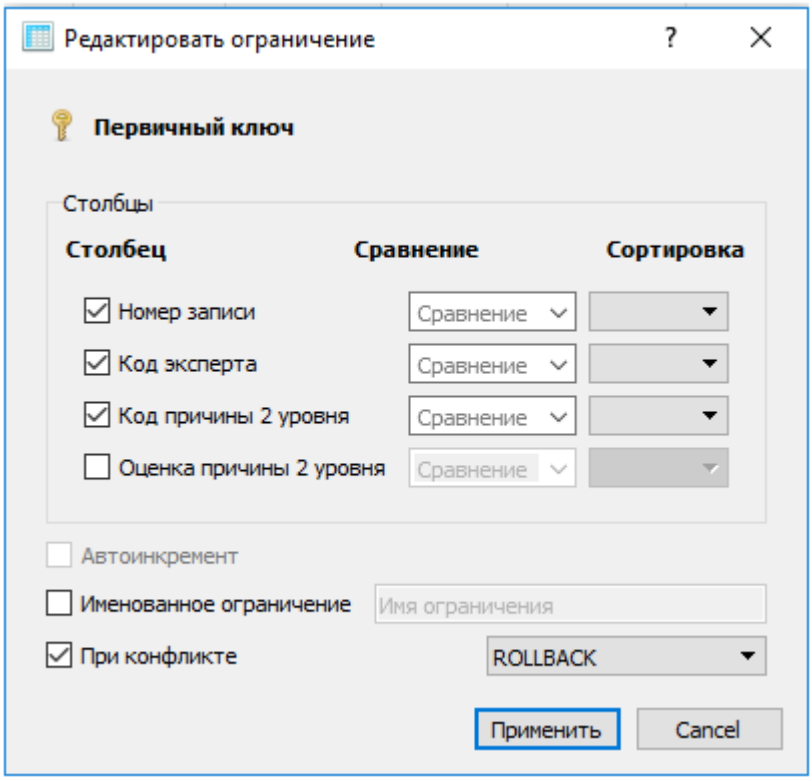


Рисунок 9.33 – Окно Редактировать ограничение

После выбора ограничений требуется подтверждение изменений структуры БД путем нажатия на советующую кнопку .

На рисунке 9.34 показана вкладка Структура, место расположения кнопки Подтверждение изменений структуры и созданный столбец таблицы БД со всеми наложенными на него ограничениями.

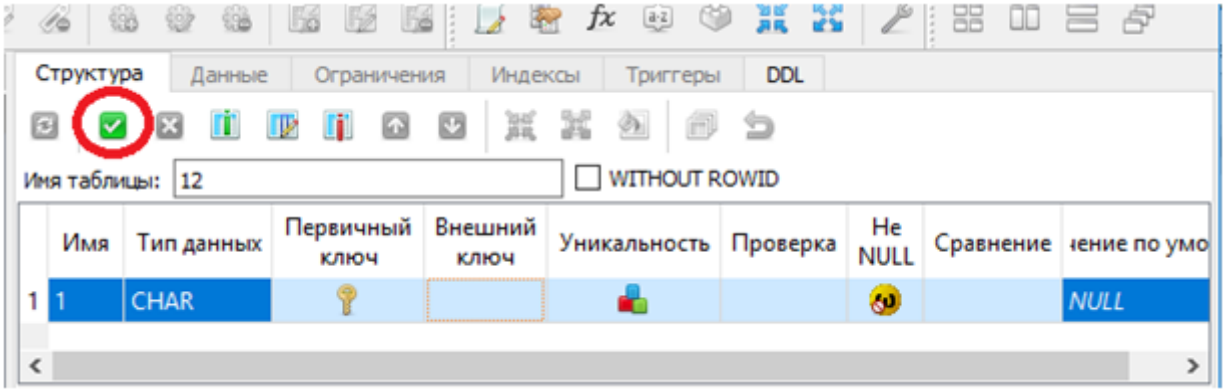


Рисунок 9.34 – Вкладка Структура, место расположения кнопки Подтверждение изменений структуры и созданный столбец таблицы БД с ограничениями

В результате создания нескольких таблиц будет получена структура (рисунок 9.35).

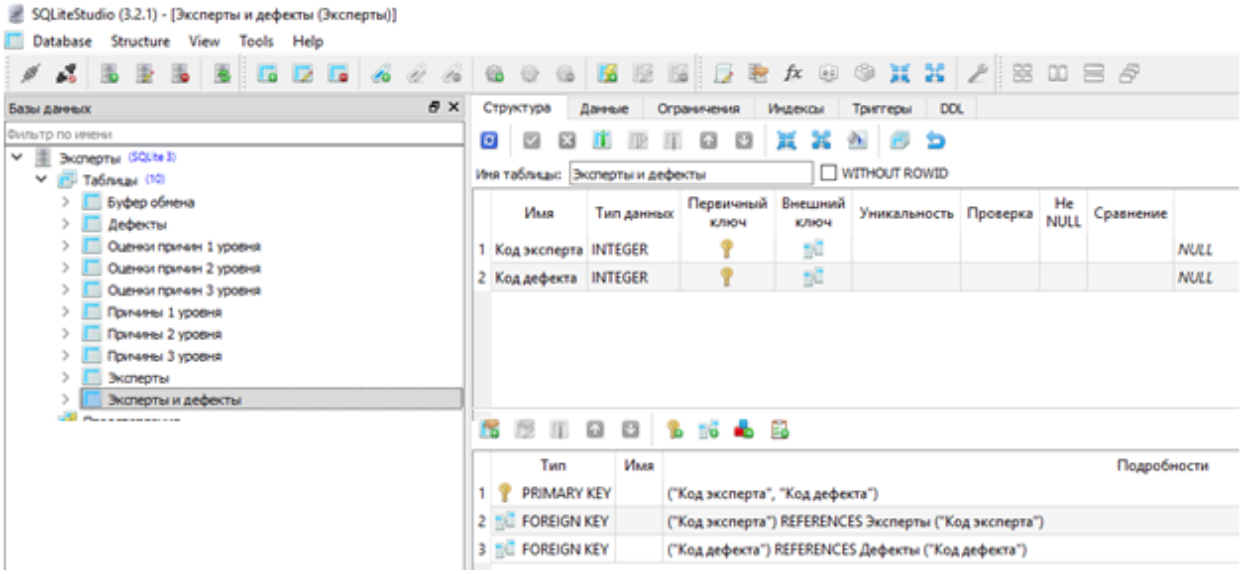


Рисунок 9.35 – Структура таблиц некоторой БД

После создания таблиц следует перейти к установке связей между ними. Для этого в блоке 2 вкладки Структура необходимо нажать кнопку Добавить внешний ключ таблицы (рисунок 9.36).

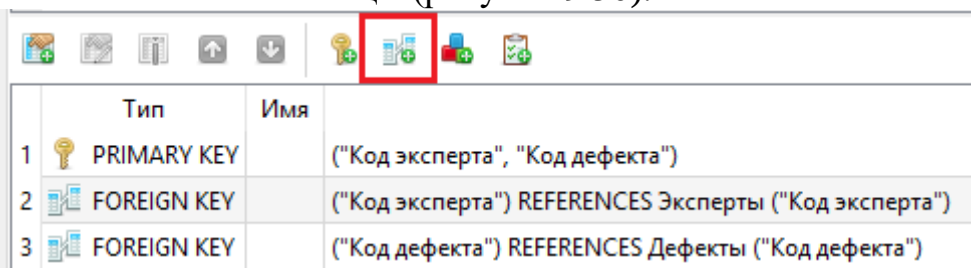


Рисунок 9.36 – Кнопка Добавить внешний ключ таблицы

После этого откроется окно Новое ограничение (рисунок 9.37).

В окне Новое ограничение необходимо:

- заполнить поле «Внешняя таблица» (в выпадающем списке выбрать таблицу с которой требуется установить связь);

- заполнить группу полей Столбцы (указать, по какому столбцу будут связаны текущая и внешняя таблица).

Примечание: при этом в окне навигации Базы данных выделена текущая таблица (на ней стоит указатель). Внешняя таблица выбирается в соответствии с процессом информационного моделирования (в соответствии с инфологической моделью БД).

Остальные группы заполняются по необходимости.

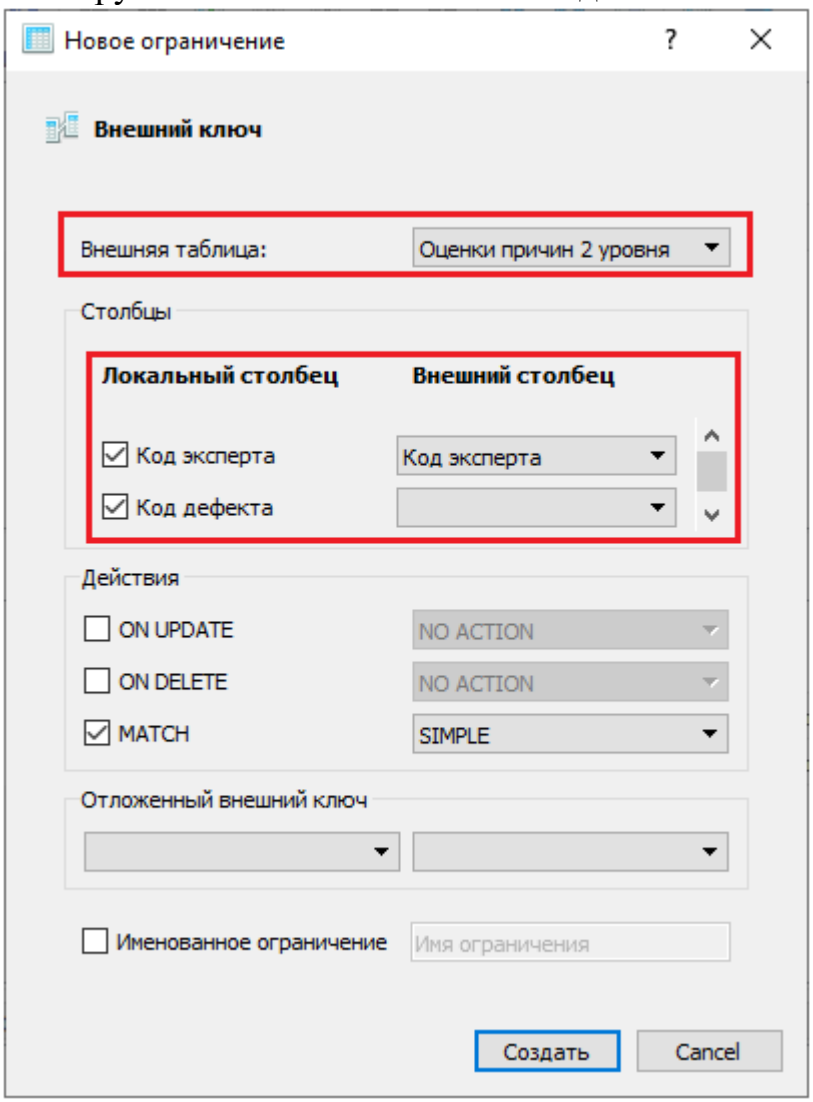


Рисунок 9.37 – Окно Новое ограничение

Примечание: Внешний ключ (FOREIGN KEY) – это ограничение уровня таблицы в реляционных базах данных, в том числе и в системе управления базами данных SQLiteStudio3. Внешние ключи определяют правила, по которым будут связаны таблицы в базах данных формата SQLite.

Помимо того, что внешние ключи определяют то, как будут связаны таблицы в базах данных SQLite, они еще нужны для обеспечения целостности данных в базах данных. В SQLite внешний ключ объявляется при помощи

конструкции FOREIGN KEY, а таблица, на которую ссылается внешний ключ указывается после ключевого слово REFERENCE. Указывается не только таблица, но и столбец, на который идет ссылка. Когда связываются таблицы при помощи внешнего ключа, то одна таблица является родительской, а вторая таблица является дочерней. Внешний ключ всегда ссылается на родительскую таблицу, другими словами конструкция FOREIGN KEY и REFERENCE указывается в дочерней таблице. Столбец, который ссылается и столбец, на который ссылается, должны иметь одинаковый тип данных.

В результате в блоке 2 должна появиться информация, как показано на рисунке 9.38.

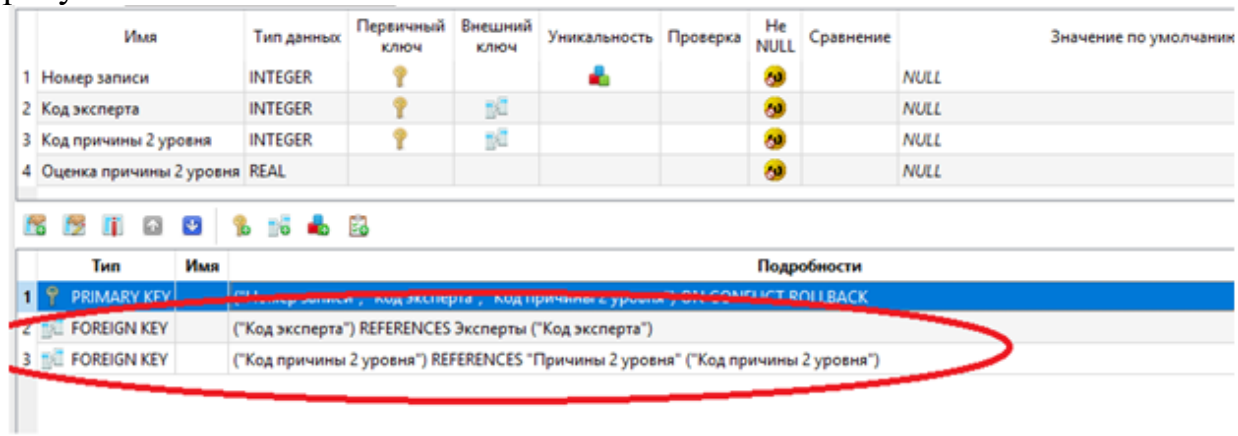


Рисунок 9.38 – Информация относительно связей между таблицами

Подобным образом устанавливаются все необходимые связи между таблицами.

После установки связей между таблицами, необходимо приступить к вводу данных. Это происходит во вкладке Данные.

Процесс внесения данных в таблицы был описан выше.

На рисунке 9.39 показаны записи (данные), внесенные в одну из таблиц ранее представленной структуры БД (заполнена таблица Оценки причин второго уровня).

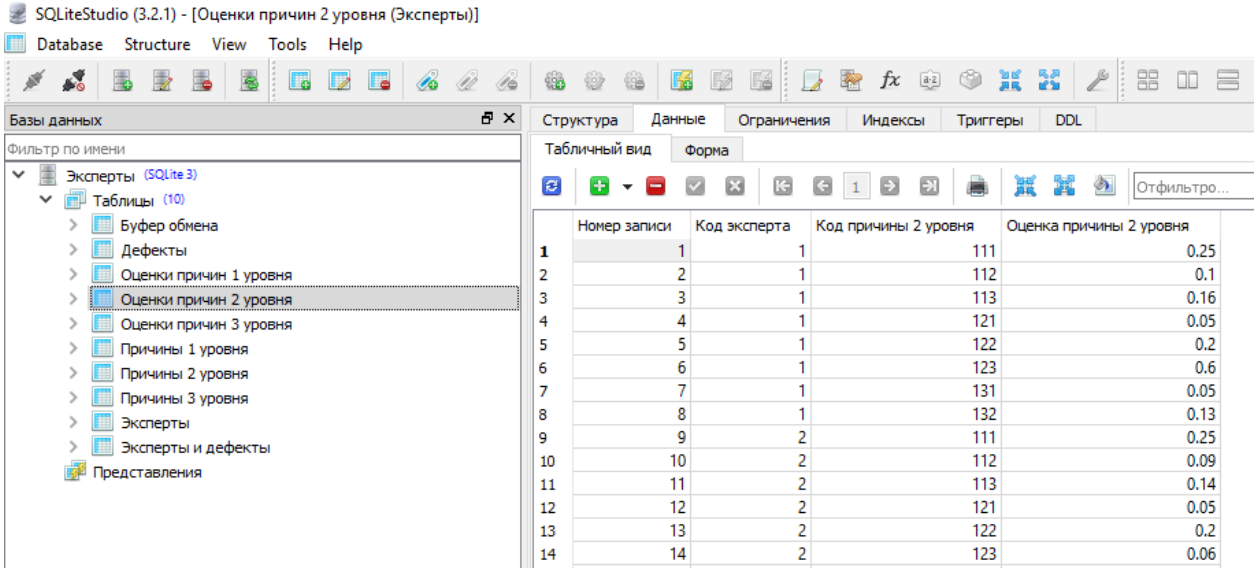


Рисунок 9. 39 – Заполненная таблица Оценки причин второго уровня

На рисунке 9.40 показаны записи (данные), внесенные в одну из таблиц ранее представленной структуры БД (заполнена таблица Причины второго уровня).

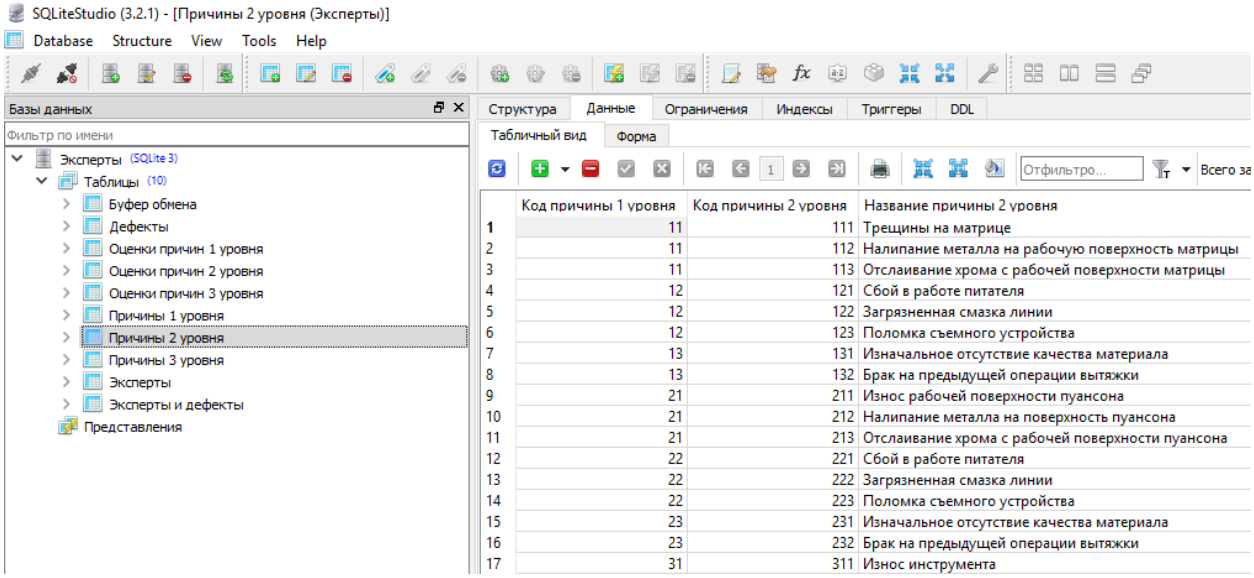


Рисунок 9.40 – Заполненная таблица Причины второго уровня

В случае необходимости можно переносить данные из одной таблицы в другую (как правило из разных баз данных).

Для этого необходимо произвести экспорт данных таблицы в файл формата *.csv.

Экспорт таблицы производится путем выбора соответствующего элемента контекстного меню (рисунок 9.41).

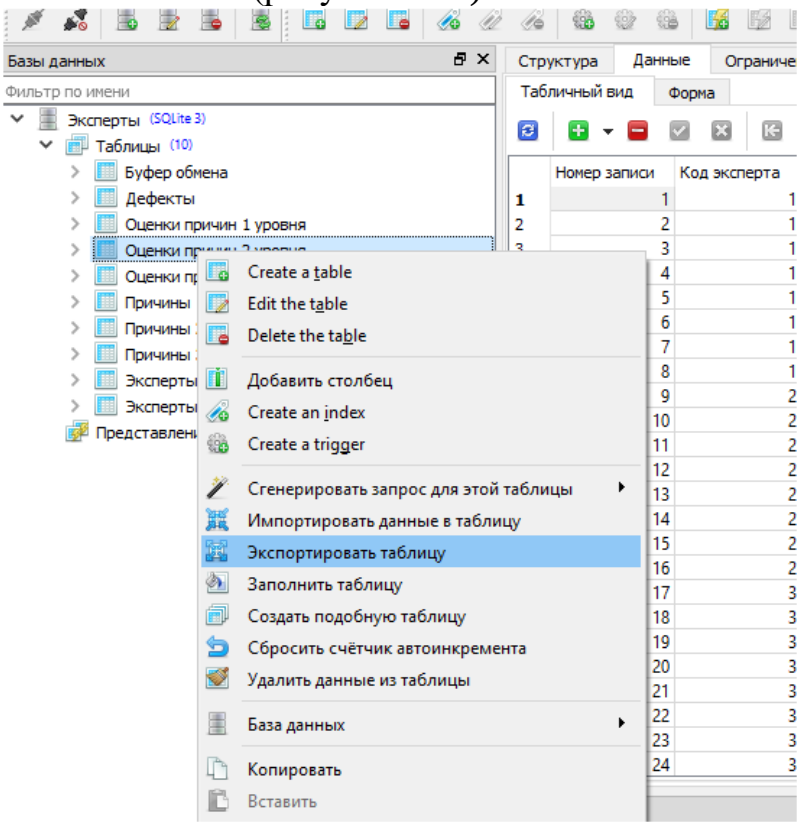


Рисунок 9.41 – Команда Экспортировать таблицу контекстного меню

После выбора команды Экспортировать таблицу откроется окно Экспорт (Экспортируемая таблица) (рисунок 9.42).

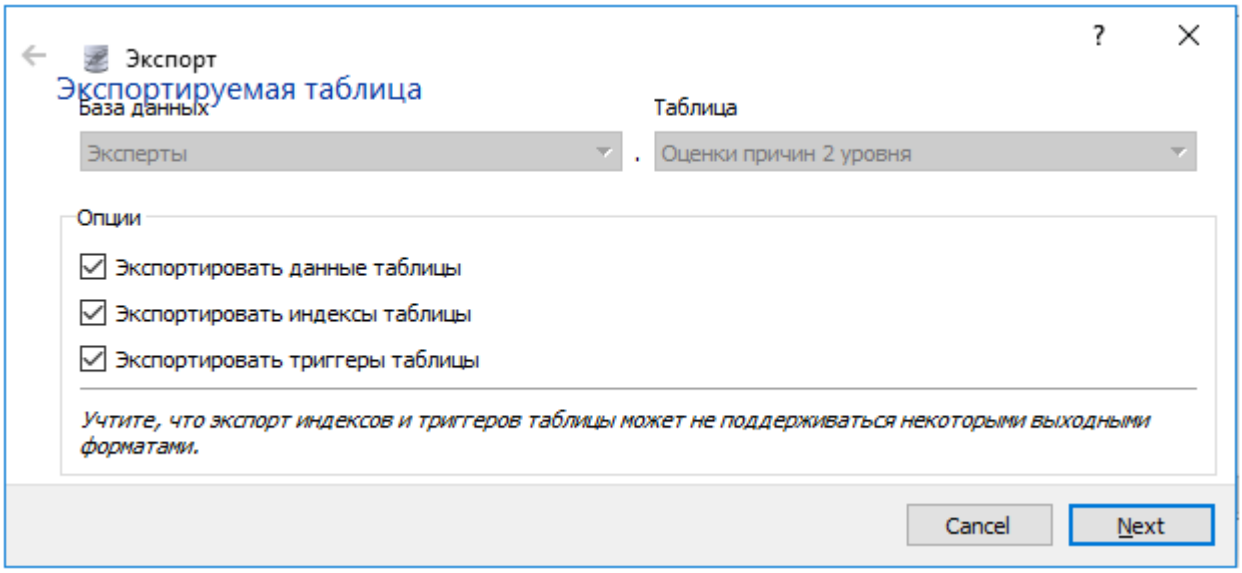


Рисунок 9.42 – Окно Экспорт (Экспортируемая таблица)

В окне Экспорт (Экспортируемая таблица) следует указать, что из таблицы будет экспортироваться (указать Опции):

- экспортировать данные таблицы;
- экспортировать индексы таблицы;
- экспортировать триггеры таблицы.

Примечание 1: индексы таблицы SQLite – это важный и полезный инструмент, который позволяет оптимизировать выборку из базы данных, значительно сокращая время на получение нужных данных. При этом заметить разницу можно на очень больших таблицах, содержащих десятки и сотни тысяч строк. Если таблицы БД не имеют индекса полей, то при запросах на выборку будут перебираться все строки подряд, пока не будет найдено искомое значение. Таким образом, индексы в SQLite (также как и в MySQL) – это определенная сортировка данных в таблице для ускорения поиска данных. Сортировка происходит на «низком» (машинном) уровне, для сортировки достаточно указать, какие поля нужно индексировать. Индексы, прежде всего, нужно создавать по тем полям, которые часто попадают в условия (чаще всего в условие «where») sql-запросов.

Примечание 2: триггеры таблицы SQLite – это хранимая процедура, которая не вызывается непосредственно, а исполняется при наступлении определенного события (вставка, удаление, обновление строки). Иными словами, триггер – это правило, которое помещается разработчиком БД в таблицу, и при выполнении DELETE, UPDATE или INSERT совершает дополнительные полезные для работы БД действия.

После указания опций относительно экспорта, следует нажать на кнопку Next. Окно Экспорт обновится, откроется окно Формат экспорта и опции (рисунок 9.43).

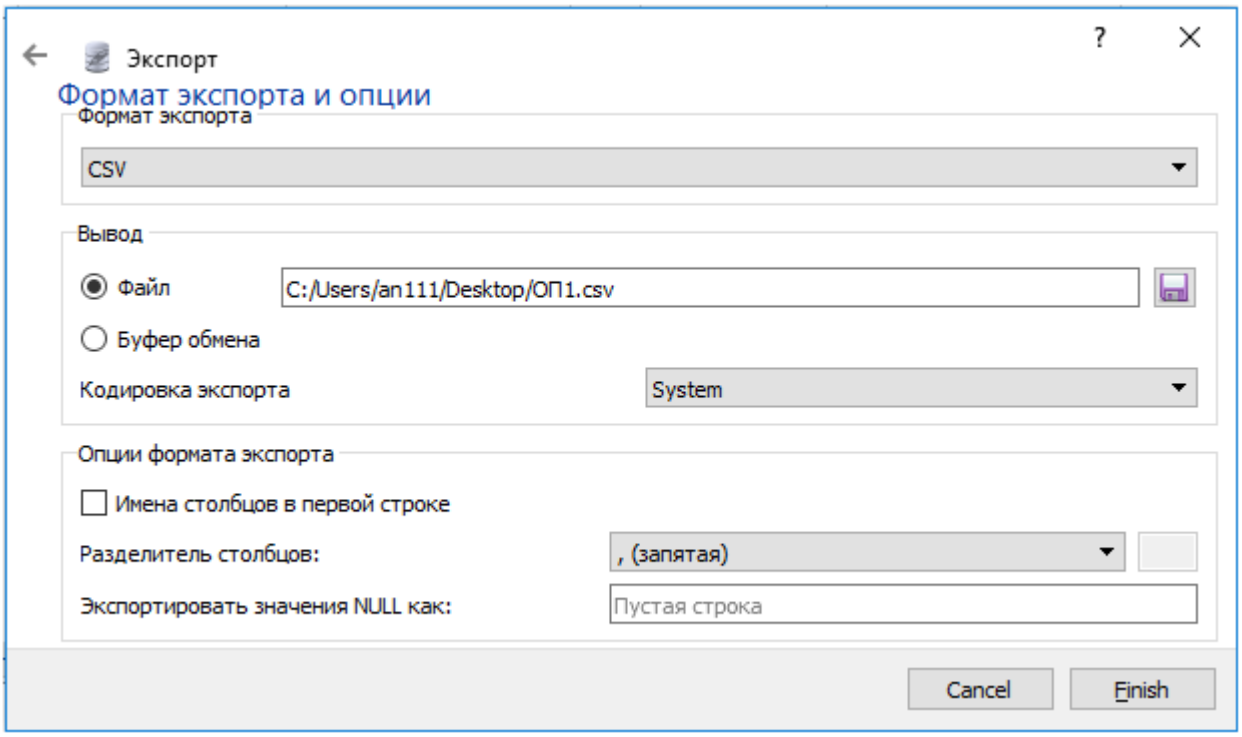


Рисунок 9.43 – Окно Экспорт, Формат экспорта и опции

В окне Экспорт, Формат экспорта и опции указывается:

- формат экспорта (указан выше);
- вывод или путь расположения файла с экспортированными данными;
- некоторые опции формата экспорта.

После завершения экспорта в блоке Статус должно появиться сообщение, как показано на рисунке 9.44.

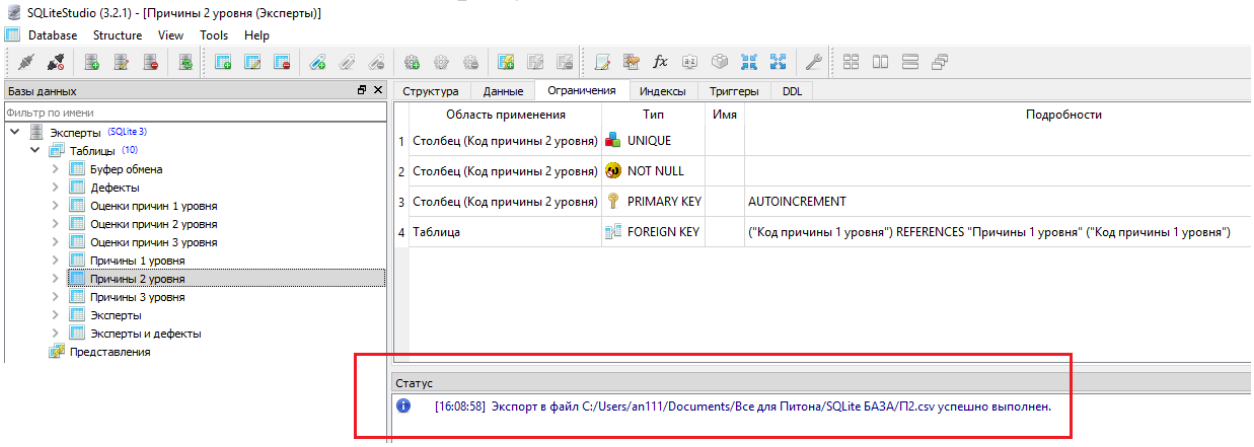


Рисунок 9.44 – Информационное сообщение в блоке Статус

Данные в таблицы можно импортировать. Импорт данных в таблицу производится путем выбора соответствующего элемента контекстного меню (рисунок 9.45).

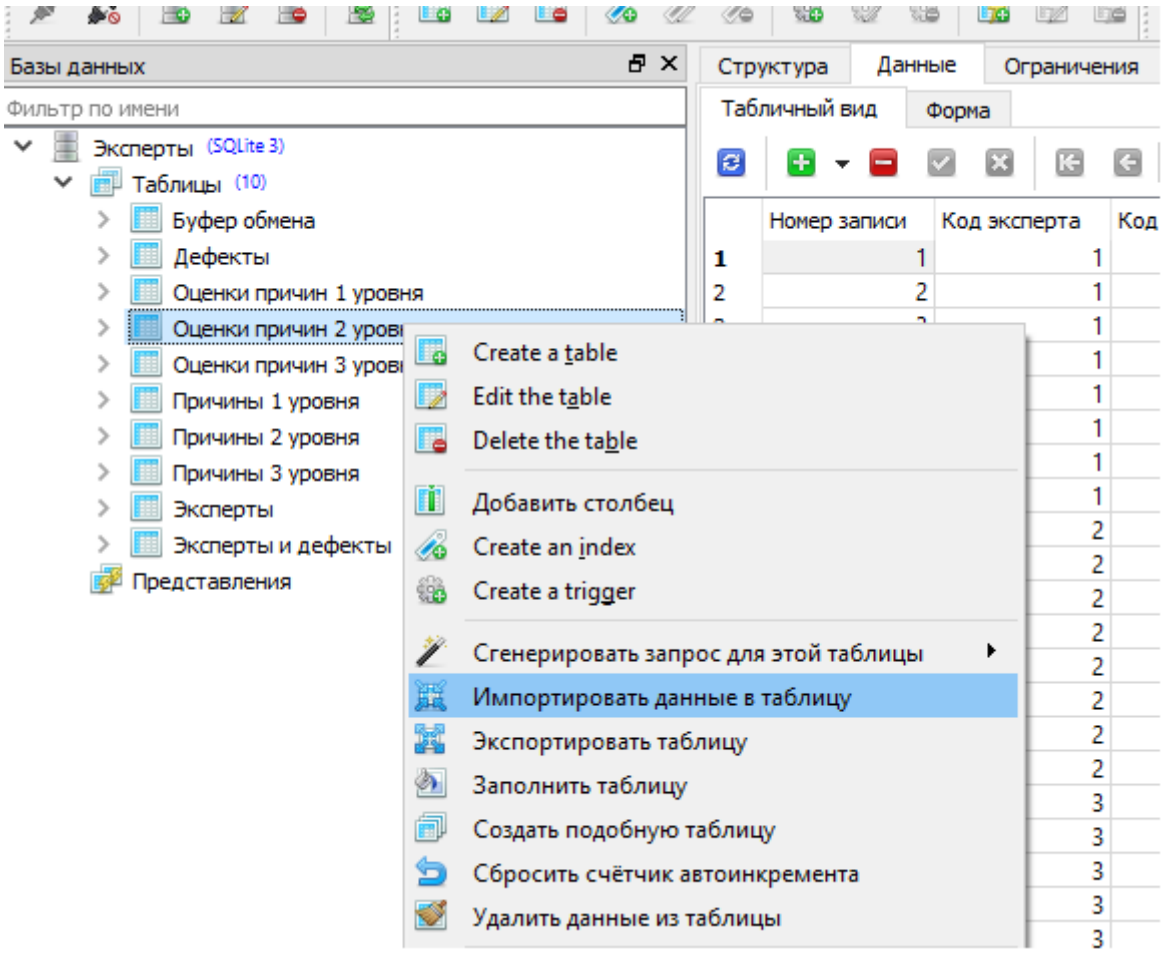


Рисунок 9.45 – Команда контекстного меню Импортировать данные в таблицу

После выбора команды контекстного меню Импортировать данные в таблицу, открывается окно Импорт данных, Таблица для импорта (рисунок 9.46).

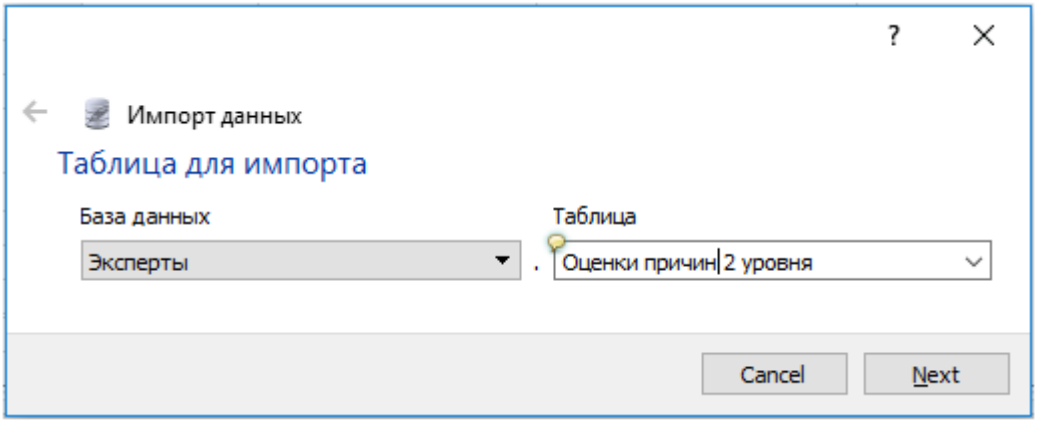


Рисунок 9.46 – Окно Импорт данных, Таблица для импорта

В окне Импорт данных, Таблица для импорта данных выбирается база и таблица, в которую следует импортировать данные, а также файл формата *.csv, из которого планируется сделать импорт.

Для корректного размещения данных поля импортируемых объектов должны полностью совпадать (название, тип данных, расположение

относительно друг друга, количество), иными словами поля таблиц, из которой осуществляется импорт и в которую реализуются импорт, совпадают.

После нажатия на кнопку Next окно Импорт данных, Таблица для импорта данных обновится. Откроется окно Импорт данных, Источник данных для импорта (рисунок 9.47).

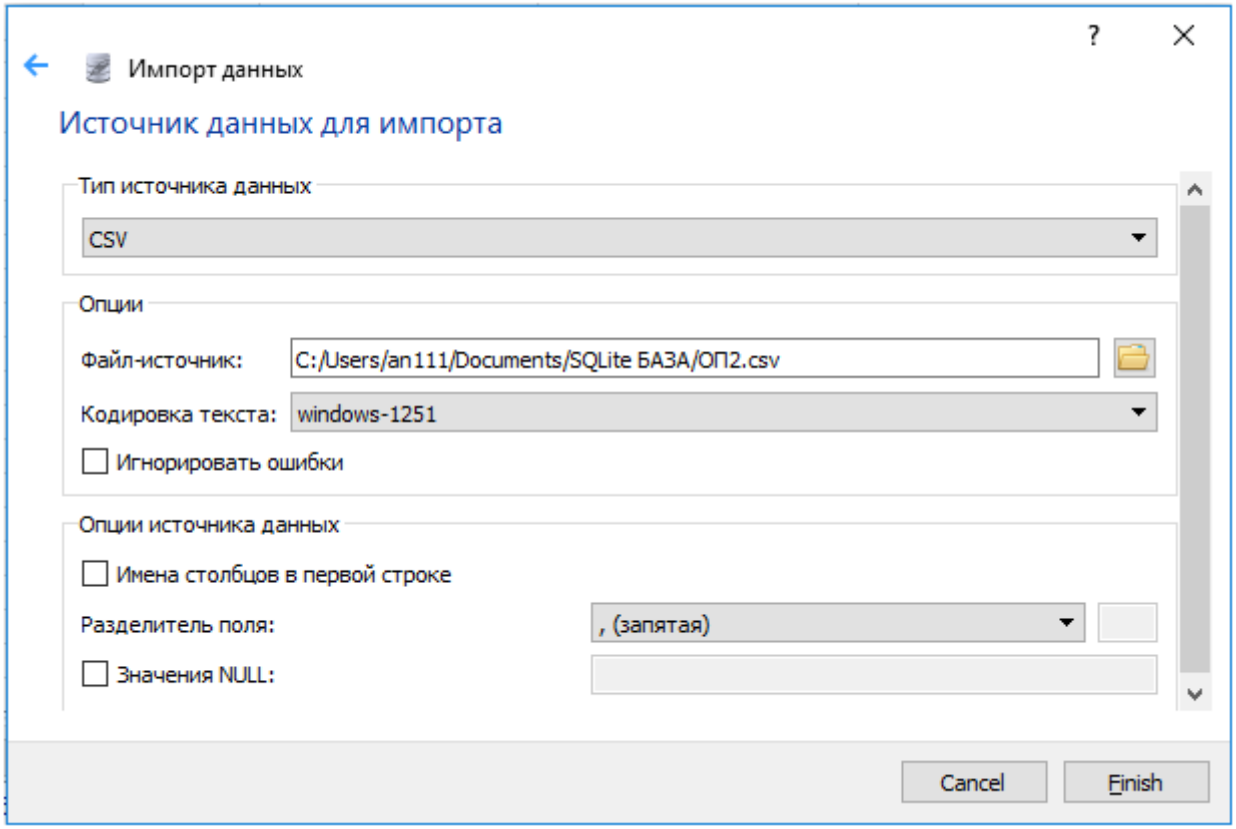


Рисунок 9.47 – Окно Импорт данных, Источник данных для импорта

В окне Импорт данных, Источник данных для импорта указывается:

- тип источника данных (формат файла);
- Опции, а именно файл-источник (его путь) и кодировка текста;
- Опции источника данных.

Примечание: Для адекватного отображения данных в таблице надо выбирать кодировку Windows-1251 (при работе с операционной системой Windows).

Запросы к таблицам формируются путем выбора команды контекстного меню Сгенерировать запрос для этой таблицы. Предусмотрены следующие виды запросов:

- SELECT;
- INSERT;
- UPDATE;
- DELETE.

На рисунке 9.48 показана команда контекстного меню Сгенерировать запрос для этой таблицы и виды запросов, предусмотренные в SQLiteStudio.

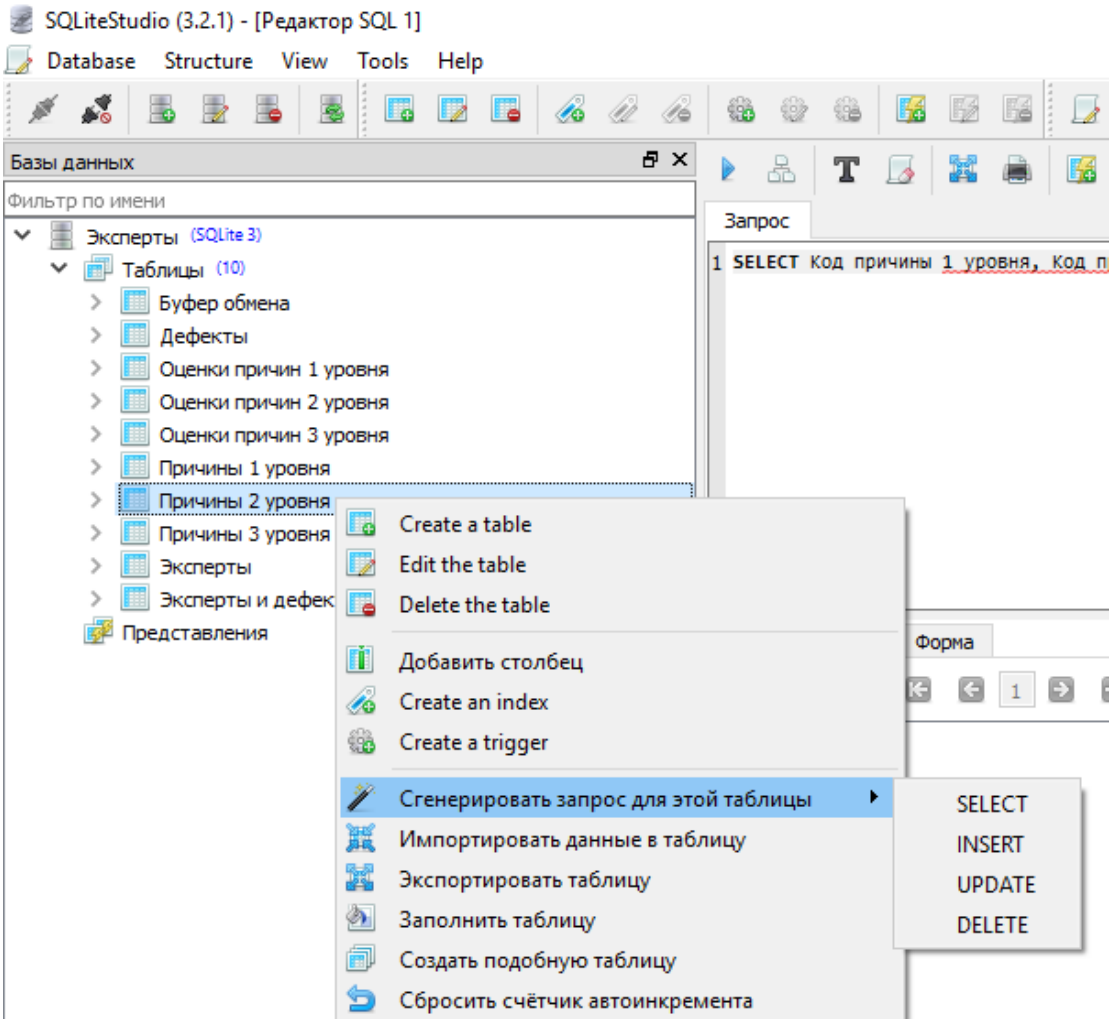


Рисунок 9.48 – Команда контекстного меню

Сгенерировать запрос для этой таблицы и виды запросов, предусмотренные в SQLiteStudio

9.3. Оборудование и программное обеспечение

1. Персональная ЭВМ.
2. Операционная система Windows/Linux.
3. SQLiteStudio (3.2.1) или иная версия SQLiteStudio.
4. Среда программирования IDLE или другая среда, содержащая язык программирования Python.
5. MS Office, Libre Office (Open Office).

9.4. Задание на работу

1. Изучить теоретические положения лабораторной работы.
2. Получить результаты работы по созданию РБД в соответствии с примером 16.
3. Сгенерировать 5 различных запросов к РБД Знания экспертов.
4. Открыть созданную РБД через собственную СУБД, разработанную в лабораторной работе №8.
5. Отобразить в отдельном окне СУБД результат выполнения произвольных запросов к БД.
6. В SQLiteStudio создать РБД в соответствии с вариантом самостоятельной работы (вариант представлен ниже).

Примечание: для некоторых студентов предполагаются дополнительные варианты.

7. В РБД, созданной в соответствии с вариантом, сгенерировать несколько различных произвольных запросов; открыть РБД через собственную СУБД.
8. Показать результаты работы преподавателю.
9. Ответить на контрольные вопросы.
10. Оформить отчет.

9.5. Контрольные вопросы

1. Что такое инфологическая модель БД?
2. Что такое каноническая модель БД?
3. Что такое ключевое поле БД?
4. Каким образом в SQLiteStudio создаются таблицы?
5. Каким образом в SQLiteStudio заполняются таблицы данными?
6. Что такое ограничения (относительно понятия база данных)?
7. Что такое индексы таблиц?
8. Что такое триггеры таблиц?
9. Как установить ключевые поля в таблиц в SQLiteStudio?
10. Каким образом в SQLiteStudio создаются связи между таблицами?
11. Каким образом импортируются/экспортируются данные?
12. Какие виды запросов предусмотрены в SQLiteStudio?

9.6. Правила оформления отчета

Отчет должен содержать:

1. Номер, название и цель лабораторной работы.
2. Описание процесса выполнения примеров и заданий.
3. Вывод по работе.

9.7. Варианты самостоятельной работы

Вариант 1

На рисунке 9.51а представлена схема данных реляционной базы Аптека.

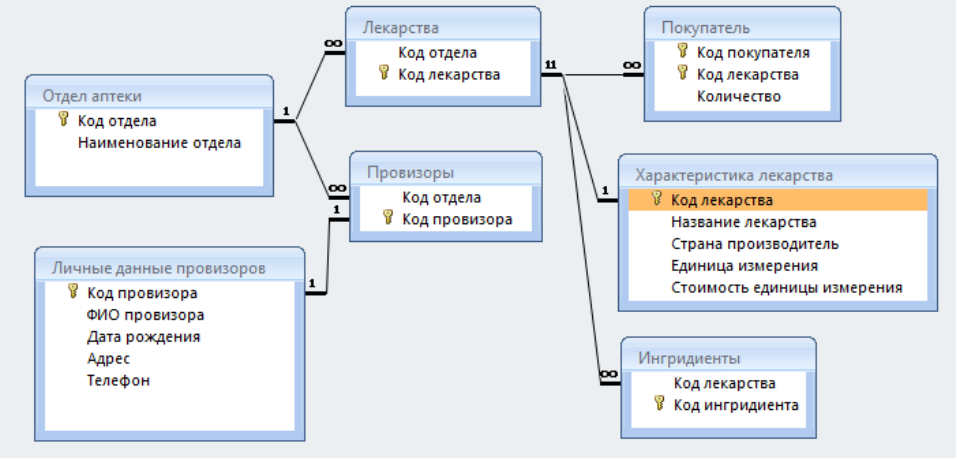


Рисунок 51а – Схема данных реляционной базы Аптека

На рисунке 9.51б представлена каноническая модель базы данных Аптека.

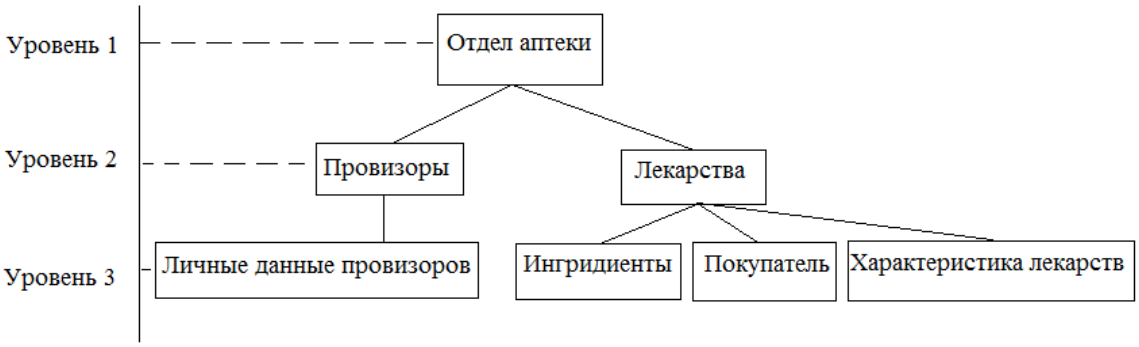


Рисунок 51б – Каноническая модель базы данных Аптека

Вариант 2

На рисунке 52а представлена схема данных реляционной базы Театр.

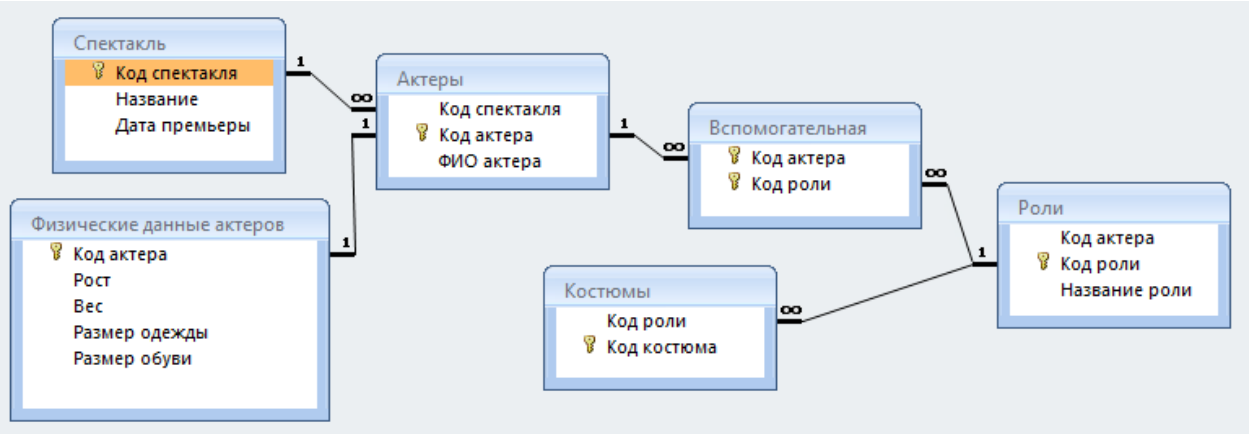


Рисунок 52а – Схема данных реляционной базы Театр

На рисунке 52б показана каноническая модель базы данных Театр.

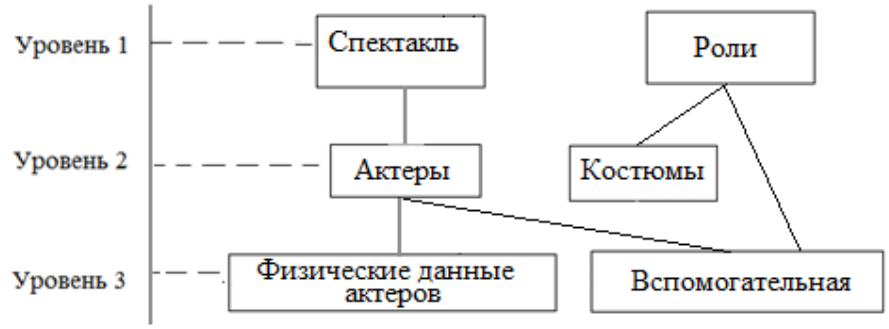


Рисунок 52б – Каноническая модель базы данных Театр

Вариант 3

На рисунке 53а представлена схема данных реляционной базы Торговая фирма.

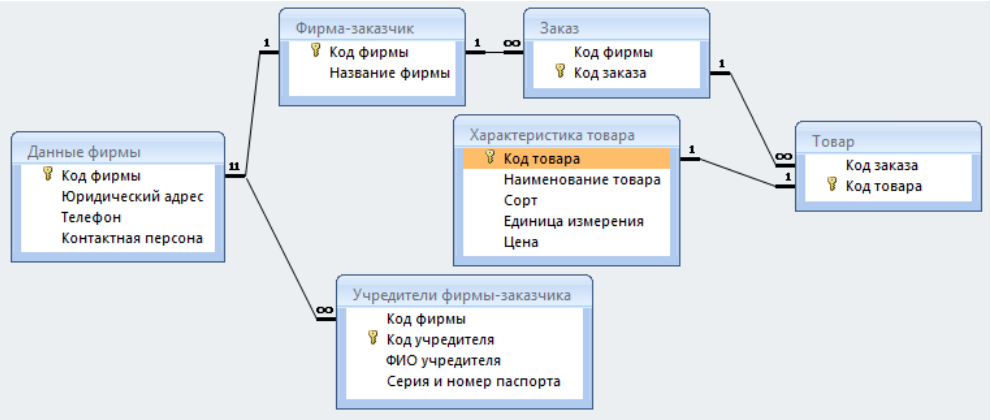


Рисунок 53а – Схема данных реляционной базы Торговая фирма

На рисунке 53б показана каноническая модель базы данных Торговая фирма.

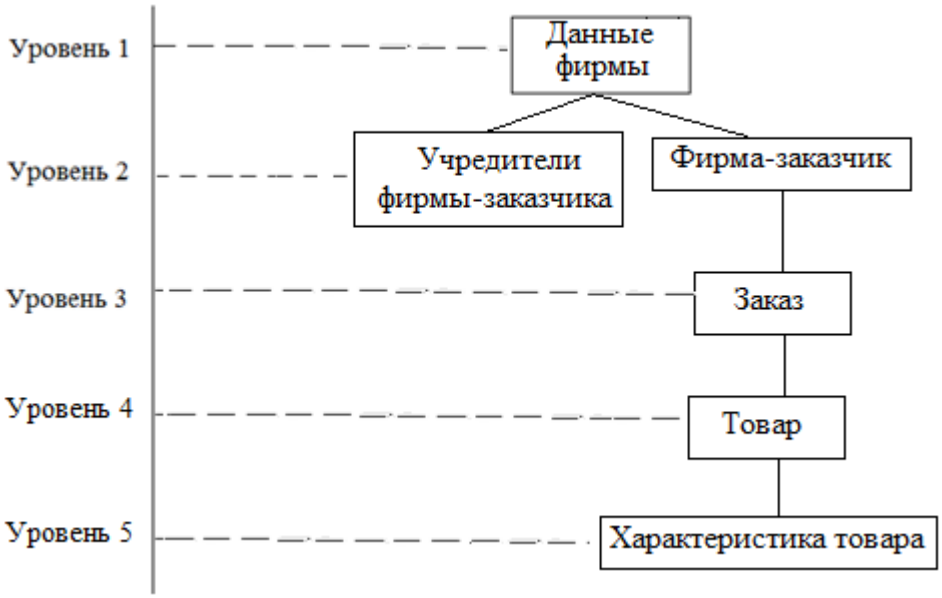


Рисунок 53б – Каноническая модель базы данных Торговая фирма

Вариант 4

На рисунке 54а представлена схема данных реляционной базы Архив.

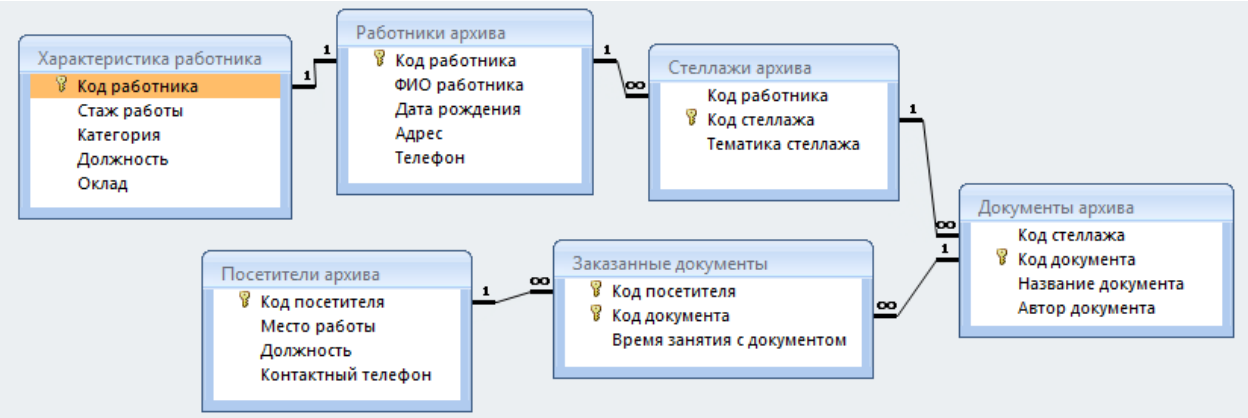


Рисунок 54а – Схема данных реляционной базы Архив

На рисунке 54б представлена каноническая модель реляционной базы Архив.

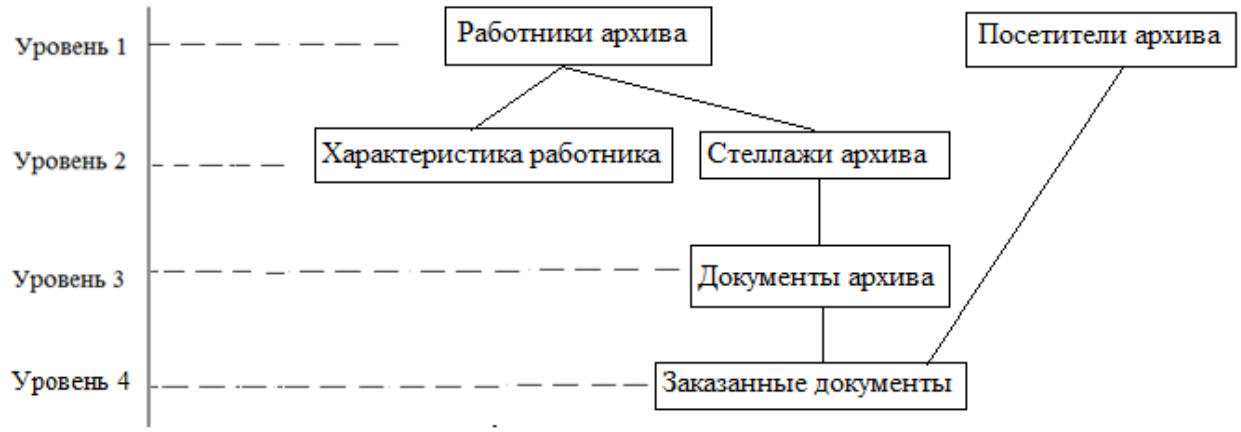


Рисунок 54б – Каноническая модель реляционной базы Архив

Вариант 5

На рисунке 55а представлена схема данных реляционной базы Парикмахерская.

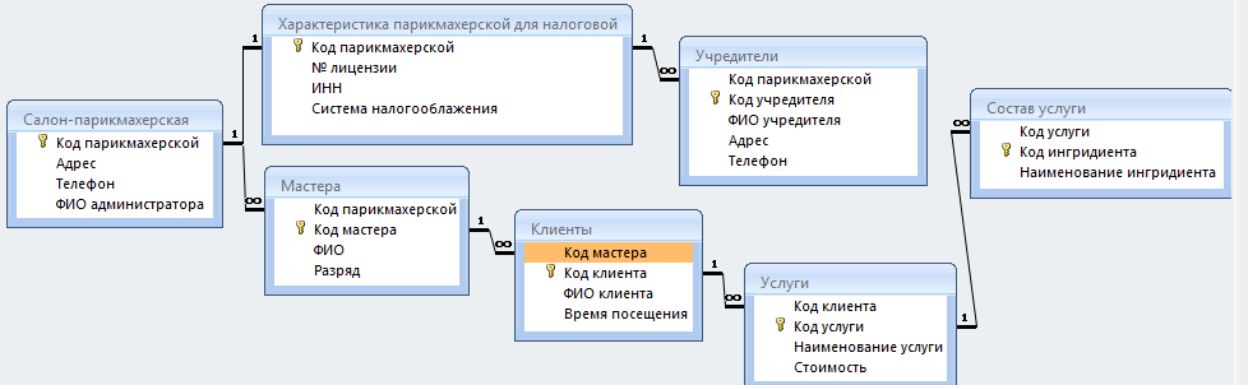


Рисунок 55а – Схема данных реляционной базы Парикмахерская

На рисунке 55б представлена каноническая модель реляционной базы Парикмахерская.

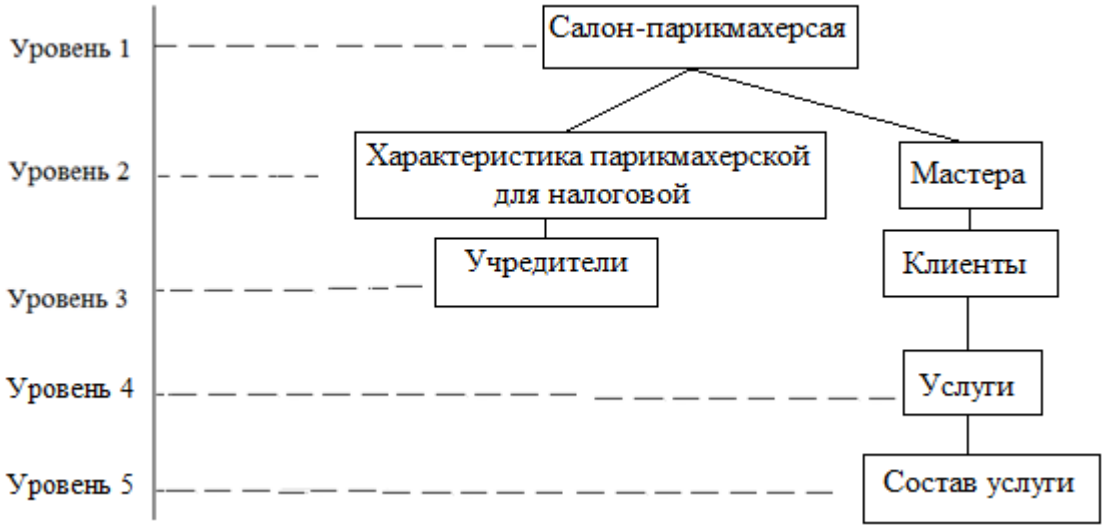


Рисунок 55б – Каноническая модель реляционной базы Парикмахерская

Дополнительные варианты

Вариант 1

Разработать инфологическую и каноническую модель РБД в соответствии с предметной областью Отдел кадров предприятия и реализовать РБД с соответствующими предметной области запросами в SQLiteStudio.

Вариант 2

Разработать инфологическую и каноническую модель РБД в соответствии с предметной областью Коммерческий банк и реализовать РБД с соответствующими предметной области запросами в SQLiteStudio.

Вариант 3

Разработать инфологическую и каноническую модель РБД в соответствии с предметной областью Производственная фирма и реализовать РБД с соответствующими предметной области запросами в SQLiteStudio.

Вариант 4

Разработать инфологическую и каноническую модель РБД в соответствии с предметной областью Деканат ВУЗа и реализовать РБД с соответствующими предметной области запросами в SQLiteStudio.

Вариант 5

Разработать инфологическую и каноническую модель РБД в соответствии с предметной областью Управляющая компания и реализовать РБД с соответствующими предметной области запросами в SQLiteStudio.

Вариант 6

Разработать инфологическую и каноническую модель РБД в соответствии с предметной областью Администрация города и реализовать РБД с соответствующими предметной области запросами в SQLiteStudio.

Лабораторная работа №10
Шифрование в Python
10.1. Цель лабораторной работы
Научиться шифровать в Python
10.2 Теоретические положения
ЦЕЛЬ И ЗАДАЧА РАБОТЫ

Получение практических навыков криптографического преобразования информации с использованием блочных алгоритмов симметричного шифрования типа ГОСТ 28147-89.

ЗАДАНИЕ НА РАБОТУ

1. Разработать функции, реализующие базовые циклы зашифрования/расшифрования, аналогичные циклам 32-З и 32-Р ГОСТ 28147-89.

2. Разработать программы шифрования/расшифрования одного полного блока данных с использованием созданных функций в соответствии с вариантом задания: размер блока: 32; размер ключа: 256; число основных шагов: 64.

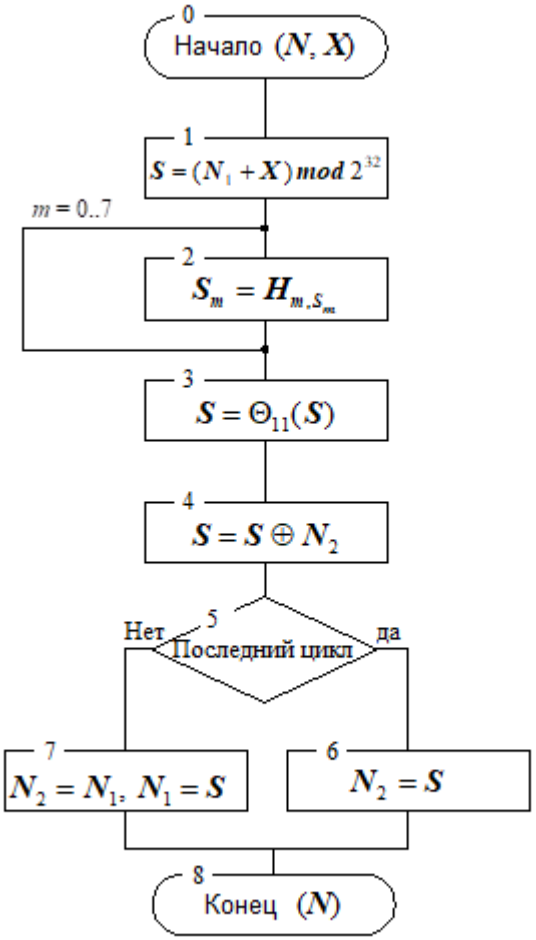


Рис. 1. Схема основного шага криптопреобразования алгоритма ГОСТ 28147-89.

Шаг 1. Сложение с ключом. Младшая половина преобразуемого блока складывается по модулю 232 с используемым на шаге элементом ключа, результат передается на следующий шаг;

Шаг 2. Поблочная замена. 32-битовое значение, полученное на предыдущем шаге, интерпретируется как массив из восьми 4-битовых блоков кода: S=(S0,S1,S2,S3,S4,S5,S6,S7).

Далее значение каждого из восьми блоков заменяется на новое, которое выбирается по таблице замен следующим образом: значение блока Si заменяется на Si-тый по порядку элемент (нумерация с нуля) i-того узла замен (т.е. i-той строки таблицы замен, нумерация также с нуля). Другими словами, в качестве замены для значения блока выбирается элемент из таблицы замен с номером строки, равным номеру заменяемого блока, и номером столбца, равным значению заменяемого блока как 4-битового целого неотрицательного числа.

Шаг 3. Циклический сдвиг на 11 бит влево. Результат предыдущего шага сдвигается циклически на 11 бит в сторону старших разрядов и передается на следующий шаг. На схеме алгоритма символом Θ11 обозначена функция циклического сдвига своего аргумента на 11 бит в сторону старших разрядов.

Шаг 4. Побитовое сложение: значение, полученное на шаге 3, побитно складывается по модулю 2 со старшей половиной преобразуемого блока.

Шаг 5. Обмен: младшая часть преобразуемого блока сдвигается на место старшей, а на ее место помещается результат выполнения предыдущего шага.

Шаг 6. Полученное значение преобразуемого блока возвращается как результат выполнения алгоритма основного шага криптопреобразования.

СХЕМА АЛГОРИТМА

Данная программа позволяет зашифровывать и расшифровывать файлы. Для шифровки и расшифровки используется ГОСТ 28147-89.

На рисунках 1 и 2 приведена схема алгоритма программы.

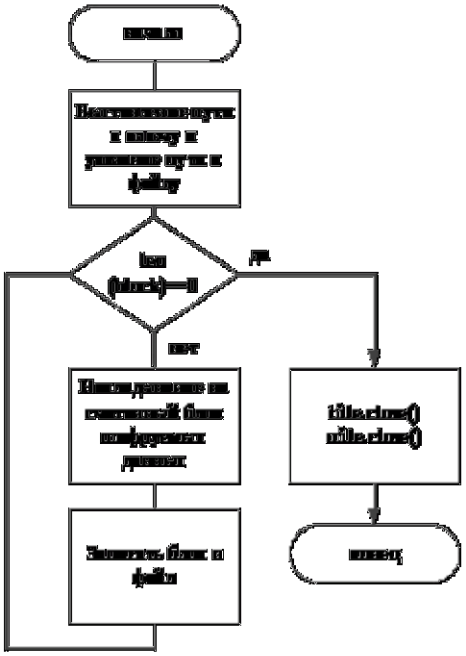


Рисунок 1 – Схема алгоритма шифрования файла

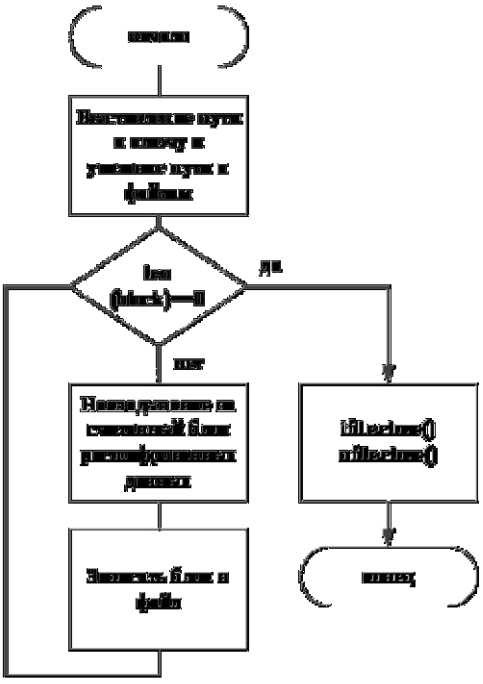


Рисунок 2 – Схема алгоритма расшифрования файла

ТЕКСТ ПРОГРАММЫ

Ниже представлен текст программы на языке Python, выполняющей шифрование по ГОСТ 28147-89.

```
class Replace_table(object):  # класс таблицы замен
    __table__ = []  # таблица замен
    __kp__ = -1  # key parts последней генерации
    __pm__ = -1  # максимальное число последней генерации

    def get(key_parts, part_max):
        if (len(Replace_table.__table__) == 0) \
                or (key_parts != Replace_table.__kp__) \
                or (part_max != Replace_table.__pm__):
            for i in range(0, key_parts):   #Очистка таблицы
                row = []
                for j in range(0, part_max):   #Заполнение
таблицы
                    row.append((i + j) % part_max)
                Replace_table.__table__.append(row)
            Replace_table.__kp__ = key_parts
            Replace_table.__pm__ = part_max
        return Replace_table.__table__


def split_num(n, bsize, count):
    '''
    функция побитового разбиения числа
    :param n: число
    :param bsize: размер числа в битах
    :param count: количество блоков
    :return:
    '''
    res = []
    for i in range(0, count):
        n1 = n >> int(bsize / count)
        n0 = n - (n1 << int(bsize / count))
        res.append(n0)
        n = n1
    return res


def concat_num(n, bsize):
    '''
    функция побитовой сборки числа
    :param n: массив чисел
    :param bsize: размер числа в битах
    :return: число
    '''
    count = len(n)
    res = 0
    b = int(bsize / count)
    n = n[::-1]
    for i in range(0, count):
        res = res << b
        res += n[i]
    return res
```

```
def btint(bytes):
    '''
    преобразование массива байт в число
    :param bytes: массив байт
    :return:
    '''
    return int.from_bytes(bytes, byteorder='big', signed=False)


def itbytes(n, bsize):
    '''
    преобразование числа в массив байт
    :param n: число
    :param bsize: размер массива
    :return: массив байт
    '''
    return int.to_bytes(n, bsize, byteorder='big', signed=False)


def mv_left(n, mv, size):
    '''
    циклическй сдвиг влево
    :param n: число
    :param mv: сдвиг
    :param size: размер числа в битах
    :return:
    '''
    n1 = n >> (size - mv)
    n0 = n - (n1 << (size - mv))
    return n1 + (n0 << mv)


def step(n, x, bsize, mv=11):
    '''
    основной шаг криптопреобразования алгоритма
    :param n: шифруемый блок
    :param x: часть ключа
    :param bsize: размер блока в битах
    :param mv: смещение в п.3
    :return: зашифрованный блок
    '''
    half_size = bsize >> 1  # размер части ключа
    n = split_num(n, bsize, 2)
    s = (n[0] + x) % (2 ** half_size)  # шаг 1 младшая половина
преобразуемого блока складывается по модулю с используемым на
шаге элементом ключа
    key_parts = 8
    part_max = int(2 ** (half_size / key_parts))
    s = split_num(s, half_size, key_parts)
    h = Replace_table.get(key_parts, part_max)
    for m in range(0, 8):  # шаг 2 замена элементов с таблицы
замен
```

```
        s[m] = h[m][s[m]]
    s = concat_num(s, half_size)

    mv_left(s, mv, half_size)  # шаг 3 циклический сдвиг влево
на 11 бит

    s = s ^ n[1]  # шаг 4 побитовое сложение предыдущего шага
    n[1] = n[0]  # шаг 5 сдвиг по цепочке
    n[0] = s
    n = concat_num(n, bsize)
    return n # шаг 6 Полученное значение преобразуемого блока
возвращается как результат выполнения алгоритма основного шага
криптопреобразования.


def encrypt_64(n, k, bsize):
    '''
    цикл зашифрования
    :param n: блок данных
    :param k: ключ
    :param bsize: размер блока
    :return: зашифрованный блок
    '''
    k = split_num(k, bsize, 2)
    for i in range(0, 24):
        for j in range(0, len(k)):
            n = step(n, k[j], bsize)
    for i in range(0, 8):
        for j in range(0, len(k))[::-1]:
            n = step(n, k[j], bsize)

    n = mv_left(n, bsize >> 1, bsize)
    return n


def xor(ifile, ofile, s, key, bsize):
    '''
    Шифрование
    :param ifile: входной файл
    :param ofile: выходной файл
    :param s: синхропосылка
    :param key: ключ
    :param bsize: размер блока
    '''
    half_size = bsize >> 1
    c = split_num(s, bsize, 2)
    s = encrypt_64(s, key, bsize)  # начальное преобразование
синхронпосылки

    while True:
        block = ifile.read(bsize >> 3)
        if len(block) == 0:
            break
```

```
        t = btint(block)

        ofilename1 = 'data/sync.key.out'
        ofile1 = open(ofilename1, 'w')
        s1 = itbytes(s, bsize >> 3)
        ofile1.write(str(s1))
        ofile1.close()

        t = t ^ encrypt_64(s, key, bsize) # Зашифрования
очередного блока
        ofile.write(itbytes(t, bsize >> 3))

key_filename = './data/key.key'
keyfile = open(key_filename, 'rb')
key_bytes = keyfile.read()
keyfile.close()
key = btint(key_bytes)

sync_filename = './data/sync.key'
syncfile = open(sync_filename, 'rb')
sync_bytes = syncfile.read()
syncfile.close()
sync = btint(sync_bytes)

block_size_bytes = 4
block_size = block_size_bytes * 8

ifilename =''
ofilename = ''
ifilename = input('Введите имя файла: ')
if ifilename.find('.lock') >= 0:
    ofilename = input('Введите имя выходного файла: ')
else:
    ofilename = ifilename + '.lock'


ifile = open(ifilename, 'rb')
ofile = open(ofilename, 'wb')
xor(ifile, ofile, sync, key, block_size)
ifile.close()
ofile.close()
```

ТЕСТОВЫЙ ПРИМЕР

На рисунках 3-6 представлен результат работы программы.



Рисунок 3 – Содержимое исходного файла

```
========== RESTART: L:\Учеба\учеба наши дни\ИБ\LABS\IB_4\aproj\4.py ==========
Введите имя файла: data\ship_it.png
>>>
```

Рисунок 4 – Шифрование файла

```
========== RESTART: L:\Учеба\учеба наши дни\ИБ\LABS\IB_4\aproj\4.py ==========
Введите имя файла: data\ship_it.png.lock
Введите имя выходного файла: data\recode_shipit.png
>>>
```

Рисунок 5 – Расшифровка файла



Рисунок 6 – Содержимое расшифрованного файла

ВЫВОД

В ходе выполнения данной лабораторной работы были получены практические навыки криптографического кодирования информации с использованием блочных алгоритмов симметричного шифрования по ГОСТ 28147-89, написана программа, реализующая данный алгоритм.